

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

**«Белорусский государственный университет информатики и  
радиоэлектроники»**

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

## **ОТЧЕТ**

по лабораторной работе №2

по дисциплине

**«Современные технологии программирования масштабируемых  
приложений»**

**«Депозитные операции с физическими лицами»**

### **Выполнил:**

Лебедев Артем Владимирович  
магистрант группы 556301

### **Преподаватель:**

Хмелёва Ангелина Владимировна  
кандидат технических наук  
доцент кафедры ПОИТ

Минск, 2026 г.

# СОДЕРЖАНИЕ

|     |                                                             |    |
|-----|-------------------------------------------------------------|----|
| 1   | Цель работы .....                                           | 1  |
| 2   | Задание .....                                               | 2  |
| 3   | Проектирование .....                                        | 3  |
| 3.1 | Декомпозиция на микросервисы .....                          | 4  |
| 3.2 | План счетов и номер счёта .....                             | 4  |
| 3.3 | Схема данных .....                                          | 5  |
| 3.4 | Схема проводок .....                                        | 6  |
| 3.5 | Расчёт процентов .....                                      | 7  |
| 3.6 | Заккрытие банковского дня .....                             | 7  |
| 3.7 | Локализация в распределённой системе .....                  | 8  |
| 3.8 | REST-интерфейсы .....                                       | 9  |
| 4   | Реализация .....                                            | 10 |
| 4.1 | Структура проекта .....                                     | 11 |
| 4.2 | Главная книга .....                                         | 11 |
| 4.3 | Заключение депозитного договора .....                       | 12 |
| 4.4 | Контроль корректности данных договора .....                 | 13 |
| 4.5 | Обработка банковского дня .....                             | 14 |
| 5   | Тестирование .....                                          | 14 |
| 6   | Результаты работы .....                                     | 16 |
|     | Заключение .....                                            | 27 |
|     | Список использованных источников .....                      | 28 |
|     | Приложение А Исходный код сервисов счетов и депозитов ..... | 29 |

## **1 ЦЕЛЬ РАБОТЫ**

Реализовать микросервис депозитных операций с физическими лицами: справочник «План счетов», форму заключения депозитного договора с контролем вводимых данных, открытие счетов клиента по правилам банковского учёта, проводки по схеме «Депозитная программа», счёт фонда развития банка и процедуру «Закрытие банковского дня» с отчётом о состоянии счетов. Данные о клиенте берутся из модуля «Клиенты» первой лабораторной работы.

## 2 ЗАДАНИЕ

Вариант №4 соответствует банку «Решение» в списке банков Республики Беларусь (<https://select.by/banki>). Для него выбраны две депозитные программы для физических лиц (таблица 2.1); условия приближены к действующим предложениям банка. Сам условный банк в учебной системе называется BankEt: программы банка «Решение» служат образцом для его справочника видов депозита.

Таблица 2.1 – Депозитные программы по образцу банка «Решение»

|                     | «Свободное накопление»     | «Online-решение»    |
|---------------------|----------------------------|---------------------|
| Вид вклада          | отзывный, до востребования | срочный безотзывный |
| Выплата процентов   | ежемесячно                 | в конце срока       |
| Досрочный отзыв     | возможен в любой день      | невозможен          |
| Ставка, BYN         | 7,00 % годовых             | 12,90 % годовых     |
| Ставка, USD         | 1,50 % годовых             | 3,00 % годовых      |
| Минимальная сумма   | 100                        | 500                 |
| Срок договора       | 1–12 месяцев               | 3–13 месяцев        |
| Счёт основной суммы | 3404                       | 3414                |
| Счёт процентов      | 3470                       | 3471                |

Требования задания:

1. разработать справочник «План счетов» и внести в него счета, необходимые для обслуживания выбранных депозитных линий;
2. разработать форму заключения депозитного договора (вид депозита и валюта из справочников, номер договора, даты начала и окончания, срок, сумма, процент) с контролем корректности данных;
3. при заключении договора создавать не менее двух счетов — для основной суммы и для процентов — с 13-значным номером, кодом и активностью из плана счетов, дебетом, кредитом, сальдо и названием по ФИО клиента;
4. предусмотреть счёт фонда развития банка (СФРБ) со стартовым капиталом 1 млн белорусских рублей;
5. разработать процедуру «Закрытие банковского дня» и отчёт о состоянии счетов; при активных депозитных программах в нём должно быть не менее шести счетов.

## 3 ПРОЕКТИРОВАНИЕ

### 3.1 Декомпозиция на микросервисы

Депозитные операции затрагивают три разных области ответственности, и каждой соответствует свой микросервис со своей базой данных (рисунок 3.1):

- `client-service` — клиенты банка (лабораторная работа № 1);
- `account-service` — главная книга: план счетов, лицевые счета, проводки, банковский день;
- `deposit-service` — депозитные программы и договоры, расчёт процентов.

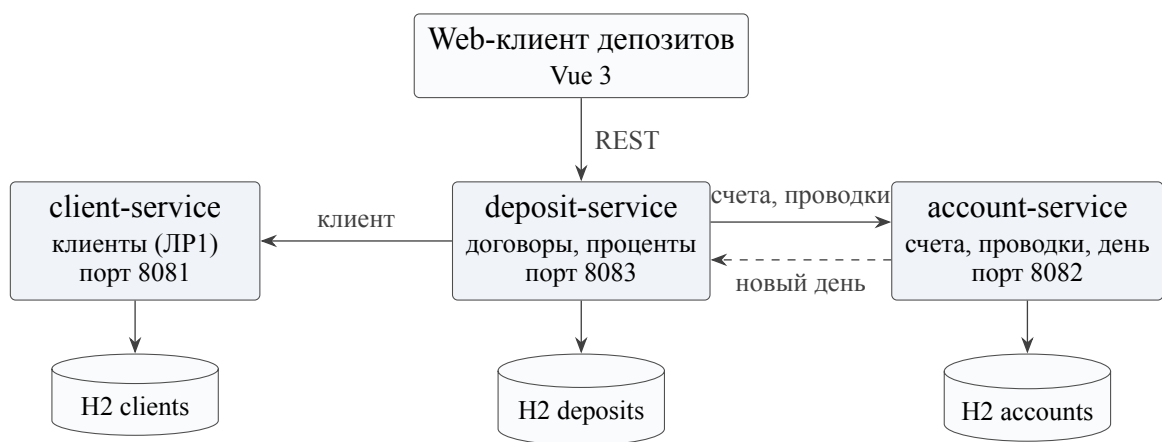


Рисунок 3.1 – Микросервисы депозитного контура и REST-вызовы между ними

Главная книга выделена в отдельный сервис намеренно. Со счетами работают и депозиты, и кредиты следующей лабораторной работы, а правила учёта — расчёт сальдо по активности счёта, запрет отрицательного остатка, неделимость проводки — должны соблюдаться в одном месте независимо от того, какой сервис инициировал операцию. Сервис депозитов не хранит ни оборотов, ни остатков: он знает только номера счетов своего договора и просит главную книгу провести операцию.

Сервисы общаются синхронно по REST. Клиенты смежных сервисов описаны декларативными интерфейсами Spring (@HttpExchange) в общей библиотеке `bank-common`; там же находятся единый формат ошибки, который позволяет передать сообщение главной книги через сервис депозитов до web-клиента без искажений, и локализация — сервисы отвечают на русском, английском или белорусском языке.

### 3.2 План счетов и номер счёта

Справочник «План счетов» (таблица 3.1) хранится в базе сервиса счетов и заполняется при старте. Активность балансового счёта определяет правило

расчёта сальдо: у активного счёта остаток равен дебету минус кредит, у пассивного — кредиту минус дебет.

Таблица 3.1 – Счета плана счетов, используемые депозитными программами

| Код  | Активность | Наименование                                               | Назначение в работе       |
|------|------------|------------------------------------------------------------|---------------------------|
| 1010 | активный   | Денежные средства в кассе                                  | касса банка               |
| 3404 | пассивный  | Вклады (депозиты) до востребования физических лиц          | сумма отзывного вклада    |
| 3414 | пассивный  | Срочные вклады (депозиты) физических лиц                   | сумма срочного вклада     |
| 3470 | пассивный  | Начисленные процентные расходы по вкладам до востребования | проценты отзывного вклада |
| 3471 | пассивный  | Начисленные процентные расходы по срочным вкладам          | проценты срочного вклада  |
| 7327 | пассивный  | Фонд развития банка                                        | СФРБ                      |

Номер лицевого счёта состоит из 13 цифр:

$$\underbrace{b_1 b_2 b_3 b_4}_{\text{балансовый счёт}} \quad \underbrace{c_1 c_2 c_3 c_4 c_5}_{\text{код клиента}} \quad \underbrace{n_1 n_2 n_3}_{\text{номер счёта клиента}} \quad \underbrace{k}_{\text{ключ}}, \quad k = \left( 3 \sum_{i=1}^{12} d_i w_i \right) \bmod 10, \quad (3.1)$$

где  $d_i$  — первые 12 цифр номера,  $w_i$  — веса, циклически повторяющие последовательность 7, 1, 3. Код клиента равен его идентификатору в модуле «Клиенты», у собственных счетов банка он нулевой. Веса 7, 1, 3 и множитель 3 взаимно просты с 10, поэтому ошибка в любой одной цифре номера меняет ключ и обнаруживается при проверке.

### 3.3 Схема данных

Базы сервисов независимы (рисунок 3.2). Договор ссылается на клиента и на счета не внешними ключами, а идентификатором клиента и номерами счетов: целостность таких ссылок обеспечивается проверкой через REST в момент заключения договора.

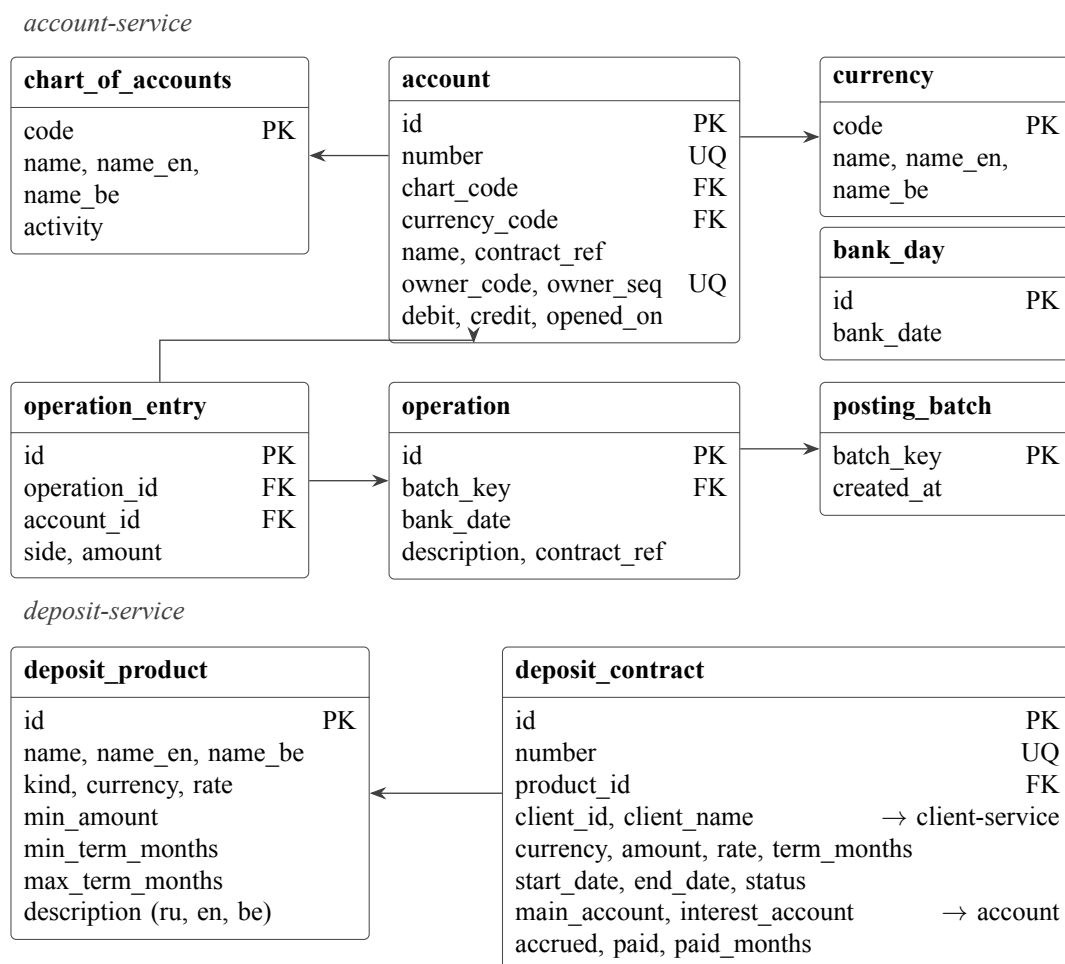


Рисунок 3.2 – Схемы данных сервисов счетов и депозитов

### 3.4 Схема проводок

Проводки депозитной программы выполняются в точности по упрощённой схеме из задания (таблица 3.2). Знак «+» означает увеличение остатка счёта, «–» — уменьшение: для активной кассы остаток растёт по дебету, для трёх пассивных счетов — по кредиту. Касса в схеме работает как транзитный счёт: всё, что внесено, в тот же момент зачисляется на счёт клиента или выдаётся, поэтому её остаток возвращается к нулю.

Таблица 3.2 – Проводки процесса «Депозитная программа»

| № | Операция                              | Касса (А) |    | СФРБ (П) |    | Счёт вклада (П) |    | Проценты (П) |    |
|---|---------------------------------------|-----------|----|----------|----|-----------------|----|--------------|----|
|   |                                       | Дт        | Кт | Дт       | Кт | Дт              | Кт | Дт           | Кт |
| 1 | Внесение денег в кассу                | +         |    |          |    |                 |    |              |    |
| 2 | Перевод денег с кассы на текущий счёт |           | –  |          |    |                 | +  |              |    |
| 3 | Использование денег банком            |           |    |          | +  | –               |    |              |    |
| 4 | Начисление процентов по депозиту      |           |    | –        |    |                 |    |              | +  |
| 5 | Перевод процентов в кассу             | +         |    |          |    |                 |    | –            |    |
| 6 | Вывод процентов из кассы              |           | –  |          |    |                 |    |              |    |
| 7 | Окончание депозита                    |           |    | –        |    |                 | +  |              |    |
| 8 | Перевод депозита в кассу              | +         |    |          |    | –               |    |              |    |
| 9 | Вывод денег из кассы                  |           | –  |          |    |                 |    |              |    |

Операции 1–3 выполняются при заключении договора, 4 — при каждом закрытии дня, 5–6 — в день выплаты процентов, 7–9 — по окончании срока или при досрочном отзыве.

### 3.5 Расчёт процентов

Проценты простые, начисляются за фактическое число дней при 365 днях в году. К дате  $t$  по договору на сумму  $S$  со ставкой  $r$  процентов годовых, заключённому в день  $t_0$ , накоплено

$$I(t) = \text{round}\left(\frac{S \cdot r \cdot (t - t_0)}{100 \cdot 365}, 2\right), \quad (3.2)$$

а проводка очередного дня делается на разность  $I(t)$  и уже начисленной суммы. Такой накопительный расчёт устойчивее суммирования дневных процентов: копейки округления не теряются, а пропущенный или повторно обработанный день не искажает итог. Например, вклад 10 000 BYN под 7 % за 31 день приносит  $10000 \cdot 7 \cdot 31 / 36500 = 59,45$  BYN.

По отзывному вкладу накопленные проценты выплачиваются, когда истекает очередной полный месяц договора; по срочному — только в день окончания, вместе с возвратом суммы вклада.

### 3.6 Закрытие банковского дня

Банковский день хранится в сервисе счетов, но сам этот сервис не знает правил депозитов. Поэтому закрытие дня устроено как оповещение (рисунок 3.3): сервис счетов объявляет участникам дату нового дня, каждый участник начисляет проценты и проводит выплаты через обычный REST главной книги и возвращает протокол сделанного, после чего дата сдвигается.

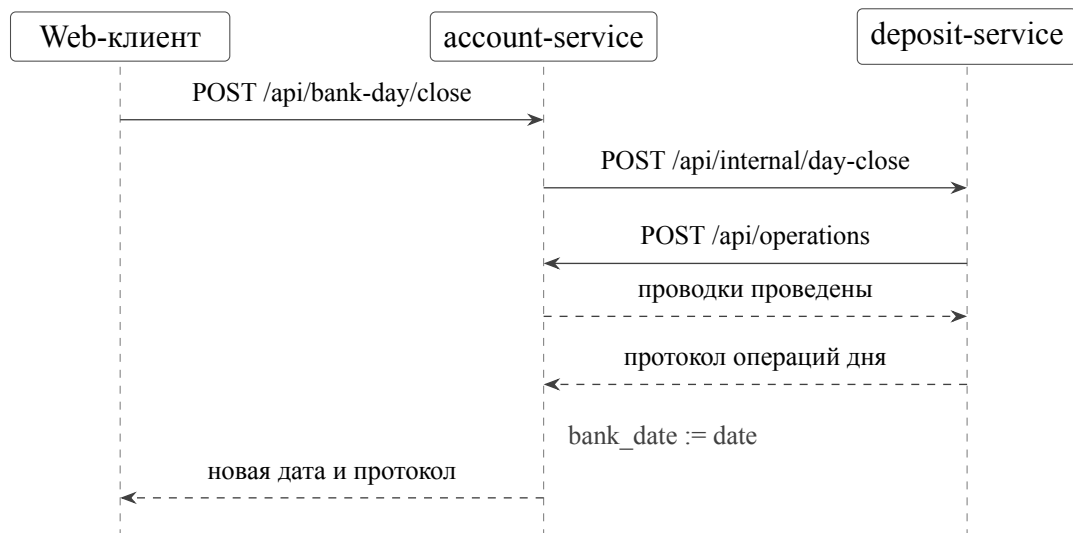


Рисунок 3.3 – Взаимодействие сервисов при закрытии банковского дня

Распределённая процедура должна переживать сбои, поэтому в неё заложены три свойства.

1. **Идемпотентность.** Каждый пакет проводок несёт ключ вида DEP-Д-000001-ACCRUE-2026-10-02. Главная книга запоминает ключи и на повторный запрос возвращает уже проведённые операции, не проводя их второй раз.
2. **Дата сдвигается последней.** Если участник ответил ошибкой, день остаётся открытым и закрытие можно повторить: уже обработанные договоры повторно не начисляются.
3. **Навёрстывание.** Недоступный участник не блокирует закрытие дня. Его расчёт накопительный (формула 3.2), поэтому при следующем закрытии он начислит проценты сразу за все пропущенные дни.

### 3.7 Локализация в распределённой системе

Интерфейс системы доступен на трёх языках, и в системе из нескольких сервисов это ставит два вопроса, которых нет в отдельном приложении.

**Как язык доходит до сервиса, к которому пользователь не обращается напрямую.** Web-клиент передаёт выбранный язык в заголовке Ассерп-Language. Каждый сервис, обращаясь к смежному, повторяет этот заголовок: перехватчик REST-клиента подставляет язык текущего запроса (листинг 3.1). Поэтому отказ главной книги «Недостаточно средств» приходит в форму депозитов на языке пользователя, а протокол закрытия дня, собранный из ответов участников, написан на одном языке.

Листинг 3.1 – Передача языка смежному сервису (bank-common, HttpApis)

```

public static RestClient.Builder withLanguage(RestClient.Builder builder) {
    return builder.requestInterceptor((request, body, execution) -> {
        request.getHeaders().setAcceptLanguageAsLocales(List.of(Messages.locale()));
        return execution.execute(request, body);
    });
}

```

```
}
```

**На каком языке хранить тексты, которые система пишет в базу сама.** Название проводки записывается в журнал один раз, а читают журнал пользователи с разными языками. Поэтому в главной книге хранится не текст, а код вида `@posting.deposit.accrual`, который превращается в текст при чтении (листинг 3.2). Так же хранятся наименования собственных счетов банка: `@account.cash|BYN`. Счета клиентов названы их ФИО — это данные, и они не переводятся. Переводы справочников (план счетов, валюты, виды депозита) хранятся в столбцах `name_en` и `name_be` рядом с русским наименованием.

Листинг 3.2 – Раскрытие хранимого кода на языке читающего (`bank-common, Localized`)

```
public static String code(String key, Object... args) {
    StringBuilder code = new StringBuilder(MARK).append(key);
    for (Object arg : args) {
        code.append(SEPARATOR).append(arg);
    }
    return code.toString();
}

/** Обычный текст (например, ФИО клиента в названии счёта) возвращается как есть. */
public static String render(String text) {
    if (text == null || !text.startsWith(MARK)) {
        return text;
    }
    String[] parts = text.substring(MARK.length()).split("\\\" + SEPARATOR);
    return Messages.get(parts[0], (Object[]) Arrays.copyOfRange(parts, 1, parts.length));
}
```

### 3.8 REST-интерфейсы

Операции двух сервисов приведены в таблице 3.3.

Таблица 3.3 – REST-интерфейсы сервисов счетов и депозитов

| Метод                  | Адрес                                   | Назначение                                              |
|------------------------|-----------------------------------------|---------------------------------------------------------|
| <i>account-service</i> |                                         |                                                         |
| GET                    | /api/chart-of-accounts, /api/currencies | справочники «План счетов» и валют                       |
| GET                    | /api/accounts?contractRef=              | счета договора или клиента                              |
| GET                    | /api/accounts/system                    | счёт кассы или фонда развития в заданной валюте         |
| POST                   | /api/accounts                           | открытие лицевого счёта                                 |
| POST                   | /api/operations                         | атомарный пакет проводок с ключом идемпотентности       |
| GET                    | /api/operations                         | журнал проводок                                         |
| GET                    | /api/report                             | отчёт о состоянии счетов с итогами оборотов             |
| GET                    | /api/bank-day                           | текущий банковский день                                 |
| POST                   | /api/bank-day/close?days=               | закрытие одного или нескольких дней                     |
| <i>deposit-service</i> |                                         |                                                         |
| GET                    | /api/products, /api/meta                | виды депозитов, валюты, банковский день, номер договора |
| GET                    | /api/clients                            | список клиентов из модуля «Клиенты»                     |
| GET                    | /api/deposits, /api/deposits/{id}       | договоры; договор со счетами и проводками               |
| POST                   | /api/deposits                           | заключение договора                                     |
| POST                   | /api/deposits/{id}/withdraw             | досрочный отзыв вклада                                  |
| POST                   | /api/internal/day-close                 | обработка нового банковского дня                        |

## 4 РЕАЛИЗАЦИЯ

### 4.1 Структура проекта

К проекту первой работы добавлены три модуля Maven (листинг 4.1).

Листинг 4.1 – Модули лабораторной работы № 2

|                  |                                                                        |
|------------------|------------------------------------------------------------------------|
| bank-common/     | общая библиотека сервисов                                              |
| api/             | ApiError, BankException, обработчик ошибок REST                        |
| i18n/            | Messages, Localized, I18nConfig – локализация                          |
| client/, ledger/ | REST-клиенты ClientApi и LedgerApi, записи запросов и ответов          |
| account-service/ | главная книга                                                          |
| domain/          | Account, ChartAccount, Operation, AccountNumbers, Activity             |
| service/         | LedgerService, BankDayService, DayCloseNotifier, LedgerInitializer     |
| web/             | LedgerController, BankDayController                                    |
| resources/       | schema.sql, data.sql (план счетов), static/ (отчёт по счетам)          |
| deposit-service/ | депозиты                                                               |
| domain/          | DepositProduct, DepositContract, DepositKind                           |
| service/         | DepositService, DepositPostings, InterestCalculator, DayCloseProcessor |
| web/             | DepositController, ReferenceController, DayCloseController             |
| resources/       | schema.sql, data.sql (программы банка), static/ (форма договора)       |

### 4.2 Главная книга

Операция в главной книге — это набор строк «счёт, сторона, сумма». Сервис принимает не одиночную операцию, а пакет (листинг 4.2): заключение договора состоит из трёх операций схемы, и они должны быть проведены все вместе либо не проведены вовсе. Пакет выполняется в одной транзакции базы данных. Счета выбираются запросом `select ... for update`, чтобы два параллельных пакета не потеряли обновление оборотов. После каждой операции проверяется остаток затронутых счетов: отрицательное сальдо означает нехватку средств, и тогда исключение откатывает весь пакет вместе с записью о ключе.

Листинг 4.2 – Проведение пакета операций (LedgerService.post)

```
@Transactional
public List<OperationView> post(BatchRequest request) {
    if (batches.existsById(request.batchKey())) {           // повтор запроса: операции уже
        проведены
        return operations.findByBatchKeyOrderById(request.batchKey()).stream().map(
            OperationView::from).toList();
    }
    batches.saveAndFlush(new PostingBatch(request.batchKey()));

    LocalDate date = request.bankDate() != null ? request.bankDate() : bankDay.current();
    Map<String, Account> locked = new LinkedHashMap<>();
    List<Operation> posted = new ArrayList<>();
    for (OperationRequest item : request.operations()) {
        Operation operation = new Operation();
        ...
        Set<Account> touched = new LinkedHashSet<>();
        for (EntryRequest entry : item.entries()) {
```

```

        Account account = locked.computeIfAbsent(entry.account(), number -> accounts.
findByNumberForUpdate(number)
        .orElseThrow(() -> BankException.notFound(Messages.get("ledger.
accountNotFound", number))));
        ...
        account.apply(entry.side(), entry.amount());
        operation.addEntry(account, entry.side(), entry.amount());
        touched.add(account);
    }
    touched.forEach(account -> checkNotOverdrawn(account, item.description()));
    posted.add(operations.save(operation));
}
operations.flush();
return posted.stream().map(OperationView::from).toList();
}

```

При старте сервис открывает в каждой валюте кассу и фонд развития банка и зачисляет в фонд стартовый капитал. Размер капитала задаётся параметром `bank.fund.capital`; для этой работы сервис запускается со значением 1 000 000 BYN.

### 4.3 Заключение депозитного договора

Сервис депозитов проверяет условия договора, запрашивает клиента у модуля «Клиенты», открывает в главной книге два счёта и проводит операции 1–3 схемы (листинг 4.3). Балансовые счета берутся из вида вклада: 3404 и 3470 для вклада до востребования, 3414 и 3471 для срочного. Названием счёта служит ФИО клиента.

#### Листинг 4.3 – Открытие счетов и проводки приёма вклада (DepositService.open)

```

ClientInfo client = findClient(request.clientId());

DepositKind kind = product.getKind();
String number = request.number();
String currency = product.getCurrency();
String main = ledger.open(new OpenAccount(kind.getMainChartCode(), currency,
    client.id().intValue(), client.fullName(), number)).number();
String interest = ledger.open(new OpenAccount(kind.getInterestChartCode(), currency,
    client.id().intValue(), client.fullName(), number)).number();
post(key(number, "OPEN"), number, today,
    DepositPostings.open(cash(currency), fund(currency), main, request.amount()));

```

Сами проводки собраны в классе `DepositPostings`, который читается как таблица схемы (листинг 4.4). Название операции передаётся кодом; русский текст «Внесение денег в кассу» и его переводы находятся в общем словаре.

#### Листинг 4.4 – Проводки приёма вклада и выплаты процентов (DepositPostings)

```

public static List<Posting> open(String cash, String fund, String main, BigDecimal amount) {
    return List.of(

```

```

        // внесение денег в кассу
        Posting.of(code("posting.deposit.cashIn"), Entry.debit(cash, amount)),
        // перевод денег с кассы на текущий счёт
        Posting.of(code("posting.deposit.toAccount"), Entry.credit(cash, amount), Entry.
credit(main, amount)),
        // использование денег банком
        Posting.of(code("posting.deposit.toFund"), Entry.debit(main, amount), Entry.credit
(fund, amount)));
    }

    public static Posting accrual(String fund, String interest, BigDecimal amount) {
        // начисление процентов по депозиту
        return Posting.of(code("posting.deposit.accrual"), Entry.debit(fund, amount), Entry.credit
(interest, amount));
    }

    public static List<Posting> payout(String cash, String interest, BigDecimal amount) {
        return List.of(
            // перевод процентов в кассу
            Posting.of(code("posting.deposit.interestToCash"), Entry.debit(interest, amount),
            Entry.debit(cash, amount)),
            // вывод процентов из кассы
            Posting.of(code("posting.deposit.interestOut"), Entry.credit(cash, amount)));
    }
}

```

Открытие счёта в главной книге идемпотентно по паре «договор, балансовый счёт», а пакет проводок — по ключу DEP-<номер>-OPEN. Если сбой произойдёт после обращения к главной книге, но до сохранения договора, повтор запроса с тем же номером договора не создаст ни вторых счетов, ни вторых проводок.

#### 4.4 Контроль корректности данных договора

Как и в первой работе, проверка двойная. Web-клиент проверяет форму до отправки, сервис повторяет проверки и добавляет те, что возможны только на сервере (таблица 4.1). Ставка и дата начала в форме недоступны для редактирования, но сервис всё равно сверяет их: запрос может прийти мимо формы или из устаревшей страницы.

Таблица 4.1 – Проверки формы депозитного договора

| Поле           | Правило                                         | Где проверяется |
|----------------|-------------------------------------------------|-----------------|
| Клиент         | выбран; существует в модуле «Клиенты»           | клиент, сервис  |
| Номер договора | формат Д-000001; не занят                       | клиент, сервис  |
| Вид депозита   | запись справочника; доступен в выбранной валюте | клиент, сервис  |
| Сумма          | денежная; не меньше минимума программы          | клиент, сервис  |
| Срок           | целое число месяцев в пределах программы        | клиент, сервис  |
| Процент        | равен ставке программы                          | сервис          |
| Дата начала    | равна текущему банковскому дню                  | сервис          |
| Дата окончания | равна дате начала плюс срок                     | сервис          |

## 4.5 Обработка банковского дня

Обработка договора на дату нового дня показана в листинге 4.5. Каждый договор обрабатывается в собственной транзакции, а ключи пакетов включают дату, поэтому повторный вызов для того же дня ничего не меняет.

### Листинг 4.5 – Начисление, выплата и закрытие договора (DepositService.processDay)

```
boolean matured = !date.isBefore(contract.getEndDate());
LocalDate upTo = matured ? contract.getEndDate() : date;

BigDecimal target = InterestCalculator.accrued(contract.getAmount(), contract.getRate(),
    contract.getStartDate(), upTo);
BigDecimal delta = target.subtract(contract.getAccrued());
if (delta.signum() > 0) {
    post(key(number, "ACCRUE-" + date), number, date,
        List.of(DepositPostings.accrual(fund(currency), contract.getInterestAccount(),
            delta)));
    contract.setAccrued(target);
}

int months = InterestCalculator.fullMonths(contract.getStartDate(), upTo);
boolean monthlyPayout = contract.getProduct().getKind() == DepositKind.REVOCABLE && months >
    contract.getPaidMonths();
if (matured || monthlyPayout) {
    payInterest(contract, date);
    contract.setPaidMonths(months);
}
if (matured) {
    close(contract, date);
}
```

## 5 ТЕСТИРОВАНИЕ

Сервисы и общая библиотека покрыты 86 тестами (таблица 5.1). Чтобы тестировать сервис депозитов без запуска соседних микросервисов, смежные сервисы заменяются двойниками: клиент модуля «Клиенты» — заглушкой Mockito, а главная книга — классом InMemoryLedger, который реализует тот же интерфейс LedgerApi в памяти и соблюдает те же правила (сальдо по активности, запрет отрицательного остатка, идемпотентность). Это позволяет сверять в тестах обороты счетов с таблицей 3.2.

Таблица 5.1 – Тесты сервисов счетов и депозитов

| Класс тестов                                     | Тестов | Что проверяется                                            |
|--------------------------------------------------|--------|------------------------------------------------------------|
| <i>bank-common</i><br>MessagesTest               | 10     | выбор языка, хранимые коды, передача языка между сервисами |
| <i>account-service</i><br>AccountNumbersTest     | 7      | структура номера, контрольный ключ, сальдо по активности   |
| LedgerServiceIT                                  | 8      | счета, обороты, атомарность пакета, идемпотентность        |
| BankDayIT                                        | 6      | заккрытие дня, недоступный и отказавший участник, REST     |
| LedgerLanguageIT                                 | 4      | отчёт, журнал и сообщения на трёх языках                   |
| <i>deposit-service</i><br>InterestCalculatorTest | 13     | формула процентов, полные месяцы                           |
| DepositFlowIT                                    | 7      | проводки по схеме, выплаты, окончание и отзыв вклада       |
| DepositApiIT                                     | 28     | контроль данных договора, ошибки смежных сервисов          |
| DepositLanguageIT                                | 3      | программы, проверки и протокол дня на трёх языках          |
| Всего                                            | 86     |                                                            |

Пример теста, который проходит весь срок срочного вклада и сверяет обороты со схемой, приведён в листинге 5.1; итог прогона — в листинге 5.2.

Листинг 5.1 – Тест срочного вклада (DepositFlowIT)

```
@Test
@DisplayName("Срочный вклад: проценты выплачиваются в конце срока вместе с возвратом суммы")
void irrevocableDepositPaysAtMaturity() {
    DepositView deposit = open("Д-000002", IRREVOCABLE_BYN, "5000.00", 3);

    closeDays(91); // 31.12: до окончания срока один
    // день
    assertThat(view(deposit).paid()).isEqualToComparingTo("0");
    assertThat(ledger.balance(deposit.interestAccount()).isEqualToComparingTo("160.81");

    List<String> events = closeDays(1); // 01.01.2027: окончание договора
    // , 92 дня
```

```

DepositView closed = view(deposit);
assertThat(closed.status()).isEqualTo("CLOSED");
assertThat(closed.paid()).isEqualToComparingTo("162.58");

// все счета клиента обнулились, касса транзитом пропустила вклад и проценты, фонд
потратил проценты
assertTurnover(deposit.mainAccount(), "10000.00", "10000.00", "0.00");
assertTurnover(deposit.interestAccount(), "162.58", "162.58", "0.00");
assertTurnover(cash, "10162.58", "10162.58", "0.00");
assertThat(ledger.balance(fund)).isEqualToComparingTo(InMemoryLedger.CAPITAL.subtract(new
BigDecimal("162.58")));
}

```

## Листинг 5.2 – Результат выполнения тестов

```

$ ./mvnw -pl account-service,deposit-service -am verify

Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.182 s -- in by.bsuir.bank.common.i18n.MessagesTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.066 s -- in by.bsuir.bank.account.AccountNumbersTest
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.362 s -- in by.bsuir.bank.account.LedgerLanguageIT
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.064 s -- in by.bsuir.bank.account.BankDayIT
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.348 s -- in by.bsuir.bank.account.LedgerServiceIT
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.063 s -- in by.bsuir.bank.deposit.InterestCalculatorTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.101 s -- in by.bsuir.bank.deposit.DepositLanguageIT
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.344 s -- in by.bsuir.bank.deposit.DepositFlowIT
Tests run: 28, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.469 s -- in by.bsuir.bank.deposit.DepositApiIT

Tests run: 86, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

```

## 6 РЕЗУЛЬТАТЫ РАБОТЫ

Сценарий демонстрации: два клиента открывают вклады по разным программам, банк работает три месяца, оба договора завершаются. Сервис счетов запущен со стартовым капиталом фонда развития 1 000 000 BYN.

После старта в отчёте два собственных счёта банка в белорусских рублях — касса и фонд развития (рисунок 6.1); справочник «План счетов» показан на рисунке 6.2.

BankEt учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Рус

Eng

Бел

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.10.2026

Заккрыть день

30

Заккрыть дней

ОТЧЁТ НА ДАТУ

01.10.2026

СЧЕТОВ В ОТЧЁТЕ

2

ФОНД РАЗВИТИЯ, BYN

1 000 000,00

КАССА, BYN

0,00

Отчёт по счетам

Журнал проводок

План счетов

BYN

USD

EUR

| НОМЕР СЧЁТА        | КОД  | НАЗВАНИЕ СЧЁТА                                  | АКТИВНОСТЬ | ДОГОВОР | ДЕБЕТ | КРЕДИТ       | САЛЬДО       |
|--------------------|------|-------------------------------------------------|------------|---------|-------|--------------|--------------|
| 1010000000019      | 1010 | Касса банка, BYN<br>Денежные средства в кассе   | Активный   |         | 0,00  | 0,00         | 0,00         |
| 7327000000029      | 7327 | Фонд развития банка, BYN<br>Фонд развития банка | Пассивный  |         | 0,00  | 1 000 000,00 | 1 000 000,00 |
| Итого обороты, BYN |      |                                                 |            |         | 0,00  | 1 000 000,00 |              |

Рисунок 6.1 – Отчёт по счетам после запуска: касса и фонд развития

BankEt

учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Рус

Eng

Бел

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ

01.10.2026

Закрыть день

30

Закрыть дней

ОТЧЁТ НА ДАТУ

01.10.2026

СЧЕТОВ В ОТЧЁТЕ

2

ФОНД РАЗВИТИЯ, BYN

1 000 000,00

КАССА, BYN

0,00

Отчёт по счетам

Журнал проводок

План счетов

| БАЛАНСОВЫЙ СЧЁТ | НАИМЕНОВАНИЕ                                                           | АКТИВНОСТЬ |
|-----------------|------------------------------------------------------------------------|------------|
| 1010            | Денежные средства в кассе                                              | Активный   |
| 1201            | Корреспондентский счёт в Национальном банке                            | Активный   |
| 2400            | Кредиты физическим лицам                                               | Активный   |
| 2470            | Начисленные процентные доходы по кредитам физическим лицам             | Активный   |
| 3012            | Текущие (расчётные) счета коммерческих организаций                     | Пассивный  |
| 3014            | Текущие (расчётные) счета физических лиц                               | Пассивный  |
| 3404            | Вклады (депозиты) до востребования физических лиц                      | Пассивный  |
| 3414            | Срочные вклады (депозиты) физических лиц                               | Пассивный  |
| 3470            | Начисленные процентные расходы по вкладам (депозитам) до востребования | Пассивный  |
| 3471            | Начисленные процентные расходы по срочным вкладам (депозитам)          | Пассивный  |
| 7327            | Фонд развития банка                                                    | Пассивный  |

Рисунок 6.2 – Справочник «План счетов»

Форма заключения договора приведена на рисунке 6.3. Клиент выбирается из списка, полученного от модуля «Клиенты», вид депозита и валюта — из справочников; ставка, дата начала и дата окончания подставляются автоматически. На рисунке 6.4 показан контроль данных: номер договора не соответствует формату, сумма меньше минимальной, срок больше допустимого по программе.

**BankEt** учебная АБС Клиенты Счета **Депозиты** Кредиты Банкомат Рус Eng Бел

**Депозитные договоры**  
Вклады физических лиц: заключение договора, начисление и выплата процентов

БАНКОВСКИЙ ДЕНЬ 01.10.2026 **Закрывать день** 30 **Закрывать дней**

**Новый депозитный договор**

Клиент \* Лебедев Артем Владимирович (паспорт MP 3141592) Номер договора \* Д-000001

Валюта \* BYN — Белорусский рубль Вид депозита \* «Свободное накопление» — отзывный, до востребования, 7 % годовых  
Отзывный вклад до востребования, проценты выплачиваются ежемесячно. Сумма от 100,00 BYN, срок 1–12 мес.

Сумма вклада, BYN \* 10000 Срок договора, мес. \* 3 Процент по вкладу, % годовых 7

Дата начала 01.10.2026 Дата окончания 01.01.2027

**ЗаклЮчить договор** **Отмена**

Рисунок 6.3 – Форма заключения депозитного договора

**BankEt** учебная АБС Клиенты Счета **Депозиты** Кредиты Банкомат Рус Eng Бел

**Депозитные договоры**  
Вклады физических лиц: заключение договора, начисление и выплата процентов

БАНКОВСКИЙ ДЕНЬ 01.10.2026 **Закрывать день** 30 **Закрывать дней**

**Новый депозитный договор**

Форма заполнена с ошибками — исправьте отмеченные поля

Клиент \* Лебедев Артем Владимирович (паспорт MP 3141592) Номер договора \* Д-12  
Формат номера договора: Д-000001

Валюта \* BYN — Белорусский рубль Вид депозита \* «Online-решение» — срочный безотзывный, 12.9 % годовых  
Срочный безотзывный вклад, проценты выплачиваются в конце срока. Сумма от 500,00 BYN, срок 3–13 мес.

Сумма вклада, BYN \* 100 Срок договора, мес. \* 24 Процент по вкладу, % годовых 12.9  
Минимальная сумма вклада — 500,00 BYN Срок по программе: от 3 до 13 мес.

Дата начала 01.10.2026 Дата окончания 01.10.2028

**ЗаклЮчить договор** **Отмена**

Рисунок 6.4 – Контроль корректности данных договора

После заключения договора открывается его карточка (рисунок 6.5): созданы счета 3404 и 3470 на имя клиента, проведены три операции приёма вклада. У счёта вклада обороты по дебету и кредиту равны 10 000, сальдо нулевое — деньги переданы в фонд развития, как и предписывает схема.

BankEt

учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Рус

Eng

Бел

Депозитные договоры

Вклады физических лиц: заключение договора, начисление и выплата процентов

БАНКОВСКИЙ ДЕНЬ 01.10.2026

Заккрыть день

30

Заккрыть дней

Договор Д-000001 заключён, открыты счета 3404000010017 и 3470000010025

Договор Д-000001 · Лебедевич Артем Владимирович

Отозвать вклад

К списку

ВИД ДЕПОЗИТА

«Свободное накопление»

Отзывный, до востребования

СУММА ВКЛАДА

10 000,00 BYN

СТАВКА

7 % годовых

СТАТУС

Действует

СРОК

3 мес.

ПЕРИОД

01.10.2026 — 01.01.2027

НАЧИСЛЕНО ПРОЦЕНТОВ

0,00 BYN

ВЫПЛАЧЕНО ПРОЦЕНТОВ

0,00 BYN

Счета договора

| НОМЕР СЧЁТА   | КОД  | НАЗНАЧЕНИЕ                                                                                             | АКТИВНОСТЬ | ДЕБЕТ     | КРЕДИТ    | САЛДО |
|---------------|------|--------------------------------------------------------------------------------------------------------|------------|-----------|-----------|-------|
| 3404000010017 | 3404 | Вклады (депозиты) до востребования физических лиц<br>Лебедевич Артем Владимирович                      | Пассивный  | 10 000,00 | 10 000,00 | 0,00  |
| 3470000010025 | 3470 | Начисленные процентные расходы по вкладам (депозитам) до востребования<br>Лебедевич Артем Владимирович | Пассивный  | 0,00      | 0,00      | 0,00  |

Проводки по договору

| ДАТА       | ОПЕРАЦИЯ                              | СЧЁТ               | ДЕБЕТ     | КРЕДИТ    |
|------------|---------------------------------------|--------------------|-----------|-----------|
| 01.10.2026 | Использование денег банком            | 3404000010017 3404 | 10 000,00 |           |
|            |                                       | 7327000000029 7327 |           | 10 000,00 |
| 01.10.2026 | Перевод денег с кассы на текущий счёт | 1010000000019 1010 |           | 10 000,00 |
|            |                                       | 3404000010017 3404 |           | 10 000,00 |
| 01.10.2026 | Внесение денег в кассу                | 1010000000019 1010 | 10 000,00 |           |

Рисунок 6.5 – Карточка договора: счета и проводки приёма вклада

Вторым договором открыт срочный вклад «Online-решение» на 5 000 BYN. В отчёте по счетам (рисунок 6.6) теперь шесть счетов: касса, фонд развития и по два счёта на каждый договор. Фонд развития вырос на сумму обоих вкладов до 1 015 000 BYN.

BankEt учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Рус

Eng

Бел

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.10.2026

Закрыть день

30

Закрыть дней

ОТЧЁТ НА ДАТУ

01.10.2026

СЧЕТОВ В ОТЧЁТЕ

6

ФОНД РАЗВИТИЯ, BYN

1 015 000,00

КАССА, BYN

0,00

Отчёт по счетам

Журнал проводок

План счетов

BYN

USD

EUR

| НОМЕР СЧЁТА        | КОД  | НАЗВАНИЕ СЧЁТА                                                                                       | АКТИВНОСТЬ | ДОГОВОР  | ДЕБЕТ     | КРЕДИТ       | САЛДО        |
|--------------------|------|------------------------------------------------------------------------------------------------------|------------|----------|-----------|--------------|--------------|
| 1010000000019      | 1010 | Касса банка, BYN<br>Денежные средства в кассе                                                        | Активный   |          | 15 000,00 | 15 000,00    | 0,00         |
| 3404000010017      | 3404 | Лебедев Артем Владимирович<br>Вклады (депозиты) до востребования физических лиц                      | Пассивный  | Д-000001 | 10 000,00 | 10 000,00    | 0,00         |
| 3414000020015      | 3414 | Савицкая Марина Олеговна<br>Срочные вклады (депозиты) физических лиц                                 | Пассивный  | Д-000002 | 5 000,00  | 5 000,00     | 0,00         |
| 3470000010025      | 3470 | Лебедев Артем Владимирович<br>Начисленные процентные расходы по вкладам (депозитам) до востребования | Пассивный  | Д-000001 | 0,00      | 0,00         | 0,00         |
| 3471000020025      | 3471 | Савицкая Марина Олеговна<br>Начисленные процентные расходы по срочным вкладам (депозитам)            | Пассивный  | Д-000002 | 0,00      | 0,00         | 0,00         |
| 7327000000029      | 7327 | Фонд развития банка, BYN<br>Фонд развития банка                                                      | Пассивный  |          | 0,00      | 1 015 000,00 | 1 015 000,00 |
| Итого обороты, BYN |      |                                                                                                      |            |          | 30 000,00 | 1 045 000,00 |              |

Рисунок 6.6 – Отчёт по счетам при двух активных депозитных программах

Процедура «Закрытие банковского дня» выполнена 31 раз подряд (рисунок 6.7). По отзывному вкладу проценты за месяц в размере 59,45 BYN начислены и выплачены, процентный счёт обнулился. По срочному вкладу начислено  $5000 \cdot 12,9 \cdot 31/36500 = 54,78$  BYN, и эта сумма остаётся на счёте 3471 до конца срока. Фонд развития уменьшился на сумму начисленных процентов.

BankEt

учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Рус

Eng

Бел

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.11.2026

Закрыть день

31

Закрыть дней

ОТЧЁТ НА ДАТУ

01.11.2026

СЧЕТОВ В ОТЧЁТЕ

6

ФОНД РАЗВИТИЯ, BYN

1 014 885,77

КАССА, BYN

0,00

Протокол закрытия дня

27.10.2026: Д-000002 — начислены проценты 1.77 BYN  
28.10.2026: Д-000001 — начислены проценты 1.92 BYN  
28.10.2026: Д-000002 — начислены проценты 1.76 BYN  
29.10.2026: Д-000001 — начислены проценты 1.92 BYN  
29.10.2026: Д-000002 — начислены проценты 1.77 BYN  
30.10.2026: Д-000001 — начислены проценты 1.92 BYN  
30.10.2026: Д-000002 — начислены проценты 1.77 BYN  
31.10.2026: Д-000001 — начислены проценты 1.91 BYN  
31.10.2026: Д-000002 — начислены проценты 1.76 BYN  
01.11.2026: Д-000001 — начислены проценты 1.92 BYN  
01.11.2026: Д-000001 — выплачены проценты 59.45 BYN  
01.11.2026: Д-000002 — начислены проценты 1.77 BYN  
... всего записей: 63

Отчёт по счетам

Журнал проводок

План счетов

BYN

USD

EUR

| НОМЕР СЧЁТА        | КОД  | НАЗВАНИЕ СЧЁТА                                                                                         | АКТИВНОСТЬ | ДОГОВОР  | ДЕБЕТ     | КРЕДИТ       | САЛЬДО       |
|--------------------|------|--------------------------------------------------------------------------------------------------------|------------|----------|-----------|--------------|--------------|
| 1010000000019      | 1010 | Касса банка, BYN<br>Денежные средства в кассе                                                          | Активный   |          | 15 059,45 | 15 059,45    | 0,00         |
| 3404000010017      | 3404 | Лебедевич Артем Владимирович<br>Вклады (депозиты) до востребования физических лиц                      | Пассивный  | Д-000001 | 10 000,00 | 10 000,00    | 0,00         |
| 3414000020015      | 3414 | Савицкая Марина Олеговна<br>Срочные вклады (депозиты) физических лиц                                   | Пассивный  | Д-000002 | 5 000,00  | 5 000,00     | 0,00         |
| 3470000010025      | 3470 | Лебедевич Артем Владимирович<br>Начисленные процентные расходы по вкладам (депозитам) до востребования | Пассивный  | Д-000001 | 59,45     | 59,45        | 0,00         |
| 3471000020025      | 3471 | Савицкая Марина Олеговна<br>Начисленные процентные расходы по срочным вкладам (депозитам)              | Пассивный  | Д-000002 | 0,00      | 54,78        | 54,78        |
| 7327000000029      | 7327 | Фонд развития банка, BYN<br>Фонд развития банка                                                        | Пассивный  |          | 114,23    | 1 015 000,00 | 1 014 885,77 |
| Итого обороты, BYN |      |                                                                                                        |            |          | 30 233,13 | 1 045 173,68 |              |

Рисунок 6.7 – Отчёт по счетам через месяц и протокол закрытия дня

Карточка отзывного вклада через месяц и журнал проводок главной книги показаны на рисунках 6.8 и 6.9: видны ежедневные начисления, перевод процентов в кассу и их выдача.

22

BankEt

учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Рус

Eng

Бел

Депозитные договоры

Вклады физических лиц: заключение договора, начисление и выплата процентов

БАНКОВСКИЙ ДЕНЬ

01.11.2026

Заккрыть день

30

Заккрыть дней

Договор Д-000001 · Лебедевич Артем Владимирович

Отозвать вклад

К списку

ВИД ДЕПОЗИТА

«Свободное накопление»

Отзывный, до востребования

СУММА ВКЛАДА

10 000,00 BYN

СТАВКА

7 % годовых

СТАТУС

Действует

СРОК

3 мес.

ПЕРИОД

01.10.2026 — 01.01.2027

НАЧИСЛЕНО ПРОЦЕНТОВ

59,45 BYN

ВЫПЛАЧЕНО ПРОЦЕНТОВ

59,45 BYN

Счета договора

| НОМЕР СЧЁТА   | КОД  | НАЗНАЧЕНИЕ                                                                                             | АКТИВНОСТЬ | ДЕБЕТ     | КРЕДИТ    | САЛДО |
|---------------|------|--------------------------------------------------------------------------------------------------------|------------|-----------|-----------|-------|
| 3484000010017 | 3404 | Вклады (депозиты) до востребования физических лиц<br>Лебедевич Артем Владимирович                      | Пассивный  | 10 000,00 | 10 000,00 | 0,00  |
| 3470000010025 | 3470 | Начисленные процентные расходы по вкладам (депозитам) до востребования<br>Лебедевич Артем Владимирович | Пассивный  | 59,45     | 59,45     | 0,00  |

Проводки по договору

| ДАТА       | ОПЕРАЦИЯ                         | СЧЁТ               | ДЕБЕТ | КРЕДИТ |
|------------|----------------------------------|--------------------|-------|--------|
| 01.11.2026 | Вывод процентов из кассы         | 1010000000019 1010 |       | 59,45  |
| 01.11.2026 | Перевод процентов в кассу        | 3470000010025 3470 | 59,45 |        |
|            |                                  | 1010000000019 1010 | 59,45 |        |
| 01.11.2026 | Начисление процентов по депозиту | 7327000000029 7327 | 1,92  |        |
|            |                                  | 3470000010025 3470 |       | 1,92   |
| 31.10.2026 | Начисление процентов по депозиту | 7327000000029 7327 | 1,91  |        |
|            |                                  | 3470000010025 3470 |       | 1,91   |
| 30.10.2026 | Начисление процентов по депозиту | 7327000000029 7327 | 1,92  |        |
|            |                                  | 3470000010025 3470 |       | 1,92   |
| 29.10.2026 | Начисление процентов по депозиту | 7327000000029 7327 | 1,92  |        |
|            |                                  | 3470000010025 3470 |       | 1,92   |
| 28.10.2026 | Начисление процентов по депозиту | 7327000000029 7327 | 1,92  |        |
|            |                                  | 3470000010025 3470 |       | 1,92   |
| 27.10.2026 | Начисление процентов по депозиту | 7327000000029 7327 | 1,91  |        |

Рисунок 6.8 – Карточка отзывного вклада после выплаты процентов

23

BankEt

учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Рус

Eng

Бел

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.11.2026

Закрыть день

31

Закрыть дней

ОТЧЁТ НА ДАТУ

01.11.2026

СЧЕТОВ В ОТЧЁТЕ

6

ФОНД РАЗВИТИЯ, BYN

1 014 885,77

КАССА, BYN

0,00

Протокол закрытия дня

27.10.2026: Д-000002 — начислены проценты 1.77 BYN  
28.10.2026: Д-000001 — начислены проценты 1.92 BYN  
28.10.2026: Д-000002 — начислены проценты 1.76 BYN  
29.10.2026: Д-000001 — начислены проценты 1.92 BYN  
29.10.2026: Д-000002 — начислены проценты 1.77 BYN  
30.10.2026: Д-000001 — начислены проценты 1.92 BYN  
30.10.2026: Д-000002 — начислены проценты 1.77 BYN  
31.10.2026: Д-000001 — начислены проценты 1.91 BYN  
31.10.2026: Д-000002 — начислены проценты 1.76 BYN  
01.11.2026: Д-000001 — начислены проценты 1.92 BYN  
01.11.2026: Д-000001 — выплачены проценты 59.45 BYN  
01.11.2026: Д-000002 — начислены проценты 1.77 BYN  
... всего записей: 63

Отчёт по счетам

Журнал проводок

План счетов

| №  | ДАТА       | ОПЕРАЦИЯ                         | ДОГОВОР  | СЧЁТ                                     | ДЕБЕТ | КРЕДИТ |
|----|------------|----------------------------------|----------|------------------------------------------|-------|--------|
| 73 | 01.11.2026 | Начисление процентов по депозиту | Д-000002 | 73270000000029 Фонд развития банка, BYN  | 1,77  |        |
|    |            |                                  |          | 3471000020025 Савицкая Марина Олеговна   |       | 1,77   |
| 72 | 01.11.2026 | Вывод процентов из кассы         | Д-000001 | 10100000000019 Касса банка, BYN          |       | 59,45  |
| 71 | 01.11.2026 | Перевод процентов в кассу        | Д-000001 | 3470000010025 Лебедев Артем Владимирович | 59,45 |        |
|    |            |                                  |          | 10100000000019 Касса банка, BYN          | 59,45 |        |
| 70 | 01.11.2026 | Начисление процентов по депозиту | Д-000001 | 73270000000029 Фонд развития банка, BYN  | 1,92  |        |
|    |            |                                  |          | 3470000010025 Лебедев Артем Владимирович |       | 1,92   |
| 69 | 31.10.2026 | Начисление процентов по депозиту | Д-000002 | 73270000000029 Фонд развития банка, BYN  | 1,76  |        |
|    |            |                                  |          | 3471000020025 Савицкая Марина Олеговна   |       | 1,76   |
| 68 | 31.10.2026 | Начисление процентов по депозиту | Д-000001 | 73270000000029 Фонд развития банка, BYN  | 1,91  |        |

Рисунок 6.9 – Журнал проводок главной книги

Те же проводки после переключения интерфейса на английский язык показаны на рисунке 6.10. Названия операций и счетов банка переведены, хотя записи в базе не менялись: журнал хранит коды, а текст подставляется при чтении. Счета клиентов по-прежнему названы их ФИО.

| <div> <div>BankEt</div> <div>training core banking</div> <div>Clients</div> <div>Accounts</div> <div>Deposits</div> <div>Loans</div> <div>ATM</div> <div> <div>Рус</div> <div>Eng</div> <div>Бел</div> </div> </div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |            |                                       |          |                                          |       |        |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|---------------------------------------|----------|------------------------------------------|-------|--------|----|------|-----------|----------|---------|-------|--------|----|------------|--------------------------|----------|------------------------------------------|------|--|--|--|--|--|----------------------------------------|--|------|----|------------|--------------------------------------|----------|-----------------------------------|--|-------|----|------------|---------------------------------------|----------|------------------------------------------|-------|--|--|--|--|--|-----------------------------------|-------|--|----|------------|--------------------------|----------|------------------------------------------|------|--|--|--|--|--|------------------------------------------|--|------|----|------------|--------------------------|----------|------------------------------------------|------|--|--|--|--|--|----------------------------------------|--|------|----|------------|--------------------------|----------|------------------------------------------|------|--|
| <div> <div>Bank accounts</div> <div>Account status report and the "Closing of the banking day" procedure</div> <div> <div>BANKING DAY 01.11.2026</div> <div>Close day</div> <div>30</div> <div>Close days</div> </div> </div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |            |                                       |          |                                          |       |        |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
| <div> <div>REPORT DATE</div> <div>01.11.2026</div> <div>ACCOUNTS IN THE REPORT</div> <div>6</div> <div>DEVELOPMENT FUND, BYN</div> <div>1,014,885.77</div> <div>CASH DESK, BYN</div> <div>0.00</div> </div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |            |                                       |          |                                          |       |        |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
| <div> <div>Account report</div> <div>Posting journal</div> <div>Chart of accounts</div> </div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |            |                                       |          |                                          |       |        |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
| <table> <tr> <th>Nº</th><th>DATE</th><th>OPERATION</th><th>CONTRACT</th><th>ACCOUNT</th><th>DEBIT</th><th>CREDIT</th></tr> <tr> <td>73</td><td>01.11.2026</td><td>Deposit interest accrued</td><td>Д-000002</td><td>7327000000029 Bank development fund, BYN</td><td>1.77</td><td></td></tr> <tr> <td></td><td></td><td></td><td></td><td>3471000020025 Савицкая Марина Олеговна</td><td></td><td>1.77</td></tr> <tr> <td>72</td><td>01.11.2026</td><td>Interest paid out from the cash desk</td><td>Д-000001</td><td>1010000000019 Bank cash desk, BYN</td><td></td><td>59.45</td></tr> <tr> <td>71</td><td>01.11.2026</td><td>Interest transferred to the cash desk</td><td>Д-000001</td><td>3470000010025 Лебедев Артем Владимирович</td><td>59.45</td><td></td></tr> <tr> <td></td><td></td><td></td><td></td><td>1010000000019 Bank cash desk, BYN</td><td>59.45</td><td></td></tr> <tr> <td>70</td><td>01.11.2026</td><td>Deposit interest accrued</td><td>Д-000001</td><td>7327000000029 Bank development fund, BYN</td><td>1.92</td><td></td></tr> <tr> <td></td><td></td><td></td><td></td><td>3470000010025 Лебедев Артем Владимирович</td><td></td><td>1.92</td></tr> <tr> <td>69</td><td>31.10.2026</td><td>Deposit interest accrued</td><td>Д-000002</td><td>7327000000029 Bank development fund, BYN</td><td>1.76</td><td></td></tr> <tr> <td></td><td></td><td></td><td></td><td>3471000020025 Савицкая Марина Олеговна</td><td></td><td>1.76</td></tr> <tr> <td>68</td><td>31.10.2026</td><td>Deposit interest accrued</td><td>Д-000001</td><td>7327000000029 Bank development fund, BYN</td><td>1.91</td><td></td></tr> </table> |            |                                       |          |                                          |       |        | Nº | DATE | OPERATION | CONTRACT | ACCOUNT | DEBIT | CREDIT | 73 | 01.11.2026 | Deposit interest accrued | Д-000002 | 7327000000029 Bank development fund, BYN | 1.77 |  |  |  |  |  | 3471000020025 Савицкая Марина Олеговна |  | 1.77 | 72 | 01.11.2026 | Interest paid out from the cash desk | Д-000001 | 1010000000019 Bank cash desk, BYN |  | 59.45 | 71 | 01.11.2026 | Interest transferred to the cash desk | Д-000001 | 3470000010025 Лебедев Артем Владимирович | 59.45 |  |  |  |  |  | 1010000000019 Bank cash desk, BYN | 59.45 |  | 70 | 01.11.2026 | Deposit interest accrued | Д-000001 | 7327000000029 Bank development fund, BYN | 1.92 |  |  |  |  |  | 3470000010025 Лебедев Артем Владимирович |  | 1.92 | 69 | 31.10.2026 | Deposit interest accrued | Д-000002 | 7327000000029 Bank development fund, BYN | 1.76 |  |  |  |  |  | 3471000020025 Савицкая Марина Олеговна |  | 1.76 | 68 | 31.10.2026 | Deposit interest accrued | Д-000001 | 7327000000029 Bank development fund, BYN | 1.91 |  |
| Nº                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | DATE       | OPERATION                             | CONTRACT | ACCOUNT                                  | DEBIT | CREDIT |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
| 73                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 01.11.2026 | Deposit interest accrued              | Д-000002 | 7327000000029 Bank development fund, BYN | 1.77  |        |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |            |                                       |          | 3471000020025 Савицкая Марина Олеговна   |       | 1.77   |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
| 72                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 01.11.2026 | Interest paid out from the cash desk  | Д-000001 | 1010000000019 Bank cash desk, BYN        |       | 59.45  |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
| 71                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 01.11.2026 | Interest transferred to the cash desk | Д-000001 | 3470000010025 Лебедев Артем Владимирович | 59.45 |        |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |            |                                       |          | 1010000000019 Bank cash desk, BYN        | 59.45 |        |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
| 70                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 01.11.2026 | Deposit interest accrued              | Д-000001 | 7327000000029 Bank development fund, BYN | 1.92  |        |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |            |                                       |          | 3470000010025 Лебедев Артем Владимирович |       | 1.92   |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
| 69                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 31.10.2026 | Deposit interest accrued              | Д-000002 | 7327000000029 Bank development fund, BYN | 1.76  |        |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |            |                                       |          | 3471000020025 Савицкая Марина Олеговна   |       | 1.76   |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |
| 68                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 31.10.2026 | Deposit interest accrued              | Д-000001 | 7327000000029 Bank development fund, BYN | 1.91  |        |    |      |           |          |         |       |        |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                                      |          |                                   |  |       |    |            |                                       |          |                                          |       |  |  |  |  |  |                                   |       |  |    |            |                          |          |                                          |      |  |  |  |  |  |                                          |  |      |    |            |                          |          |                                          |      |  |  |  |  |  |                                        |  |      |    |            |                          |          |                                          |      |  |

Рисунок 6.10 – Тот же журнал проводок на английском языке

После закрытия ещё 61 дня наступает 1 января 2027 года — дата окончания обоих договоров. Вклады возвращены, договоры закрыты (рисунок 6.11). По отзывному вкладу за 92 дня выплачено 176,44 BYN, по срочному — 162,58 BYN одной суммой. В итоговом отчёте (рисунок 6.12) сальдо всех клиентских счетов и кассы нулевое, а фонд развития равен стартовому капиталу за вычетом выплаченных процентов:  $1\,000\,000 - 176,44 - 162,58 = 999\,660,98$  BYN.

BankEt

учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Рус

Eng

Бел

Депозитные договоры

Вклады физических лиц: заключение договора, начисление и выплата процентов

БАНКОВСКИЙ ДЕНЬ 01.01.2027

Заккрыть день

61

Заккрыть дней

Протокол закрытия дня

31.12.2026: Д-000001 — начислены проценты 1.92 BYN

31.12.2026: Д-000002 — начислены проценты 1.77 BYN

01.01.2027: Д-000001 — начислены проценты 1.92 BYN

01.01.2027: Д-000001 — выплачены проценты 59.45 BYN

01.01.2027: Д-000001 — срок договора истёк, вклад 10000.00 BYN возвращён

01.01.2027: Д-000002 — начислены проценты 1.77 BYN

01.01.2027: Д-000002 — выплачены проценты 162.58 BYN

01.01.2027: Д-000002 — срок договора истёк, вклад 5000.00 BYN возвращён

... всего записей: 127

Договоров: 2

Заклучить договор

| ДОГОВОР  | КЛИЕНТ                       | ВИД ДЕПОЗИТА                                       | СУММА         | СТАВКА | НАЧАЛО     | ОКОНЧАНИЕ  | НАЧИСЛЕНО | ВЫПЛАЧЕНО | СТАТУС  |
|----------|------------------------------|----------------------------------------------------|---------------|--------|------------|------------|-----------|-----------|---------|
| Д-000002 | Савицкая Марина Олеговна     | Online-решение<br>Срочный безотзывный              | 5 000,00 BYN  | 12.9 % | 01.10.2026 | 01.01.2027 | 162,58    | 162,58    | Заккрыт |
| Д-000001 | Лебедевич Артем Владимирович | Свободное накопление<br>Отзывный, до востребования | 10 000,00 BYN | 7 %    | 01.10.2026 | 01.01.2027 | 176,44    | 176,44    | Заккрыт |

Рисунок 6.11 – Список договоров после окончания срока

BankEt

учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Рус

Eng

Бел

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.01.2027

Заккрыть день

30

Заккрыть дней

ОТЧЁТ НА ДАТУ

01.01.2027

СЧЕТОВ В ОТЧЁТЕ

6

ФОНД РАЗВИТИЯ, BYN

999 660,98

КАССА, BYN

0,00

Отчёт по счетам

Журнал проводок

План счетов

BYN

USD

EUR

| НОМЕР СЧЁТА        | КОД  | НАЗВАНИЕ СЧЁТА                                                                                         | АКТИВНОСТЬ | ДОГОВОР  | ДЕБЕТ     | КРЕДИТ       | САЛЬДО     |
|--------------------|------|--------------------------------------------------------------------------------------------------------|------------|----------|-----------|--------------|------------|
| 1010000000019      | 1010 | Касса банка, BYN<br>Денежные средства в кассе                                                          | Активный   |          | 30 339,02 | 30 339,02    | 0,00       |
| 3404000010017      | 3404 | Лебедевич Артем Владимирович<br>Вклады (депозиты) до востребования физических лиц                      | Пассивный  | Д-000001 | 20 000,00 | 20 000,00    | 0,00       |
| 3414000020015      | 3414 | Савицкая Марина Олеговна<br>Срочные вклады (депозиты) физических лиц                                   | Пассивный  | Д-000002 | 10 000,00 | 10 000,00    | 0,00       |
| 3470000010025      | 3470 | Лебедевич Артем Владимирович<br>Начисленные процентные расходы по вкладам (депозитам) до востребования | Пассивный  | Д-000001 | 176,44    | 176,44       | 0,00       |
| 3471000020025      | 3471 | Савицкая Марина Олеговна<br>Начисленные процентные расходы по срочным вкладам (депозитам)              | Пассивный  | Д-000002 | 162,58    | 162,58       | 0,00       |
| 7327000000029      | 7327 | Фонд развития банка, BYN<br>Фонд развития банка                                                        | Пассивный  |          | 15 339,02 | 1 015 000,00 | 999 660,98 |
| Итого обороты, BYN |      |                                                                                                        |            |          | 76 017,06 | 1 075 678,04 |            |

Рисунок 6.12 – Итоговый отчёт по счетам

## ЗАКЛЮЧЕНИЕ

В работе реализованы два микросервиса: главная книга банка и сервис депозитных операций. Главная книга ведёт справочник «План счетов», открывает лицевые счета с 13-значным номером и контрольным ключом, проводит операции атомарными пакетами и формирует отчёт о состоянии счетов. Сервис депозитов поддерживает две программы по образцу банка «Решение» — отзывный вклад с ежемесячной выплатой процентов и срочный безотзывный вклад с выплатой в конце срока; при заключении договора он открывает два счёта клиента и выполняет проводки строго по схеме из задания.

Процедура «Закрытие банковского дня» построена как взаимодействие сервисов: главная книга объявляет новую дату, сервис депозитов начисляет и выплачивает проценты. Идемпотентные ключи проводок, сдвиг даты после успешной обработки и накопительный расчёт процентов делают процедуру устойчивой к повторам и к временной недоступности участника — это те свойства, без которых распределённая система не может надёжно работать с деньгами.

Данные о клиентах берутся из модуля первой работы по REST. Интерфейс и ответы сервисов доступны на трёх языках: язык пользователя передаётся между сервисами, а названия проводок хранятся кодами и переводятся при чтении. Корректность проверена 86 автоматическими тестами, в том числе сверкой оборотов всех счетов со схемой проводок на полном жизненном цикле договора.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Об установлении Плана счетов бухгалтерского учёта в банках и небанковских кредитно-финансовых организациях Республики Беларусь: постановление Правления Национального банка Республики Беларусь от 29.08.2013 № 506 [Электронный ресурс]. — Режим доступа: <https://www.nbrb.by/legislation/accountingreporting>. — Дата доступа: 01.10.2026.
2. Вклады для физических лиц / ЗАО «Банк «Решение» [Электронный ресурс]. — Режим доступа: <https://rbank.by/life/deposits/>. — Дата доступа: 01.10.2026.
3. Ньюмен, С. Создание микросервисов / С. Ньюмен. — 2-е изд. — СПб.: Питер, 2023. — 624 с.
4. Ричардсон, К. Микросервисы. Паттерны разработки и рефакторинга / К. Ричардсон. — СПб.: Питер, 2019. — 544 с.
5. Spring Framework Reference: REST Clients, HTTP Interface [Электронный ресурс]. — Режим доступа: <https://docs.spring.io/spring-framework/reference/integration/rest-clients.html>. — Дата доступа: 01.10.2026.

## ПРИЛОЖЕНИЕ А (обязательное)

### Исходный код сервисов счетов и депозитов

#### Общая библиотека сервисов (bank-common)

##### Листинг А.1 – bank-common/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>by.bsuir.bank</groupId>
    <artifactId>bank</artifactId>
    <version>1.0.0</version>
  </parent>

  <artifactId>bank-common</artifactId>
  <name>bank-common</name>
  <description>Общая библиотека сервисов: формат ошибок и REST-клиенты смежных микросервисов</description>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- test-jar: тестовый двойник главной книги InMemoryLedger для тестов сервисов депозитов и кредитов -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <executions>
          <execution>
            <goals>
              <goal>test-jar</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
</project>
```

##### Листинг А.2 – bank-common/src/main/resources/common-messages.properties

```
# Общие тексты сервисов – русский язык (по умолчанию)
validation.required=Обязательное поле
error.checkFields=Проверьте правильность заполнения полей
error.badRequest=Некорректный формат запроса
error.downstream=Ошибка смежного сервиса
error.unavailable=Смежный сервис недоступен, повторите операцию позже
```

```

# Наименования собственных счетов банка
account.cash=Касса банка, {0}
account.fund=Фонд развития банка, {0}
account.operator=Расчётный счёт оператора {0}

# Проводки: главная книга хранит ключ, текст подставляется при чтении
posting.fund.capital=Формирование фонда развития банка
posting.deposit.cashIn=Внесение денег в кассу
posting.deposit.toAccount=Перевод денег с кассы на текущий счёт
posting.deposit.toFund=Использование денег банком
posting.deposit.accrual=Начисление процентов по депозиту
posting.deposit.interestToCash=Перевод процентов в кассу
posting.deposit.interestOut=Вывод процентов из кассы
posting.deposit.maturity=Окончание депозита
posting.deposit.toCash=Перевод депозита в кассу
posting.deposit.cashOut=Вывод денег из кассы
posting.credit.issue=Выделение кредита банком
posting.credit.toCash=Перевод кредита в кассу
posting.credit.cashOut=Получение кредита через кассу
posting.credit.interestIn=Внесение процентов в кассу
posting.credit.interestFromCash=Перевод процентов из кассы
posting.credit.interestCharge=Начисление процентов банком
posting.credit.cashIn=Внесение денег в кассу
posting.credit.repay=Погашение долга за кредит из кассы
posting.credit.partReturn=Возврат части кредита в фонд банка
posting.credit.maturity=Окончание кредита
posting.atm.payment=Оплата услуг связи {0}, номер {1}

# Проверки договоров — общие для депозитов и кредитов
validation.amountPositive=Сумма должна быть больше нуля
validation.amountFormat=Денежная сумма: не более 2 знаков после запятой
validation.termPositive=Срок должен быть больше нуля
contract.duplicate=Договор уже существует
contract.duplicateNumber=Договор с таким номером уже заключён
contract.productCurrency=Программа недоступна в валюте {0}
contract.rate=Ставка по программе — {0} %
contract.termRange=Срок по программе: от {0} до {1} мес.
contract.startDate=Договор заключается текущим банковским днём: {0}
contract.endDate=Дата окончания не соответствует сроку договора
contract.clientNotFound=Клиент не найден в модуле «Клиенты»

```

## Листинг А.3 —

### bank-common/src/main/resources/common-messages\_be.properties

```

# Агульныя тэксты сэрвісаў — беларуская мова
validation.required=Абавязковае поле
error.checkFields=Праверце правільнасць запаўнення палёў
error.badRequest=Некарэктны фармат запиту
error.downstream=Памылка сумежнага сэрвісу
error.unavailable=Сумежны сэрвіс недаступны, паўтарыце аперацыю пазней

account.cash=Каса банка, {0}
account.fund=Фонд развіцця банка, {0}
account.operator=Разліковы рахунак аператара {0}

posting.fund.capital=Фарміраванне фонду развіцця банка
posting.deposit.cashIn=Унясенне грошай у касу
posting.deposit.toAccount=Перавод грошай з касы на бягучы рахунак
posting.deposit.toFund=Выкарыстанне грошай банкам
posting.deposit.accrual=Налічэнне працэнтаў па дэпазіце
posting.deposit.interestToCash=Перавод працэнтаў у касу
posting.deposit.interestOut=Вывад працэнтаў з касы
posting.deposit.maturity=Заканчэнне дэпазіту
posting.deposit.toCash=Перавод дэпазіту ў касу
posting.deposit.cashOut=Вывад грошай з касы
posting.credit.issue=Выдзяленне крэдыту банкам
posting.credit.toCash=Перавод крэдыту ў касу
posting.credit.cashOut=Атрыманне крэдыту праз касу
posting.credit.interestIn=Унясенне працэнтаў у касу
posting.credit.interestFromCash=Перавод працэнтаў з касы
posting.credit.interestCharge=Налічэнне працэнтаў банкам
posting.credit.cashIn=Унясенне грошай у касу
posting.credit.repay=Пагашэнне доўгу па крэдыце з касы
posting.credit.partReturn=Вяртанне часткі крэдыту ў фонд банка
posting.credit.maturity=Заканчэнне крэдыту
posting.atm.payment=Аплата паслуг сувязі {0}, нумар {1}

validation.amountPositive=Сума павінна быць большай за нуль
validation.amountFormat=Грашовая сума: не больш за 2 знакі пасля коскі

```

```
validation.termPositive=Тэрмін павінен быць большым за нуль
contract.duplicate=Дагавор ужо існуе
contract.duplicateNumber=Дагавор з такім нумарам ужо заключаны
contract.productCurrency=Праграма недаступная ў валюце {0}
contract.rate=Стаўка па праграме – {0} %
contract.termRange=Тэрмін па праграме: ад {0} да {1} мес.
contract.startDate=Дагавор заключаецца бягучым банкаўскім днём: {0}
contract.endDate=Дата заканчэння не адпавядае тэрміну дагавора
contract.clientNotFound=Кліент не знойдзены ў модулі «Кліенты»
```

## Листинг А.4 – bank-common/src/main/resources/common-messages\_en.properties

```
# Common service texts – English
validation.required=Required field
error.checkFields=Please check the highlighted fields
error.badRequest=Malformed request
error.downstream=Adjacent service error
error.unavailable=An adjacent service is unavailable, please try again later

account.cash=Bank cash desk, {0}
account.fund=Bank development fund, {0}
account.operator=Settlement account of operator {0}

posting.fund.capital=Bank development fund formed
posting.deposit.cashIn=Cash paid in at the cash desk
posting.deposit.toAccount=Transfer from the cash desk to the current account
posting.deposit.toFund=Funds used by the bank
posting.deposit.accrual=Deposit interest accrued
posting.deposit.interestToCash=Interest transferred to the cash desk
posting.deposit.interestOut=Interest paid out from the cash desk
posting.deposit.maturity=Deposit matured
posting.deposit.toCash=Deposit transferred to the cash desk
posting.deposit.cashOut=Cash paid out from the cash desk
posting.credit.issue=Loan allocated by the bank
posting.credit.toCash=Loan transferred to the cash desk
posting.credit.cashOut=Loan received at the cash desk
posting.credit.interestIn=Interest paid in at the cash desk
posting.credit.interestFromCash=Interest transferred from the cash desk
posting.credit.interestCharge=Interest charged by the bank
posting.credit.cashIn=Cash paid in at the cash desk
posting.credit.repay=Loan repayment from the cash desk
posting.credit.partReturn=Part of the loan returned to the bank fund
posting.credit.maturity=Loan closed
posting.atm.payment=Mobile service payment {0}, number {1}

validation.amountPositive=The amount must be greater than zero
validation.amountFormat=Amount: no more than 2 decimal places
validation.termPositive=The term must be greater than zero
contract.duplicate=The contract already exists
contract.duplicateNumber=A contract with this number has already been signed
contract.productCurrency=The programme is not available in {0}
contract.rate=Programme rate: {0} %
contract.termRange=Programme term: from {0} to {1} months
contract.startDate=The contract is signed on the current banking day: {0}
contract.endDate=The end date does not match the contract term
contract.clientNotFound=The client was not found in the Clients module
```

## Листинг А.5 – bank-common/src/main/java/by/bsuir/bank/common/HttpApis.java

```
package by.bsuir.bank.common;

import by.bsuir.bank.common.i18n.Messages;
import org.springframework.web.client.RestClient;
import org.springframework.web.client.support.RestClientAdapter;
import org.springframework.web.service.invoker.HttpServiceProxyFactory;

import java.util.List;

/** Создание REST-клиента смежного сервиса по интерфейсу с аннотациями @HttpExchange. */
public final class HttpApis {

    public static <T> T proxy(RestClient.Builder builder, String baseUrl, Class<T> api) {
        RestClient client = withLanguage(builder).baseUrl(baseUrl).build();
        return HttpServiceProxyFactory.builderFor(RestClientAdapter.create(client)).build().createClient(api);
    }
}
```

```

/** Язык пользователя передаётся смежному сервису, чтобы его сообщения пришли на том же языке. */
public static RestClient.Builder withLanguage(RestClient.Builder builder) {
    return builder.requestInterceptor((request, body, execution) -> {
        request.getHeaders().setAcceptLanguageAsLocales(List.of(Messages.locale()));
        return execution.execute(request, body);
    });
}

private HttpApis() {
}
}

```

## Листинг А.6 –

### bank-common/src/main/java/by/bsuir/bank/common/IntegrationConfig.java

```

package by.bsuir.bank.common;

import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.ledger.LedgerApi;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;

/** REST-клиенты сервисов «Клиенты» и «Счета» для сервисов депозитов и кредитов. */
@Configuration
@EnableConfigurationProperties(ServiceUrls.class)
public class IntegrationConfig {

    @Bean
    public ClientApi clientApi(RestClient.Builder builder, ServiceUrls urls) {
        return HttpApis.proxy(builder, urls.clients(), ClientApi.class);
    }

    @Bean
    public LedgerApi ledgerApi(RestClient.Builder builder, ServiceUrls urls) {
        return HttpApis.proxy(builder, urls.accounts(), LedgerApi.class);
    }
}

```

## Листинг А.7 –

### bank-common/src/main/java/by/bsuir/bank/common/ServiceUrls.java

```

package by.bsuir.bank.common;

import org.springframework.boot.context.properties.ConfigurationProperties;

/** Адреса смежных микросервисов (bank.services.* в application.yml, переопределяются переменными окружения). */
@ConfigurationProperties("bank.services")
public record ServiceUrls(String clients, String accounts, String deposits, String credits) {
}

```

## Листинг А.8 –

### bank-common/src/main/java/by/bsuir/bank/common/api/ApiError.java

```

package by.bsuir.bank.common.api;

import java.util.Map;

/** Единый формат ошибки всех сервисов: машинный код, сообщение и сообщения по полям запроса. */
public record ApiError(String code, String message, Map<String, String> fields) {
}

```

## Листинг А.9 –

### bank-common/src/main/java/by/bsuir/bank/common/api/BankException.java

```

package by.bsuir.bank.common.api;

import by.bsuir.bank.common.i18n.Messages;
import lombok.Getter;
import org.springframework.http.HttpStatus;

import java.util.Map;

```

```

/** Нарушение бизнес-правила: превращается обработчиком в ответ с кодом состояния и телом ApiError. */
@Getter
public class BankException extends RuntimeException {

    private final HttpStatus status;
    private final String code;
    private final Map<String, String> fields;

    public BankException(HttpStatus status, String code, String message, Map<String, String> fields) {
        super(message);
        this.status = status;
        this.code = code;
        this.fields = fields;
    }

    public BankException(HttpStatus status, String code, String message) {
        this(status, code, message, Map.of());
    }

    public static BankException notFound(String message) {
        return new BankException(HttpStatus.NOT_FOUND, "NOT_FOUND", message);
    }

    public static BankException invalid(String message) {
        return new BankException(HttpStatus.BAD_REQUEST, "INVALID_REQUEST", message);
    }

    public static BankException conflict(String code, String message) {
        return new BankException(HttpStatus.CONFLICT, code, message);
    }

    /** Ошибки, привязанные к полям формы. */
    public static BankException fields(Map<String, String> fields) {
        return new BankException(HttpStatus.BAD_REQUEST, "INVALID_REQUEST", Messages.get("error.checkFields"), fields);
    }
}

```

## Листинг А.10 – bank-common/src/main/java/by/bsuir/bank/common/api/CommonExceptionHandler.java

```

package by.bsuir.bank.common.api;

import by.bsuir.bank.common.i18n.Messages;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.client.ResourceAccessException;
import org.springframework.web.client.RestClientResponseException;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Set;

/** Общий обработчик ошибок REST-контроллеров. */
@RestControllerAdvice
public class CommonExceptionHandler {

    /** Ключ сообщения об обязательном поле: в аннотациях Bean Validation он раскрывается на языке запроса. */
    public static final String REQUIRED = "{validation.required}";

    private static final Set<String> REQUIRED_CODES = Set.of("NotNull", "NotBlank", "NotEmpty");

    @ExceptionHandler(BankException.class)
    public ResponseEntity<ApiError> business(BankException e) {
        return ResponseEntity.status(e.getStatus()).body(new ApiError(e.getCode(), e.getMessage(), e.getFields()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiError invalid(MethodArgumentNotValidException e) {
        Map<String, String> fields = new LinkedHashMap<>();
        // по одному сообщению на поле; незаполненное обязательное поле важнее нарушения формата
        for (FieldError error : e.getBindingResult().getFieldErrors()) {

```

```

        if (REQUIRED_CODES.contains(error.getCode()) || !fields.containsKey(error.getField())) {
            fields.put(error.getField(), String.valueOf(error.getDefaultMessage()));
        }
    }
    return new ApiError("INVALID_REQUEST", Messages.get("error.checkFields"), fields);
}

@ExceptionHandler(HttpMessageNotReadableException.class)
@ResponseStatus(HttpStatus.BAD_REQUEST)
public ApiError unreadable(HttpMessageNotReadableException e) {
    return new ApiError("INVALID_REQUEST", Messages.get("error.badRequest"), Map.of());
}

/** Смежный сервис ответил ошибкой: его сообщение передаётся дальше, 5xx превращается в 502. */
@ExceptionHandler(RestClientResponseException.class)
public ResponseEntity<ApiError> downstream(RestClientResponseException e) {
    ApiError body = null;
    try {
        body = e.getResponseBodyAs(ApiError.class);
    } catch (RuntimeException ignored) {
        // тело ответа не в формате ApiError
    }
    if (body == null || body.message() == null) {
        body = new ApiError("DOWNSTREAM_ERROR", Messages.get("error.downstream"), Map.of());
    }
    HttpStatusCode status = e.getStatusCode().is4xxClientError() ? e.getStatusCode() : HttpStatus.BAD_GATEWAY;
    return ResponseEntity.status(status).body(new ApiError(body.code(), body.message(), Map.of()));
}

/** Смежный сервис не отвечает. */
@ExceptionHandler(ResourceAccessException.class)
@ResponseStatus(HttpStatus.SERVICE_UNAVAILABLE)
public ApiError unavailable(ResourceAccessException e) {
    return new ApiError("SERVICE_UNAVAILABLE", Messages.get("error.unavailable"), Map.of());
}
}

```

## Листинг А.11 –

bank-common/src/main/java/by/bsuir/bank/common/i18n/I18nConfig.java

```

package by.bsuir.bank.common.i18n;

import org.springframework.context.MessageSource;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.LocaleResolver;
import org.springframework.web.servlet.i18n.AcceptHeaderLocaleResolver;

/**
 * Язык запроса определяется заголовком Accept-Language: web-клиент подставляет в него язык,
 * выбранный пользователем, а сервисы передают его дальше при обращении друг к другу.
 */
@Configuration
public class I18nConfig {

    /** Тот же источник текстов используется Bean Validation для сообщений вида {ключ}. */
    @Bean
    public MessageSource messageSource() {
        return Messages.source();
    }

    @Bean
    public LocaleResolver localeResolver() {
        AcceptHeaderLocaleResolver resolver = new AcceptHeaderLocaleResolver();
        resolver.setSupportedLocales(Messages.SUPPORTED);
        resolver.setDefaultLocale(Messages.DEFAULT);
        return resolver;
    }
}

```

## Листинг А.12 –

bank-common/src/main/java/by/bsuir/bank/common/i18n/Localized.java

```

package by.bsuir.bank.common.i18n;

import java.util.Arrays;

/**

```

```

* Тексты, которые система сама записывает в базу (названия проводов, наименования счетов банка),
* хранятся не на конкретном языке, а кодом вида «@ключ|аргумент|аргумент» и превращаются
* в текст при чтении – на языке того, кто читает.
*/
public final class Localized {

    private static final String MARK = "@";
    private static final String SEPARATOR = "|";

    public static String code(String key, Object... args) {
        StringBuilder code = new StringBuilder(MARK).append(key);
        for (Object arg : args) {
            code.append(SEPARATOR).append(arg);
        }
        return code.toString();
    }

    /** Обычный текст (например, ФИО клиента в названии счёта) возвращается как есть. */
    public static String render(String text) {
        if (text == null || !text.startsWith(MARK)) {
            return text;
        }
        String[] parts = text.substring(MARK.length()).split("\\\\" + SEPARATOR);
        return Messages.get(parts[0], (Object[]) Arrays.copyOfRange(parts, 1, parts.length));
    }

    private Localized() {
    }
}

```

## Листинг А.13 –

### bank-common/src/main/java/by/bsuir/bank/common/i18n/Messages.java

```

package by.bsuir.bank.common.i18n;

import org.springframework.context.MessageSource;
import org.springframework.context.NoSuchMessageException;
import org.springframework.context.i18n.LocaleContextHolder;
import org.springframework.context.support.ResourceBundleMessageSource;

import java.util.Arrays;
import java.util.List;
import java.util.Locale;

/**
 * Тексты на языке пользователя: русском, английском или белорусском.
 * Словари – messages*.properties сервиса и common-messages*.properties общей библиотеки;
 * файл без суффикса содержит русские тексты и служит языком по умолчанию.
 */
public final class Messages {

    public static final Locale RU = Locale.forLanguageTag("ru");
    public static final Locale EN = Locale.forLanguageTag("en");
    public static final Locale BE = Locale.forLanguageTag("be");
    public static final Locale DEFAULT = RU;
    public static final List<Locale> SUPPORTED = List.of(RU, EN, BE);

    private static final ResourceBundleMessageSource SOURCE = new ResourceBundleMessageSource();

    static {
        SOURCE.setBasenames("messages", "common-messages");
        SOURCE.setDefaultEncoding("UTF-8");
        SOURCE.setFallbackToSystemLocale(false);
        // вне HTTP-запроса (старт приложения, фоновые задачи) используется русский язык
        LocaleContextHolder.setDefaultLocale(DEFAULT);
    }

    public static MessageSource source() {
        return SOURCE;
    }

    /** Язык текущего запроса; неподдерживаемый язык заменяется русским. */
    public static Locale locale() {
        String language = LocaleContextHolder.getLocale().getLanguage();
        return SUPPORTED.stream().filter(locale -> locale.getLanguage().equals(language)).findFirst().orElse(DEFAULT);
    }

    /** Текст по ключу. Аргументы подставляются как строки, чтобы числа и суммы не переформатировались. */
    public static String get(String code, Object... args) {
        Object[] strings = Arrays.stream(args).map(String::valueOf).toArray();
    }
}

```

```

        try {
            return SOURCE.getMessage(code, strings, locale());
        } catch (NoSuchMessageException e) {
            return code;
        }
    }

    /** Выбор из трёх вариантов значения справочника; непереведённое значение заменяется русским. */
    public static String pick(String ru, String en, String be) {
        String language = locale().getLanguage();
        String value = "en".equals(language) ? en : "be".equals(language) ? be : ru;
        return value == null || value.isBlank() ? ru : value;
    }

    private Messages() {
    }
}

```

## Листинг А.14 –

bank-common/src/main/java/by/bsuir/bank/common/client/ClientApi.java

```

package by.bsuir.bank.common.client;

import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.service.annotation.GetExchange;
import org.springframework.web.service.annotation.HttpExchange;

import java.util.List;

/** REST-клиент модуля «Клиенты» (ЛР1). */
@HttpExchange("/api/clients")
public interface ClientApi {

    @GetExchange
    List<ClientInfo> list();

    @GetExchange("/{id}")
    ClientInfo get(@PathVariable Long id);
}

```

## Листинг А.15 –

bank-common/src/main/java/by/bsuir/bank/common/client/ClientInfo.java

```

package by.bsuir.bank.common.client;

/** Сведения о клиенте, которые нужны договорам: остальные поля ответа сервиса «Клиенты» игнорируются. */
public record ClientInfo(Long id, String fullName, String passportSeries, String passportNumber,
                        String identificationNumber) {
}

```

## Листинг А.16 –

bank-common/src/main/java/by/bsuir/bank/common/ledger/AccountInfo.java

```

package by.bsuir.bank.common.ledger;

import java.math.BigDecimal;

public record AccountInfo(String number, String chartCode, String chartName, String activity, String activityTitle,
                        String currency, String name, Integer clientId, String contractRef,
                        BigDecimal debit, BigDecimal credit, BigDecimal balance) {
}

```

## Листинг А.17 –

bank-common/src/main/java/by/bsuir/bank/common/ledger/BankDayInfo.java

```

package by.bsuir.bank.common.ledger;

import java.time.LocalDate;

public record BankDayInfo(LocalDate date) {
}

```

## Листинг А.18 –

### bank-common/src/main/java/by/bsuir/bank/common/ledger/Batch.java

```
package by.bsuir.bank.common.ledger;

import java.time.LocalDate;
import java.util.List;

/** Пакет операций, проводимый главной книгой атомарно; batchKey – ключ идемпотентности. */
public record Batch(String batchKey, String contractRef, LocalDate bankDate, List<Posting> operations) {

    public static Batch of(String batchKey, String contractRef, LocalDate bankDate, Posting... operations) {
        return new Batch(batchKey, contractRef, bankDate, List.of(operations));
    }
}
```

## Листинг А.19 –

### bank-common/src/main/java/by/bsuir/bank/common/ledger/CurrencyInfo.java

```
package by.bsuir.bank.common.ledger;

public record CurrencyInfo(String code, String name) {
}
```

## Листинг А.20 –

### bank-common/src/main/java/by/bsuir/bank/common/ledger/DayCloseInfo.java

```
package by.bsuir.bank.common.ledger;

import java.time.LocalDate;
import java.util.List;

public record DayCloseInfo(LocalDate bankDate, List<String> events) {
}
```

## Листинг А.21 –

### bank-common/src/main/java/by/bsuir/bank/common/ledger/Entry.java

```
package by.bsuir.bank.common.ledger;

import java.math.BigDecimal;

/** Строка проводки: счёт, сторона и сумма. */
public record Entry(String account, Side side, BigDecimal amount) {

    public static Entry debit(String account, BigDecimal amount) {
        return new Entry(account, Side.DEBIT, amount);
    }

    public static Entry credit(String account, BigDecimal amount) {
        return new Entry(account, Side.CREDIT, amount);
    }
}
```

## Листинг А.22 –

### bank-common/src/main/java/by/bsuir/bank/common/ledger/LedgerApi.java

```
package by.bsuir.bank.common.ledger;

import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.service.annotation.GetExchange;
import org.springframework.web.service.annotation.HttpExchange;
import org.springframework.web.service.annotation.PostExchange;

import java.util.List;

/** REST-клиент сервиса «Счета»: банковский день, счета и проводки. */
@HttpExchange("/api")
public interface LedgerApi {

    String CASH = "1010";
}
```

```

String FUND = "7327";

@GetExchange("/bank-day")
BankDayInfo bankDay();

@PostExchange("/bank-day/close")
DayCloseInfo closeDay(@RequestParam int days);

@GetExchange("/currencies")
List<CurrencyInfo> currencies();

/** Собственный счёт банка: касса (1010) или фонд развития (7327) в заданной валюте. */
@GetExchange("/accounts/system")
AccountInfo systemAccount(@RequestParam String chartCode, @RequestParam String currency);

@GetExchange("/accounts")
List<AccountInfo> accounts(@RequestParam String contractRef);

@GetExchange("/accounts/{number}")
AccountInfo account(@PathVariable String number);

@PostExchange("/accounts")
AccountInfo open(@RequestBody OpenAccount request);

@PostExchange("/operations")
List<OperationInfo> post(@RequestBody Batch batch);

@GetExchange("/operations")
List<OperationInfo> operations(@RequestParam String contractRef);
}

```

## Листинг А.23 – bank-common/src/main/java/by/bsuir/bank/common/ledger/OpenAccount.java

```

package by.bsuir.bank.common.ledger;

public record OpenAccount(String chartCode, String currency, Integer clientId, String name, String contractRef) {
}

```

## Листинг А.24 – bank-common/src/main/java/by/bsuir/bank/common/ledger/OperationInfo.java

```

package by.bsuir.bank.common.ledger;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

public record OperationInfo(Long id, LocalDate bankDate, String description, String contractRef,
    List<EntryInfo> entries) {

    public record EntryInfo(String account, String accountName, String chartCode, Side side, BigDecimal amount) {
    }
}

```

## Листинг А.25 – bank-common/src/main/java/by/bsuir/bank/common/ledger/Posting.java

```

package by.bsuir.bank.common.ledger;

import java.util.List;

/** Одна операция – строка схемы бухгалтерских проводок. */
public record Posting(String description, List<Entry> entries) {

    public static Posting of(String description, Entry... entries) {
        return new Posting(description, List.of(entries));
    }
}

```

## Листинг А.26 – bank-common/src/main/java/by/bsuir/bank/common/ledger/Side.java

```
package by.bsuir.bank.common.ledger;

public enum Side {
    DEBIT, CREDIT
}
```

## Листинг А.27 –

### bank-common/src/test/java/by/bsuir/bank/common/i18n/MessagesTest.java

```
package by.bsuir.bank.common.i18n;

import by.bsuir.bank.common.HttpApis;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.springframework.context.i18n.LocaleContextHolder;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpMethod;
import org.springframework.http.MediaType;
import org.springframework.test.web.client.MockRestServiceServer;
import org.springframework.web.client.RestClient;

import java.util.Locale;

import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.client.match.MockRestRequestMatchers.header;
import static org.springframework.test.web.client.match.MockRestRequestMatchers.method;
import static org.springframework.test.web.client.match.MockRestRequestMatchers.requestTo;
import static org.springframework.test.web.client.response.MockRestResponseCreators.withSuccess;

/** Модульные тесты локализации: выбор языка, подстановка аргументов, коды хранимых текстов, передача языка. */
class MessagesTest {

    @AfterEach
    void resetLocale() {
        LocaleContextHolder.resetLocaleContext();
    }

    @ParameterizedTest(name = "{0}: {1}")
    @CsvSource({
        "ru, Обязательное поле, 'Касса банка, BYN'",
        "en, Required field, 'Bank cash desk, BYN'",
        "be, Абавязковае поле, 'Каса банка, BYN'",
        "en-GB, Required field, 'Bank cash desk, BYN'",
        "de, Обязательное поле, 'Касса банка, BYN'"
    })
    @DisplayName("Текст выбирается по языку запроса, неподдерживаемый язык заменяется русским")
    void messageByLocale(String language, String required, String cash) {
        LocaleContextHolder.setLocale(Locale.forLanguageTag(language));

        assertThat(Messages.get("validation.required")).isEqualTo(required);
        assertThat(Messages.get("account.cash", "BYN")).isEqualTo(cash);
    }

    @Test
    @DisplayName("Вне запроса используется русский язык, неизвестный ключ возвращается как есть")
    void defaults() {
        assertThat(Messages.locale()).isEqualTo(Messages.RU);
        assertThat(Messages.get("validation.required")).isEqualTo("Обязательное поле");
        assertThat(Messages.get("no.such.key")).isEqualTo("no.such.key");
    }

    @Test
    @DisplayName("Числа подставляются без переформатирования")
    void argumentsAreNotReformatted() {
        assertThat(Messages.get("posting.atm.payment", "МТС", "0297654321"))
            .isEqualTo("Оплата услуг связи МТС, номер 0297654321");
        assertThat(Messages.get("account.cash", "1234567.5")).isEqualTo("Касса банка, 1234567.5");
    }

    @Test
    @DisplayName("Значение справочника выбирается из трёх столбцов; пустой перевод заменяется русским")
    void pick() {
        LocaleContextHolder.setLocale(Messages.BE);
        assertThat(Messages.pick("Минск", "Minsk", "Мінск")).isEqualTo("Мінск");
        assertThat(Messages.pick("Минск", "Minsk", null)).isEqualTo("Минск");

        LocaleContextHolder.setLocale(Messages.EN);
    }
}
```

```

        assertThat(Messages.pick("Минск", "Minsk", "Мінск")).isEqualTo("Minsk");
    }

    @Test
    @DisplayName("Хранимый код раскрывается на языке читающего, обычный текст не меняется")
    void localizedCodes() {
        String posting = Localized.code("posting.deposit.accrual");
        String payment = Localized.code("posting.atm.payment", "A1", "0291112233");
        assertThat(posting).isEqualTo("@posting.deposit.accrual");
        assertThat(payment).isEqualTo("@posting.atm.payment|A1|0291112233");

        assertThat(Localized.render(posting)).isEqualTo("Начисление процентов по депозиту");
        assertThat(Localized.render("Иванов Иван Иванович")).isEqualTo("Иванов Иван Иванович");
        assertThat(Localized.render(null)).isNull();

        LocaleContextHolder.setLocale(Messages.EN);
        assertThat(Localized.render(posting)).isEqualTo("Deposit interest accrued");
        assertThat(Localized.render(payment)).isEqualTo("Mobile service payment A1, number 0291112233");

        LocaleContextHolder.setLocale(Messages.BE);
        assertThat(Localized.render(posting)).isEqualTo("Налічэнне працэнтаў па дэпазіце");
    }

    @Test
    @DisplayName("Язык пользователя передаётся смежному сервису в заголовке Accept-Language")
    void languageIsPropagated() {
        RestClient.Builder builder = HttpApis.withLanguage(RestClient.builder());
        MockRestServiceServer server = MockRestServiceServer.bindTo(builder).build();
        server.expect(requestTo("http://ledger/api/bank-day"))
            .andExpect(method(HttpMethod.GET))
            .andExpect(header(HttpHeaders.ACCEPT_LANGUAGE, "be"))
            .andRespond(withSuccess("{} ", MediaType.APPLICATION_JSON));
        LocaleContextHolder.setLocale(Messages.BE);

        builder.build().get().uri("http://ledger/api/bank-day").retrieve().toBodilessEntity();

        server.verify();
    }
}

```

## Листинг А.28 –

### bank-common/src/test/java/by/bsuir/bank/common/ledger/InMemoryLedger.java

```

package by.bsuir.bank.common.ledger;

import by.bsuir.bank.common.i18n.Localized;
import org.springframework.http.HttpStatus;
import org.springframework.web.client.HttpClientErrorException;

import java.math.BigDecimal;
import java.nio.charset.StandardCharsets;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Тестовый двойник сервиса «Счета»: главная книга в памяти с теми же правилами
 * (сальдо по активности, запрет отрицательного остатка, атомарность и идемпотентность пакета).
 * Позволяет тестировать сервисы депозитов и кредитов без запуска смежного микросервиса.
 */
public class InMemoryLedger implements LedgerApi {

    public static final LocalDate START = LocalDate.of(2026, 10, 1);
    public static final BigDecimal CAPITAL = new BigDecimal("1000000.00");

    private final Map<String, AccountInfo> accounts = new LinkedHashMap<>();
    private final Map<String, List<OperationInfo>> batches = new LinkedHashMap<>();
    private LocalDate date;
    private long operationSeq;

    public InMemoryLedger() {
        reset();
    }

    /** Возвращает книгу в исходное состояние: касса и фонд развития в трёх валютах, день 01.10.2026. */
    public final void reset() {
        accounts.clear();
        batches.clear();
    }
}

```

```

        operationSeq = 0;
        date = START;
        for (String currency : List.of("BYN", "USD", "EUR")) {
            put(CASH, currency, null, Localized.code("account.cash", currency), null, BigDecimal.ZERO);
            put(FUND, currency, null, Localized.code("account.fund", currency), null, CAPITAL);
        }
    }

    public void setDate(LocalDate date) {
        this.date = date;
    }

    public BigDecimal debit(String number) {
        return account(number).debit();
    }

    public BigDecimal credit(String number) {
        return account(number).credit();
    }

    public BigDecimal balance(String number) {
        return account(number).balance();
    }

    /** Журнал операций; названия операций и счетов банка раскрываются на языке читающего. */
    public List<OperationInfo> journal() {
        return batches.values().stream().flatMap(List::stream).map(InMemoryLedger::localized).toList();
    }

    private static OperationInfo localized(OperationInfo operation) {
        return new OperationInfo(operation.id(), operation.bankDate(), Localized.render(operation.description()),
            operation.contractRef(), operation.entries().stream()
                .map(e -> new OperationInfo.EntryInfo(e.account(), Localized.render(e.accountName()), e.chartCode(),
                    e.side(), e.amount()))
                .toList());
    }

    private static AccountInfo localized(AccountInfo a) {
        return new AccountInfo(a.number(), a.chartCode(), a.chartName(), a.activity(), a.activityTitle(), a.currency(),
            Localized.render(a.name()), a.clientId(), a.contractRef(), a.debit(), a.credit(), a.balance());
    }

    @Override
    public BankDayInfo bankDay() {
        return new BankDayInfo(date);
    }

    @Override
    public DayCloseInfo closeDay(int days) {
        date = date.plusDays(days);
        return new DayCloseInfo(date, List.of());
    }

    @Override
    public List<CurrencyInfo> currencies() {
        return List.of(new CurrencyInfo("BYN", "Белорусский рубль"), new CurrencyInfo("EUR", "Евро"),
            new CurrencyInfo("USD", "Доллар США"));
    }

    @Override
    public AccountInfo systemAccount(String chartCode, String currency) {
        return accounts.values().stream()
            .filter(a -> a.chartCode().equals(chartCode) && a.currency().equals(currency) && a.clientId() == null
                && a.contractRef() == null)
            .findFirst().orElseThrow();
    }

    @Override
    public List<AccountInfo> accounts(String contractRef) {
        return accounts.values().stream().filter(a -> contractRef.equals(a.contractRef()))
            .map(InMemoryLedger::localized).toList();
    }

    @Override
    public AccountInfo account(String number) {
        AccountInfo account = accounts.get(number);
        if (account == null) {
            throw error(HttpStatus.NOT_FOUND, "NOT_FOUND", "Счёт " + number + " не найден");
        }
        return account;
    }
}

```

```

@Override
public AccountInfo open(OpenAccount request) {
    if (request.contractRef() != null) {
        var existing = accounts.values().stream()
            .filter(a -> request.contractRef().equals(a.contractRef()) && a.chartCode().equals(request.chartCode()))
            .findFirst();
        if (existing.isPresent()) {
            return existing.get();
        }
    }
    return put(request.chartCode(), request.currency(), request.clientId(), request.name(), request.contractRef(),
        BigDecimal.ZERO);
}

@Override
public List<OperationInfo> post(Batch batch) {
    if (batches.containsKey(batch.batchKey())) {
        return batches.get(batch.batchKey()).stream().map(InMemoryLedger::localized).toList();
    }
    Map<String, AccountInfo> draft = new LinkedHashMap<>(accounts);
    List<OperationInfo> posted = new ArrayList<>();
    for (Posting posting : batch.operations()) {
        List<OperationInfo.EntryInfo> entries = new ArrayList<>();
        for (Entry entry : posting.entries()) {
            AccountInfo account = draft.get(entry.account());
            if (account == null) {
                throw error(HttpStatus.NOT_FOUND, "NOT_FOUND", "Счёт " + entry.account() + " не найден");
            }
            BigDecimal debit = entry.side() == Side.DEBIT ? account.debit().add(entry.amount()) : account.debit();
            BigDecimal credit = entry.side() == Side.CREDIT ? account.credit().add(entry.amount()) : account.credit();
            draft.put(account.number(), copy(account, debit, credit));
            entries.add(new OperationInfo.EntryInfo(account.number(), account.name(), account.chartCode(),
                entry.side(), entry.amount()));
        }
        for (Entry entry : posting.entries()) {
            if (draft.get(entry.account()).balance().signum() < 0) {
                throw error(HttpStatus.CONFLICT, "INSUFFICIENT_FUNDS",
                    "Недостаточно средств на счёте " + entry.account() + " для операции «"
                    + Localized.render(posting.description()) + "»");
            }
        }
        posted.add(new OperationInfo(++operationSeq, batch.bankDate() != null ? batch.bankDate() : date,
            posting.description(), batch.contractRef(), entries));
    }
    accounts.putAll(draft);
    batches.put(batch.batchKey(), posted);
    return posted.stream().map(InMemoryLedger::localized).toList();
}

@Override
public List<OperationInfo> operations(String contractRef) {
    return journal().stream().filter(o -> contractRef.equals(o.contractRef())).toList();
}

private AccountInfo put(String chartCode, String currency, Integer clientId, String name, String contractRef,
    BigDecimal credit) {
    boolean passive = chartCode.startsWith("3") || chartCode.startsWith("7");
    String number = chartCode + String.format("%09d", accounts.size() + 1);
    AccountInfo account = copy(new AccountInfo(number, chartCode, "", passive ? "PASSIVE" : "ACTIVE",
        passive ? "Пассивный" : "Активный", currency, name, clientId, contractRef, null, null, null),
        BigDecimal.ZERO, credit);
    accounts.put(number, account);
    return account;
}

private static AccountInfo copy(AccountInfo a, BigDecimal debit, BigDecimal credit) {
    BigDecimal balance = "PASSIVE".equals(a.activity()) ? credit.subtract(debit) : debit.subtract(credit);
    return new AccountInfo(a.number(), a.chartCode(), a.chartName(), a.activity(), a.activityTitle(),
        a.currency(), a.name(), a.clientId(), a.contractRef(), debit, credit, balance);
}

/** Ошибка в том же виде, в каком её вернул бы настоящий сервис: код состояния и тело ApiError. */
private static HttpClientErrorException error(HttpStatus status, String code, String message) {
    String body = "{ \"code\": \"\" + code + \"\", \"message\": \"\" + message.replace(\"\", \"\") + \"\", \"fields\": {}}";
    return HttpClientErrorException.create(status, status.getReasonPhrase(), new org.springframework.http.HttpHeaders(),
        body.getBytes(StandardCharsets.UTF_8), StandardCharsets.UTF_8);
}
}

```

## Микросервис «Счета» (account-service)

### Листинг A.29 – account-service/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>by.bsuir.bank</groupId>
    <artifactId>bank</artifactId>
    <version>1.0.0</version>
  </parent>

  <artifactId>account-service</artifactId>
  <name>account-service</name>
  <description>ЛР2. Микросервис «Счета»: план счетов, лицевые счета, проводки, банковский день</description>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>by.bsuir.bank</groupId>
      <artifactId>bank-common</artifactId>
    </dependency>
    <dependency>
      <groupId>by.bsuir.bank</groupId>
      <artifactId>ui-kit</artifactId>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-failsafe-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

### Листинг A.30 – account-service/src/main/resources/application.yml

```
server:
  port: ${ACCOUNT_SERVICE_PORT:8082}

spring:
```

```

application:
  name: account-service
datasource:
  url: jdbc:h2:mem:accounts;DB_CLOSE_DELAY=-1
  username: sa
sql:
  init:
    mode: always
jpa:
  hibernate:
    ddl-auto: validate
    open-in-view: false
h2:
  console:
    enabled: true

bank:
fund:
  # Стартовый капитал фонда развития банка по валютам (ЛР2 – 1 млн BYN, ЛР3 – 100 млрд BYN)
  capital:
    BYN: ${BANK_FUND_CAPITAL_BYN:100000000000}
    USD: 5000000
    EUR: 5000000
  day-close:
    # Сервисы, которые обрабатывают открытие нового банковского дня: начисляют проценты, проводят платежи.
    # name – ключ названия участника в словаре (participant.<name>)
    participants:
      - name: deposits
        url: ${DEPOSIT_SERVICE_URL:http://localhost:8083}
      - name: credits
        url: ${CREDIT_SERVICE_URL:http://localhost:8084}

```

## Листинг А.31 – account-service/src/main/resources/schema.sql

```

-- Схема данных главной книги банка

create table currency (
  code   varchar(3) primary key,
  name   varchar(60) not null,
  name_en varchar(60) not null,
  name_be varchar(60) not null
);

-- Справочник «План счетов»: балансовый счёт, наименование на трёх языках и активность
create table chart_of_accounts (
  code   varchar(4) primary key,
  name   varchar(200) not null,
  name_en varchar(200) not null,
  name_be varchar(200) not null,
  activity varchar(14) not null check (activity in ('ACTIVE', 'PASSIVE', 'ACTIVE_PASSIVE'))
);

-- Лицевые счета: 13-значный номер, обороты по дебету и кредиту
create table account (
  id          bigint generated by default as identity primary key,
  number      varchar(13) not null unique,
  chart_code  varchar(4) not null references chart_of_accounts,
  currency_code varchar(3) not null references currency,
  name        varchar(200) not null,          -- ФИО клиента либо код наименования счёта банка
  owner_code  int         not null,          -- код клиента, 0 – счёт самого банка
  owner_seq   int         not null,          -- порядковый номер счёта у владельца
  contract_ref varchar(40),                  -- договор, в рамках которого открыт счёт
  debit       decimal(19, 2) not null default 0,
  credit      decimal(19, 2) not null default 0,
  opened_on   date         not null,
  constraint uq_account_owner unique (owner_code, owner_seq)
);

create index ix_account_contract on account (contract_ref);

-- Ключи идемпотентности: повтор запроса с тем же ключом не создаёт проводки второй раз
create table posting_batch (
  batch_key  varchar(80) primary key,
  created_at timestamp not null
);

-- Журнал операций и строки проводок
create table operation (
  id          bigint generated by default as identity primary key,
  batch_key   varchar(80) not null references posting_batch,

```

```

bank_date    date          not null,
description  varchar(200) not null,          -- код названия операции: текст подставляется при чтении
contract_ref varchar(40),
created_at   timestamp     not null
);

create table operation_entry (
  id          bigint generated by default as identity primary key,
  operation_id bigint        not null references operation,
  account_id  bigint        not null references account,
  side        varchar(6)    not null check (side in ('DEBIT', 'CREDIT')),
  amount      decimal(19, 2) not null check (amount > 0)
);

create index ix_entry_account on operation_entry (account_id);

-- Текущий открытый банковский день (одна строка)
create table bank_day (
  id          int primary key,
  bank_date   date not null
);

```

## Листинг А.32 – account-service/src/main/resources/data.sql

```

insert into currency (code, name, name_en, name_be) values
('BYN', 'Белорусский рубль', 'Belarusian ruble', 'Беларускі рубель'),
('USD', 'Доллар США', 'US dollar', 'Долар ЗША'),
('EUR', 'Евро', 'Euro', 'Еўра');

-- Балансовые счета, необходимые для депозитных и кредитных программ физических лиц
insert into chart_of_accounts (code, activity, name, name_en, name_be) values
('1010', 'ACTIVE', 'Денежные средства в кассе',
  'Cash on hand', 'Грашовыя сродкі ў касе'),
('1201', 'ACTIVE', 'Корреспондентский счёт в Национальном банке',
  'Correspondent account with the National Bank', 'Карэспандэнцкі рахунак у Нацыянальным банку'),
('2400', 'ACTIVE', 'Кредиты физическим лицам',
  'Loans to individuals', 'Крэдыты фізічным асобам'),
('2470', 'ACTIVE', 'Начисленные процентные доходы по кредитам физическим лицам',
  'Accrued interest income on loans to individuals', 'Налічаныя працэнтныя даходы па крэдытах фізічным асобам'),
('3012', 'PASSIVE', 'Текущие (расчётные) счета коммерческих организаций',
  'Current (settlement) accounts of commercial organisations', 'Бягучыя (разліковыя) рахункі камерцыйных арганізацый'),
('3014', 'PASSIVE', 'Текущие (расчётные) счета физических лиц',
  'Current (settlement) accounts of individuals', 'Бягучыя (разліковыя) рахункі фізічных асоб'),
('3404', 'PASSIVE', 'Вклады (депозиты) до востребования физических лиц',
  'Demand deposits of individuals', 'Уклады (дэпазіты) да запатрабавання фізічных асоб'),
('3414', 'PASSIVE', 'Срочные вклады (депозиты) физических лиц',
  'Term deposits of individuals', 'Тэрміновыя ўклады (дэпазіты) фізічных асоб'),
('3470', 'PASSIVE', 'Начисленные процентные расходы по вкладам (депозитам) до востребования',
  'Accrued interest expenses on demand deposits', 'Налічаныя працэнтныя выдаткі па ўкладах (дэпазітах) да запатрабавання'),
('3471', 'PASSIVE', 'Начисленные процентные расходы по срочным вкладам (депозитам)',
  'Accrued interest expenses on term deposits', 'Налічаныя працэнтныя выдаткі па тэрміновым укладах (дэпазітах)'),
('7327', 'PASSIVE', 'Фонд развития банка',
  'Bank development fund', 'Фонд развіцця банка');

insert into bank_day (id, bank_date) values (1, current_date);

```

## Листинг А.33 – account-service/src/main/resources/messages.properties

```

# Тексты сервиса «Счета» — русский язык (по умолчанию)
ledger.noChart=Балансового счёта {0} нет в плане счетов
ledger.noCurrency=Валюты {0} нет в справочнике
ledger.accountNotFound=Счёт {0} не найден
ledger.systemAccountNotFound=Счёт банка {0} в валюте {1} не открыт
ledger.insufficient=Недостаточно средств на счёте {0} («{1}») для операции «{2}»
ledger.sameCurrency=Счета одной операции должны быть в одной валюте

day.range=Количество дней должно быть от 1 до {0}
day.inProgress=Закрытие дня уже выполняется
day.participantFailed=Сервис «{0}» не обработал день {1}, день не закрыт
day.participantUnavailable={0}: сервис «{1}» недоступен, обработка отложена
participant.deposits=Депозиты
participant.credits=Кредиты

activity.ACTIVE=Активный
activity.PASSIVE=Пассивный
activity.ACTIVE_PASSIVE=Активно-пассивный

```

## Листинг А.34 – account-service/src/main/resources/messages\_be.properties

```
# Тэксты сэрвісу «Рахункі» – беларуская мова
ledger.noChart=Балансавага рахунку {0} няма ў плане рахункаў
ledger.noCurrency=Валюты {0} няма ў даведніку
ledger.accountNotFound=Рахунак {0} не знойдзены
ledger.systemAccountNotFound=Рахунак банка {0} у валюце {1} не адкрыты
ledger.insufficient=Недастаткова сродкаў на рахунку {0} («{1}») для аперацыі «{2}»
ledger.sameCurrency=Рахункі адной аперацыі павінны быць у адной валюце

day.range=Колькасць дзён павінна быць ад 1 да {0}
day.inProgress=Закрывае дзень ўжо выконваецца
day.participantFailed=Сэрвіс «{0}» не апрацаваў дзень {1}, дзень не закрыты
day.participantUnavailable={0}: сэрвіс «{1}» недаступны, апрацоўка адкладзена
participant.deposits=Дэпазіты
participant.credits=Крэдыты

activity.ACTIVE=Актыўны
activity.PASSIVE=Пасіўны
activity.ACTIVE_PASSIVE=Актыўна-пасіўны
```

## Листинг А.35 – account-service/src/main/resources/messages\_en.properties

```
# Account service texts – English
ledger.noChart=Balance account {0} is not in the chart of accounts
ledger.noCurrency=Currency {0} is not in the dictionary
ledger.accountNotFound=Account {0} not found
ledger.systemAccountNotFound=Bank account {0} in currency {1} is not opened
ledger.insufficient=Insufficient funds on account {0} ("{1}") for operation "{2}"
ledger.sameCurrency=Accounts of one operation must be in the same currency

day.range=The number of days must be from 1 to {0}
day.inProgress=Day closing is already in progress
day.participantFailed=Service "{0}" failed to process day {1}, the day is not closed
day.participantUnavailable={0}: service "{1}" is unavailable, processing postponed
participant.deposits=Deposits
participant.credits=Loans

activity.ACTIVE=Active
activity.PASSIVE=Passive
activity.ACTIVE_PASSIVE=Active-passive
```

## Листинг А.36 – account-service/src/main/java/by/bsuir/bank/account/AccountServiceApplication.java

```
package by.bsuir.bank.account;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;

// из общей библиотеки подключаются обработчик ошибок и локализация
@SpringBootApplication(scanBasePackages = {"by.bsuir.bank.account", "by.bsuir.bank.common.api",
    "by.bsuir.bank.common.i18n"})
@ConfigurationPropertiesScan
public class AccountServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(AccountServiceApplication.class, args);
    }
}
```

## Листинг А.37 – account-service/src/main/java/by/bsuir/bank/account/BankProperties.java

```
package by.bsuir.bank.account;

import org.springframework.boot.context.properties.ConfigurationProperties;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;

/** Настройки главной книги: стартовый капитал фонда развития и участники закрытия дня. */
@ConfigurationProperties("bank")
public record BankProperties(Fund fund, DayClose dayClose) {
```

```

    public record Fund(Map<String, BigDecimal> capital) {
    }

    public record DayClose(List<Participant> participants) {
    }

    public record Participant(String name, String url) {
    }
}

```

## Листинг А.38 – account-service/src/main/java/by/bsuir/bank/account/domain/Account.java

```

package by.bsuir.bank.account.domain;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.LocalDate;

/** Лицевой счёт: обороты по дебету и кредиту накапливаются, сальдо вычисляется по активности. */
@Entity
@Getter
@Setter
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String number;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    @JoinColumn(name = "chart_code")
    private ChartAccount chart;

    @Column(name = "currency_code")
    private String currency;

    private String name;
    private int ownerCode;
    private int ownerSeq;
    private String contractRef;
    private BigDecimal debit = BigDecimal.ZERO;
    private BigDecimal credit = BigDecimal.ZERO;
    private LocalDate openedOn;

    public BigDecimal getBalance() {
        return chart.getActivity().balance(debit, credit);
    }

    public void apply(Side side, BigDecimal amount) {
        if (side == Side.DEBIT) {
            debit = debit.add(amount);
        } else {
            credit = credit.add(amount);
        }
    }
}

```

## Листинг А.39 – account- service/src/main/java/by/bsuir/bank/account/domain/AccountNumbers.java

```

package by.bsuir.bank.account.domain;

/**
 * Номер счёта из 13 цифр: 1–4 – балансовый счёт из плана счетов, 5–9 – код клиента,

```

```

* 10-12 — порядковый номер счёта этого клиента, 13 — контрольный ключ.
*/
public final class AccountNumbers {

    private static final int[] WEIGHTS = {7, 1, 3};

    public static String generate(String chartCode, int ownerCode, int ownerSeq) {
        if (!chartCode.matches("\\d{4}") || ownerCode < 0 || ownerCode > 99_999 || ownerSeq < 1 || ownerSeq > 999) {
            throw new IllegalArgumentException("Номер счёта не помещается в 13 разрядов");
        }
        String body = chartCode + String.format("%05d%03d", ownerCode, ownerSeq);
        return body + checkDigit(body);
    }

    /** Контрольный ключ: взвешенная сумма первых 12 цифр (веса 7, 1, 3), умноженная на 3, по модулю 10. */
    static int checkDigit(String body) {
        int sum = 0;
        for (int i = 0; i < body.length(); i++) {
            sum += (body.charAt(i) - '0') * WEIGHTS[i % WEIGHTS.length];
        }
        return sum * 3 % 10;
    }

    public static boolean isValid(String number) {
        return number != null && number.matches("\\d{13}")
            && checkDigit(number.substring(0, 12)) == number.charAt(12) - '0';
    }

    private AccountNumbers() {
    }
}

```

## Листинг А.40 — account-service/src/main/java/by/bsuir/bank/account/domain/Activity.java

```

package by.bsuir.bank.account.domain;

import by.bsuir.bank.common.i18n.Messages;

import java.math.BigDecimal;

/** Активность балансового счёта определяет, какая сторона увеличивает остаток. */
public enum Activity {

    ACTIVE,
    PASSIVE,
    ACTIVE_PASSIVE;

    /** Название на языке запроса: «Активный», «Пассивный», «Активно-пассивный». */
    public String getTitle() {
        return Messages.get("activity." + name());
    }

    /** Сальдо: у активного счёта дебет минус кредит, у пассивного — кредит минус дебет. */
    public BigDecimal balance(BigDecimal debit, BigDecimal credit) {
        return this == PASSIVE ? credit.subtract(debit) : debit.subtract(credit);
    }
}

```

## Листинг А.41 — account-service/src/main/java/by/bsuir/bank/account/domain/BankDay.java

```

package by.bsuir.bank.account.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import lombok.Getter;
import lombok.Setter;

import java.time.LocalDate;

/** Текущий открытый банковский день — единственная строка таблицы. */
@Entity
@Getter
@Setter
public class BankDay {

    public static final int ID = 1;
}

```

```

    @Id
    private Integer id;

    private LocalDate bankDate;
}

```

## Листинг А.42 – account-service/src/main/java/by/bsuir/bank/account/domain/ChartAccount.java

```

package by.bsuir.bank.account.domain;

import by.bsuir.bank.common.i18n.Messages;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.AccessLevel;
import lombok.Getter;
import lombok.Setter;

/** Запись справочника «План счетов» – балансовый счёт. Наименование хранится на трёх языках. */
@Entity
@Table(name = "chart_of_accounts")
@Getter
@Setter
public class ChartAccount {

    @Id
    private String code;

    private String name;

    @Getter(AccessLevel.NONE)
    private String nameEn;

    @Getter(AccessLevel.NONE)
    private String nameBe;

    @Enumerated(EnumType.STRING)
    private Activity activity;

    public String getName() {
        return Messages.pick(name, nameEn, nameBe);
    }

    public String getActivityTitle() {
        return activity.getTitle();
    }
}

```

## Листинг А.43 – account-service/src/main/java/by/bsuir/bank/account/domain/Currency.java

```

package by.bsuir.bank.account.domain;

import by.bsuir.bank.common.i18n.Messages;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import lombok.AccessLevel;
import lombok.Getter;
import lombok.Setter;

/** Справочник валют. Наименование хранится на трёх языках. */
@Entity
@Getter
@Setter
public class Currency {

    @Id
    private String code;

    private String name;

    @Getter(AccessLevel.NONE)
    private String nameEn;
}

```

```

@Getter(AccessLevel.NONE)
private String nameBe;

public String getName() {
    return Messages.pick(name, nameEn, nameBe);
}
}

```

## Листинг А.44 – account-service/src/main/java/by/bsuir/bank/account/domain/Operation.java

```

package by.bsuir.bank.account.domain;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import jakarta.persistence.OrderBy;
import lombok.Getter;
import lombok.Setter;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

/** Банковская операция (проводка): одна строка схемы «Депозитная/Кредитная программа». */
@Entity
@Getter
@Setter
public class Operation {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String batchKey;
    private LocalDate bankDate;
    private String description;
    private String contractRef;
    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "operation", cascade = CascadeType.ALL, orphanRemoval = true)
    @OrderBy("id")
    private List<OperationEntry> entries = new ArrayList<>();

    public void addEntry(Account account, Side side, java.math.BigDecimal amount) {
        OperationEntry entry = new OperationEntry();
        entry.setOperation(this);
        entry.setAccount(account);
        entry.setSide(side);
        entry.setAmount(amount);
        entries.add(entry);
    }
}

```

## Листинг А.45 – account- service/src/main/java/by/bsuir/bank/account/domain/OperationEntry.java

```

package by.bsuir.bank.account.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;

/** Строка проводки: счёт, сторона (дебет или кредит) и сумма. */

```

```

@Entity
@Getter
@Setter
public class OperationEntry {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private Operation operation;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private Account account;

    @Enumerated(EnumType.STRING)
    private Side side;

    private BigDecimal amount;
}

```

## Листинг А.46 – account-service/src/main/java/by/bsuir/bank/account/domain/PostingBatch.java

```

package by.bsuir.bank.account.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.LocalDateTime;

/** Ключ идемпотентности пакета проводок. */
@Entity
@Getter
@Setter
@NoArgsConstructor
public class PostingBatch {

    @Id
    private String batchKey;

    private LocalDateTime createdAt;

    public PostingBatch(String batchKey) {
        this.batchKey = batchKey;
        this.createdAt = LocalDateTime.now();
    }
}

```

## Листинг А.47 – account-service/src/main/java/by/bsuir/bank/account/domain/Side.java

```

package by.bsuir.bank.account.domain;

/** Сторона счёта, по которой проходит сумма проводки. */
public enum Side {
    DEBIT, CREDIT
}

```

## Листинг А.48 – account- service/src/main/java/by/bsuir/bank/account/repository/AccountRepository.java

```

package by.bsuir.bank.account.repository;

import by.bsuir.bank.account.domain.Account;
import jakarta.persistence.LockModeType;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Lock;
import org.springframework.data.jpa.repository.Query;

import java.util.List;
import java.util.Optional;

```

```

public interface AccountRepository extends JpaRepository<Account, Long> {

    Optional<Account> findByNumber(String number);

    /** Счёт блокируется на время проводки: параллельные операции не теряют обновления оборотов. */
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select a from Account a where a.number = :number")
    Optional<Account> findByNumberForUpdate(String number);

    Optional<Account> findByContractRefAndChartCode(String contractRef, String chartCode);

    /** Собственный счёт банка (касса, фонд развития) в заданной валюте. */
    Optional<Account> findFirstByChartCodeAndCurrencyAndOwnerCodeAndContractRefIsNull(
        String chartCode, String currency, int ownerCode);

    List<Account> findAllByOrderByChartCodeAscNumberAsc();

    List<Account> findByContractRefOrderByNumber(String contractRef);

    List<Account> findByOwnerCodeOrderByNumber(int ownerCode);

    @Query("select coalesce(max(a.ownerSeq), 0) from Account a where a.ownerCode = :ownerCode")
    int maxOwnerSeq(int ownerCode);
}

```

## Листинг А.49 – account- service/src/main/java/by/bsuir/bank/account/repository/BankDayRepository.java

```

package by.bsuir.bank.account.repository;

import by.bsuir.bank.account.domain.BankDay;
import org.springframework.data.jpa.repository.JpaRepository;

public interface BankDayRepository extends JpaRepository<BankDay, Integer> {
}

```

## Листинг А.50 – account- service/src/main/java/by/bsuir/bank/account/repository/ChartAccountRepository.java

```

package by.bsuir.bank.account.repository;

import by.bsuir.bank.account.domain.ChartAccount;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ChartAccountRepository extends JpaRepository<ChartAccount, String> {
}

```

## Листинг А.51 – account- service/src/main/java/by/bsuir/bank/account/repository/CurrencyRepository.java

```

package by.bsuir.bank.account.repository;

import by.bsuir.bank.account.domain.Currency;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CurrencyRepository extends JpaRepository<Currency, String> {
}

```

## Листинг А.52 – account- service/src/main/java/by/bsuir/bank/account/repository/OperationRepository.java

```

package by.bsuir.bank.account.repository;

import by.bsuir.bank.account.domain.Operation;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import java.util.List;

public interface OperationRepository extends JpaRepository<Operation, Long> {
}

```

```

List<Operation> findByBatchKeyOrderById(String batchKey);

@Query("""
    select distinct o from Operation o join o.entries e
    where (:contractRef is null or o.contractRef = :contractRef)
    and (:account is null or e.account.number = :account)
    order by o.id desc""")
List<Operation> search(String contractRef, String account, Pageable page);
}

```

## Листинг А.53 – account- service/src/main/java/by/bsuir/bank/account/repository/PostingBatchRepository.java

```

package by.bsuir.bank.account.repository;

import by.bsuir.bank.account.domain.PostingBatch;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PostingBatchRepository extends JpaRepository<PostingBatch, String> {
}

```

## Листинг А.54 – account-service/src/main/java/by/bsuir/bank/account/dto/AccountReport.java

```

package by.bsuir.bank.account.dto;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

/** Отчёт о состоянии счетов на текущий банковский день с итогами оборотов по валютам. */
public record AccountReport(LocalDate bankDate, List<AccountView> accounts, List<Total> totals) {

    public record Total(String currency, BigDecimal debit, BigDecimal credit) {
    }
}

```

## Листинг А.55 – account-service/src/main/java/by/bsuir/bank/account/dto/AccountView.java

```

package by.bsuir.bank.account.dto;

import by.bsuir.bank.account.domain.Account;
import by.bsuir.bank.common.i18n.Localized;

import java.math.BigDecimal;
import java.time.LocalDate;

public record AccountView(
    String number,
    String chartCode,
    String chartName,
    String activity,
    String activityTitle,
    String currency,
    String name,
    Integer clientId,
    String contractRef,
    BigDecimal debit,
    BigDecimal credit,
    BigDecimal balance,
    LocalDate openedOn
) {

    public static AccountView from(Account a) {
        return new AccountView(a.getNumber(), a.getChart().getCode(), a.getChart().getName(),
            a.getChart().getActivity().name(), a.getChart().getActivity().getTitle(), a.getCurrency(),
            Localized.render(a.getName()), a.getOwnerCode() == 0 ? null : a.getOwnerCode(), a.getContractRef(),
            a.getDebit(), a.getCredit(), a.getBalance(), a.getOpenedOn());
    }
}

```

## Листинг A.56 –

account-service/src/main/java/by/bsuir/bank/account/dto/BatchRequest.java

```
package by.bsuir.bank.account.dto;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.Size;

import java.time.LocalDate;
import java.util.List;

/**
 * Пакет операций, который проводится атомарно: либо все проводки, либо ни одной.
 * batchKey – ключ идемпотентности; bankDate – дата проводок (по умолчанию текущий банковский день).
 */
public record BatchRequest(
    @NotBlank @Size(max = 80) String batchKey,
    @Size(max = 40) String contractRef,
    LocalDate bankDate,
    @NotEmpty @Valid List<OperationRequest> operations
) {
}
```

## Листинг A.57 –

account-service/src/main/java/by/bsuir/bank/account/dto/DayCloseResult.java

```
package by.bsuir.bank.account.dto;

import java.time.LocalDate;
import java.util.List;

/** Итог закрытия дня: новая дата банковского дня и протокол выполненных операций. */
public record DayCloseResult(LocalDate bankDate, List<String> events) {
}
```

## Листинг A.58 –

account-service/src/main/java/by/bsuir/bank/account/dto/EntryRequest.java

```
package by.bsuir.bank.account.dto;

import by.bsuir.bank.account.domain.Side;
import jakarta.validation.constraints.Digits;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;

import java.math.BigDecimal;

public record EntryRequest(
    @NotBlank String account,
    @NotNull Side side,
    @NotNull @Positive @Digits(integer = 17, fraction = 2) BigDecimal amount
) {
}
```

## Листинг A.59 – account-

service/src/main/java/by/bsuir/bank/account/dto/OpenAccountRequest.java

```
package by.bsuir.bank.account.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;

/** Открытие лицевого счёта. clientId не указывается для собственных счетов банка и счетов организаций. */
public record OpenAccountRequest(
    @NotBlank String chartCode,
    @NotBlank String currency,
    @Positive Integer clientId,
    @NotBlank @Size(max = 200) String name,
    @Size(max = 40) String contractRef
) {
}
```

## Листинг A.60 –

account-service/src/main/java/by/bsuir/bank/account/dto/OperationRequest.java

```
package by.bsuir.bank.account.dto;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.Size;

import java.util.List;

public record OperationRequest(
    @NotBlank @Size(max = 200) String description,
    @NotEmpty @Valid List<EntryRequest> entries
) {
}
```

## Листинг A.61 –

account-service/src/main/java/by/bsuir/bank/account/dto/OperationView.java

```
package by.bsuir.bank.account.dto;

import by.bsuir.bank.account.domain.Operation;
import by.bsuir.bank.account.domain.Side;
import by.bsuir.bank.common.i18n.Localized;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

public record OperationView(
    Long id,
    LocalDate bankDate,
    String description,
    String contractRef,
    List<EntryView> entries
) {

    public record EntryView(String account, String accountName, String chartCode, Side side, BigDecimal amount) {
    }

    public static OperationView from(Operation o) {
        // название операции и наименование счёта банка хранятся кодом и раскрываются на языке запроса
        return new OperationView(o.getId(), o.getBankDate(), Localized.render(o.getDescription()), o.getContractRef(),
            o.getEntries().stream()
                .map(e -> new EntryView(e.getAccount().getNumber(), Localized.render(e.getAccount().getName()),
                    e.getAccount().getChart().getCode(), e.getSide(), e.getAmount()))
                .toList());
    }
}
```

## Листинг A.62 – account-

service/src/main/java/by/bsuir/bank/account/service/BankDayService.java

```
package by.bsuir.bank.account.service;

import by.bsuir.bank.account.BankProperties;
import by.bsuir.bank.account.domain.BankDay;
import by.bsuir.bank.account.dto.DayCloseResult;
import by.bsuir.bank.account.repository.BankDayRepository;
import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.common.i18n.Messages;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.web.client.ResourceAccessException;
import org.springframework.web.client.RestClientException;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.locks.ReentrantLock;

/**
```

```

* Банковский день и процедура его закрытия. Сам сервис счетов не знает правил депозитов и кредитов:
* он объявляет сервисам-участникам дату нового дня, а те начисляют проценты и проводят платежи
* через REST этого же сервиса. Дата сдвигается только после успешной обработки всеми участниками.
*/
@Service
@RequiredArgsConstructor
@Slf4j
public class BankDayService {

    private static final int MAX_DAYS = 3660;
    private static final DateTimeFormatter RU_DATE = DateTimeFormatter.ofPattern("dd.MM.yyyy");

    private final BankDayRepository days;
    private final BankProperties properties;
    private final DayCloseNotifier notifier;
    private final ReentrantLock closing = new ReentrantLock();

    /** Название участника на языке запроса: в настройках задан его ключ (deposits, credits). */
    private static String title(BankProperties.Participant participant) {
        return Messages.get("participant." + participant.name());
    }

    public LocalDate current() {
        return days.findById(BankDay.ID).orElseThrow().getBankDate();
    }

    public DayCloseResult close(int count) {
        if (count < 1 || count > MAX_DAYS) {
            throw BankException.invalid(Messages.get("day.range", MAX_DAYS));
        }
        if (!closing.tryLock()) {
            throw new BankException(HttpStatus.CONFLICT, "DAY_CLOSE_IN_PROGRESS", Messages.get("day.inProgress"));
        }
        try {
            List<String> events = new ArrayList<>();
            for (int i = 0; i < count; i++) {
                LocalDate next = current().plusDays(1);
                for (BankProperties.Participant participant : properties.dayClose().participants()) {
                    events.addAll(notify(participant, next));
                }
                BankDay day = days.findById(BankDay.ID).orElseThrow();
                day.setBankDate(next);
                days.saveAndFlush(day);
            }
            return new DayCloseResult(current(), events);
        } finally {
            closing.unlock();
        }
    }

    /**
     * Недоступный участник пропускается: его обработка устойчива к пропуску дней и наверстает их
     * при следующем закрытии. Ошибка обработки, напротив, останавливает закрытие — дата не меняется.
     */
    private List<String> notify(BankProperties.Participant participant, LocalDate date) {
        try {
            return notifier.dayOpened(participant.url(), date);
        } catch (ResourceAccessException e) {
            log.warn("Сервис «{>» недоступен, день {> обработан без него", participant.name(), date);
            return List.of(Messages.get("day.participantUnavailable", RU_DATE.format(date), title(participant)));
        } catch (RestClientException e) {
            throw new BankException(HttpStatus.BAD_GATEWAY, "DAY_CLOSE_FAILED",
                Messages.get("day.participantFailed", title(participant), RU_DATE.format(date)));
        }
    }
}

```

## Листинг А.63 — account-service/src/main/java/by/bsuir/bank/account/service/DayCloseNotifier.java

```

package by.bsuir.bank.account.service;

import by.bsuir.bank.common.HttpApis;
import org.springframework.core.ParameterizedTypeReference;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

import java.time.LocalDate;
import java.util.List;
import java.util.Map;

```

```

/** REST-вызов сервиса-участника закрытия дня: POST {url}/api/internal/day-close. */
@Component
public class DayCloseNotifier {

    private final RestClient http;

    public DayCloseNotifier(RestClient.Builder builder) {
        this.http = HttpApis.withLanguage(builder).build(); // участник отвечает на языке пользователя
    }

    /** Возвращает протокол операций, выполненных участником на дату нового банковского дня. */
    public List<String> dayOpened(String baseUrl, LocalDate date) {
        return http.post()
            .uri(baseUrl + "/api/internal/day-close")
            .body(Map.of("date", date.toString()))
            .retrieve()
            .body(new ParameterizedTypeReference<>() {
            });
    }
}

```

## Листинг А.64 – account-service/src/main/java/by/bsuir/bank/account/service/LedgerInitializer.java

```

package by.bsuir.bank.account.service;

import by.bsuir.bank.account.BankProperties;
import by.bsuir.bank.account.domain.Side;
import by.bsuir.bank.account.dto.AccountView;
import by.bsuir.bank.account.dto.BatchRequest;
import by.bsuir.bank.account.dto.EntryRequest;
import by.bsuir.bank.account.dto.OpenAccountRequest;
import by.bsuir.bank.account.dto.OperationRequest;
import by.bsuir.bank.account.repository.CurrencyRepository;
import by.bsuir.bank.common.il8n.Localized;
import lombok.RequiredArgsConstructor;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

import java.math.BigDecimal;
import java.util.List;

/** При старте открывает собственные счета банка в каждой валюте: кассу и фонд развития со стартовым капиталом. */
@Component
@RequiredArgsConstructor
public class LedgerInitializer implements ApplicationRunner {

    public static final String CASH = "1010";
    public static final String FUND = "7327";

    private final LedgerService ledger;
    private final CurrencyRepository currencies;
    private final BankProperties properties;

    @Override
    public void run(ApplicationArguments args) {
        currencies.findAll().forEach(currency -> {
            String code = currency.getCode();
            ledger.open(new OpenAccountRequest(CASH, code, null, Localized.code("account.cash", code), null));
            AccountView fund = ledger.open(new OpenAccountRequest(FUND, code, null, Localized.code("account.fund", code), null));

            BigDecimal capital = properties.fund().capital().get(code);
            if (capital != null && capital.signum() > 0) {
                ledger.post(new BatchRequest("FUND-CAPITAL-" + code, null, null, List.of(
                    new OperationRequest(Localized.code("posting.fund.capital"),
                        List.of(new EntryRequest(fund.number(), Side.CREDIT, capital))))));
            }
        });
    }
}

```

## Листинг А.65 – account-service/src/main/java/by/bsuir/bank/account/service/LedgerService.java

```

package by.bsuir.bank.account.service;

```

```

import by.bsuir.bank.account.domain.Account;
import by.bsuir.bank.account.domain.AccountNumbers;
import by.bsuir.bank.account.domain.Activity;
import by.bsuir.bank.account.domain.ChartAccount;
import by.bsuir.bank.account.domain.Operation;
import by.bsuir.bank.account.domain.PostingBatch;
import by.bsuir.bank.account.dto.AccountReport;
import by.bsuir.bank.account.dto.AccountView;
import by.bsuir.bank.account.dto.BatchRequest;
import by.bsuir.bank.account.dto.EntryRequest;
import by.bsuir.bank.account.dto.OpenAccountRequest;
import by.bsuir.bank.account.dto.OperationRequest;
import by.bsuir.bank.account.dto.OperationView;
import by.bsuir.bank.account.repository.AccountRepository;
import by.bsuir.bank.account.repository.ChartAccountRepository;
import by.bsuir.bank.account.repository.CurrencyRepository;
import by.bsuir.bank.account.repository.OperationRepository;
import by.bsuir.bank.account.repository.PostingBatchRepository;
import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.common.i18n.Localized;
import by.bsuir.bank.common.i18n.Messages;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.PageRequest;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/** Главная книга: открытие счетов, проведение операций, отчёт о состоянии счетов. */
@Service
@RequiredArgsConstructor
public class LedgerService {

    public static final int BANK_OWNER = 0;

    private final AccountRepository accounts;
    private final ChartAccountRepository chart;
    private final CurrencyRepository currencies;
    private final OperationRepository operations;
    private final PostingBatchRepository batches;
    private final BankDayService bankDay;

    /**
     * Открывает лицевой счёт. Повторный запрос для того же договора и балансового счёта
     * возвращает уже открытый счёт — вызывающий сервис может безопасно повторять запрос.
     */
    @Transactional
    public AccountView open(OpenAccountRequest request) {
        ChartAccount chartAccount = chart.findById(request.chartCode())
            .orElseThrow(() -> BankException.invalid(Messages.get("ledger.noChart", request.chartCode())));
        if (!currencies.existsById(request.currency())) {
            throw BankException.invalid(Messages.get("ledger.noCurrency", request.currency()));
        }
        if (request.contractRef() != null) {
            var existing = accounts.findByContractRefAndChartCode(request.contractRef(), request.chartCode());
            if (existing.isPresent()) {
                return AccountView.from(existing.get());
            }
        }
        int ownerCode = request.clientId() == null ? BANK_OWNER : request.clientId();
        int ownerSeq = accounts.maxOwnerSeq(ownerCode) + 1;

        Account account = new Account();
        account.setNumber(AccountNumbers.generate(chartAccount.getCode(), ownerCode, ownerSeq));
        account.setChart(chartAccount);
        account.setCurrency(request.currency());
        account.setName(request.name());
        account.setOwnerCode(ownerCode);
        account.setOwnerSeq(ownerSeq);
        account.setContractRef(request.contractRef());
        account.setOpenedOn(bankDay.current());
        return AccountView.from(accounts.saveAndFlush(account));
    }

```

```

}

/**
 * Проводит пакет операций в одной транзакции. После каждой операции остаток затронутых
 * счетов не должен стать отрицательным — иначе весь пакет откатывается.
 */
@Transactional
public List<OperationView> post(BatchRequest request) {
    if (batches.existsById(request.batchKey())) {
        return operations.findByBatchKeyOrderByBatchKey(request.batchKey()).stream().map(OperationView::from).toList();
    }
    batches.saveAndFlush(new PostingBatch(request.batchKey()));

    LocalDate date = request.bankDate() != null ? request.bankDate() : bankDay.current();
    Map<String, Account> locked = new LinkedHashMap<>();
    List<Operation> posted = new ArrayList<>();
    for (OperationRequest item : request.operations()) {
        Operation operation = new Operation();
        operation.setBatchKey(request.batchKey());
        operation.setBankDate(date);
        operation.setDescription(item.description());
        operation.setContractRef(request.contractRef());
        operation.setCreatedAt(LocalDate.now());

        Set<Account> touched = new LinkedHashSet<>();
        String currency = null;
        for (EntryRequest entry : item.entries()) {
            Account account = locked.computeIfAbsent(entry.account(), number -> accounts.findByNumberForUpdate(number)
                .orElseThrow(() -> BankException.notFound(Messages.get("ledger.accountNotFound", number))));
            if (currency != null && !currency.equals(account.getCurrency())) {
                throw BankException.invalid(Messages.get("ledger.sameCurrency"));
            }
            currency = account.getCurrency();
            account.apply(entry.side(), entry.amount());
            operation.addEntry(account, entry.side(), entry.amount());
            touched.add(account);
        }
        touched.forEach(account -> checkNotOverdrawn(account, item.description()));
        posted.add(operations.save(operation));
    }
    operations.flush();
    return posted.stream().map(OperationView::from).toList();
}

private static void checkNotOverdrawn(Account account, String operation) {
    if (account.getChart().getActivity() != Activity.ACTIVE_PASSIVE
        && account.getBalance().compareTo(BigDecimal.ZERO) < 0) {
        throw new BankException(HttpStatus.CONFLICT, "INSUFFICIENT_FUNDS",
            Messages.get("ledger.insufficient", account.getNumber(), Localized.render(account.getName()),
                Localized.render(operation)));
    }
}

@Transactional(readOnly = true)
public AccountView get(String number) {
    return accounts.findByNumber(number).map(AccountView::from)
        .orElseThrow(() -> BankException.notFound(Messages.get("ledger.accountNotFound", number)));
}

/** Собственный счёт банка — касса (1010) или фонд развития (7327) — в заданной валюте. */
@Transactional(readOnly = true)
public AccountView system(String chartCode, String currency) {
    return accounts.findFirstByChartCodeAndCurrencyAndOwnerCodeAndContractRefIsNull(chartCode, currency, BANK_OWNER)
        .map(AccountView::from)
        .orElseThrow(() -> BankException.notFound(
            Messages.get("ledger.systemAccountNotFound", chartCode, currency)));
}

@Transactional(readOnly = true)
public List<AccountView> find(Integer clientId, String contractRef) {
    List<Account> found;
    if (contractRef != null) {
        found = accounts.findByContractRefOrderByNumber(contractRef);
    } else if (clientId != null) {
        found = accounts.findByOwnerCodeOrderByNumber(clientId);
    } else {
        found = accounts.findAllByOrderByChartCodeAscNumberAsc();
    }
    return found.stream().map(AccountView::from).toList();
}

@Transactional(readOnly = true)

```

```

public AccountReport report() {
    List<AccountView> all = find(null, null);
    Map<String, BigDecimal[]> sums = new LinkedHashMap<>();
    for (AccountView account : all) {
        BigDecimal[] sum = sums.computeIfAbsent(account.currency(), c -> new BigDecimal[]{BigDecimal.ZERO, BigDecimal.ZERO});
        sum[0] = sum[0].add(account.debit());
        sum[1] = sum[1].add(account.credit());
    }
    List<AccountReport.Total> totals = sums.entrySet().stream()
        .map(e -> new AccountReport.Total(e.getKey(), e.getValue()[0], e.getValue()[1]))
        .toList();
    return new AccountReport(bankDay.current(), all, totals);
}

@Transactional(readOnly = true)
public List<OperationView> journal(String contractRef, String account, int limit) {
    return operations.search(contractRef, account, PageRequest.of(0, limit)).stream()
        .map(OperationView::from)
        .toList();
}
}

```

## Листинг А.66 – account- service/src/main/java/by/bsuir/bank/account/web/BankDayController.java

```

package by.bsuir.bank.account.web;

import by.bsuir.bank.account.dto.DayCloseResult;
import by.bsuir.bank.account.service.BankDayService;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDate;
import java.util.Map;

@RestController
@RequestMapping("/api/bank-day")
@RequiredArgsConstructor
public class BankDayController {

    private final BankDayService bankDay;

    @GetMapping
    public Map<String, LocalDate> current() {
        return Map.of("date", bankDay.current());
    }

    /** Процедура «Закрытие банковского дня»: закрывает указанное число дней подряд. */
    @PostMapping("/close")
    public DayCloseResult close(@RequestParam(defaultValue = "1") int days) {
        return bankDay.close(days);
    }
}

```

## Листинг А.67 – account-service/src/main/java/by/bsuir/bank/account/web/LedgerController.java

```

package by.bsuir.bank.account.web;

import by.bsuir.bank.account.domain.ChartAccount;
import by.bsuir.bank.account.domain.Currency;
import by.bsuir.bank.account.dto.AccountReport;
import by.bsuir.bank.account.dto.AccountView;
import by.bsuir.bank.account.dto.BatchRequest;
import by.bsuir.bank.account.dto.OpenAccountRequest;
import by.bsuir.bank.account.dto.OperationView;
import by.bsuir.bank.account.repository.ChartAccountRepository;
import by.bsuir.bank.account.repository.CurrencyRepository;
import by.bsuir.bank.account.service.LedgerService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Sort;
import org.springframework.http.HttpStatus;

```

```

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api")
@RequiredArgsConstructor
public class LedgerController {

    private final LedgerService ledger;
    private final ChartAccountRepository chart;
    private final CurrencyRepository currencies;

    @GetMapping("/currencies")
    public List<Currency> currencies() {
        return currencies.findAll(Sort.by("code"));
    }

    @GetMapping("/chart-of-accounts")
    public List<ChartAccount> chartOfAccounts() {
        return chart.findAll(Sort.by("code"));
    }

    @GetMapping("/accounts")
    public List<AccountView> accounts(@RequestParam(required = false) Integer clientId,
                                     @RequestParam(required = false) String contractRef) {
        return ledger.find(clientId, contractRef);
    }

    @GetMapping("/accounts/system")
    public AccountView systemAccount(@RequestParam String chartCode, @RequestParam String currency) {
        return ledger.system(chartCode, currency);
    }

    @GetMapping("/accounts/{number}")
    public AccountView account(@PathVariable String number) {
        return ledger.get(number);
    }

    @PostMapping("/accounts")
    @ResponseStatus(HttpStatus.CREATED)
    public AccountView open(@Valid @RequestBody OpenAccountRequest request) {
        return ledger.open(request);
    }

    @PostMapping("/operations")
    @ResponseStatus(HttpStatus.CREATED)
    public List<OperationView> post(@Valid @RequestBody BatchRequest request) {
        return ledger.post(request);
    }

    @GetMapping("/operations")
    public List<OperationView> journal(@RequestParam(required = false) String contractRef,
                                     @RequestParam(required = false) String account,
                                     @RequestParam(defaultValue = "200") int limit) {
        return ledger.journal(contractRef, account, Math.min(limit, 1000));
    }

    @GetMapping("/report")
    public AccountReport report() {
        return ledger.report();
    }
}

```

## Листинг А.68 – account-service/src/main/resources/static/index.html

```

<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>BankEt</title>
    <link rel="stylesheet" href="/ui/bank.css">

```

```

</head>
<body>
  <div id="app" v-cloak></div>

  <script src="/webjars/vue/dist/vue.global.prod.js"></script>
  <script src="/ui/bank.js"></script>
  <script src="/ui/il8n.js"></script>
  <script src="/il8n.js"></script>
  <script src="/app.js"></script>
</body>
</html>

```

## Листинг А.69 – account-service/src/main/resources/static/app.js

*/\* Web-клиент главной книги: отчёт о состоянии счетов, журнал проводок, план счетов, закрытие дня. \*/*

```

const app = Bank.createApp({
  data() {
    return {
      tab: 'report',           // report | journal | chart
      report: { bankDate: null, accounts: [], totals: [] },
      journal: [],
      chart: [],
      currency: 'BYN',
      events: [],
      error: '',
    };
  },
  computed: {
    currencies() {
      return [...new Set(this.report.accounts.map(a => a.currency))];
    },
    accounts() {
      return this.report.accounts.filter(a => a.currency === this.currency);
    },
    total() {
      return this.report.totals.find(t => t.currency === this.currency) || { debit: 0, credit: 0 };
    },
    fund() {
      return this.accounts.find(a => a.chartCode === '7327' && !a.contractRef);
    },
    cash() {
      return this.accounts.find(a => a.chartCode === '1010');
    },
  },
  async mounted() {
    try {
      this.chart = await Bank.api('GET', '/api/chart-of-accounts');
      await this.reload();
    } catch (e) {
      this.error = e.message;
    }
  },
  methods: {
    async reload() {
      this.report = await Bank.api('GET', '/api/report');
      this.journal = await Bank.api('GET', '/api/operations?limit=300');
    },
    async closed(result) {
      this.error = '';
      this.events = result.events;
      await this.reload();
    },
    date: Bank.date,
    money: Bank.money,
  },
  template: `
    <bank-nav active="accounts"></bank-nav>
    <main>
      <div class="toolbar">
        <div>
          <h1>{{ t('accounts.title') }}</h1>
          <p class="subtitle">{{ t('accounts.subtitle') }}</p>
        </div>
        <bank-day @changed="closed" @failed="e => error = e.message"></bank-day>
      </div>

      <div v-if="error" id="error" class="alert error">{{ error }}</div>

      <div class="stats">

```

```

<div class="stat"><div class="label">{{ t('accounts.reportDate') }}</div><div class="value">{{ date(report.bankDate) }}</div></div>
<div class="stat"><div class="label">{{ t('accounts.count') }}</div><div class="value" id="accounts-count">{{ accounts.length }}</div></div>
<div class="stat"><div class="label">{{ t('accounts.fund', currency) }}</div><div class="value">{{ fund ? money(fund.balance) : '-' }}</div></div>
<div class="stat"><div class="label">{{ t('accounts.cash', currency) }}</div><div class="value">{{ cash ? money(cash.balance) : '-' }}</div></div>
</div>

<div v-if="events.length" id="events" class="alert info">
  <b>{{ t('events.title') }}</b>
  <div v-for="(event, i) in events.slice(-12)" :key="i">{{ event }}</div>
  <div v-if="events.length > 12" class="muted">{{ t('events.more', events.length) }}</div>
</div>

<div class="toolbar">
  <div style="display: flex; gap: 6px">
    <button id="tab-report" :class="{ primary: tab === 'report' }" @click="tab = 'report'">{{ t('accounts.tabReport') }}</button>
    <button id="tab-journal" :class="{ primary: tab === 'journal' }" @click="tab = 'journal'">{{ t('accounts.tabJournal') }}</button>
    <button id="tab-chart" :class="{ primary: tab === 'chart' }" @click="tab = 'chart'">{{ t('accounts.tabChart') }}</button>
  </div>
  <div v-if="tab === 'report'" style="display: flex; gap: 6px">
    <button v-for="c in currencies" :key="c" class="small" :class="{ primary: c === currency }" @click="currency = c">{{ c }}</button>
  </div>
</div>

<div v-if="tab === 'report'" class="card">
  <table id="report">
    <thead>
      <tr>
        <th>{{ t('table.account') }}</th><th>{{ t('table.code') }}</th><th>{{ t('accounts.name') }}</th>
        <th>{{ t('table.activity') }}</th><th>{{ t('table.contract') }}</th>
        <th class="num">{{ t('table.debit') }}</th><th class="num">{{ t('table.credit') }}</th><th class="num">{{ t('table.balance') }}</th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="a in accounts" :key="a.number">
        <td class="mono">{{ a.number }}</td>
        <td class="mono">{{ a.chartCode }}</td>
        <td>{{ a.name }}<div class="muted" style="font-size: 12.5px">{{ a.chartName }}</div></td>
        <td><span class="badge" :class="{ ok: a.activity === 'ACTIVE' }">{{ a.activityTitle }}</span></td>
        <td class="mono">{{ a.contractRef }}</td>
        <td class="num">{{ money(a.debit) }}</td>
        <td class="num">{{ money(a.credit) }}</td>
        <td class="num"><b>{{ money(a.balance) }}</b></td>
      </tr>
    </tbody>
    <tfoot>
      <tr>
        <td colspan="5">{{ t('accounts.totals', currency) }}</td>
        <td class="num">{{ money(total.debit) }}</td>
        <td class="num">{{ money(total.credit) }}</td>
        <td colspan="2"></td>
      </tr>
    </tfoot>
  </table>
</div>

<div v-if="tab === 'journal'" class="card">
  <table id="journal">
    <thead>
      <tr>
        <th>{{ t('table.date') }}</th><th>{{ t('table.operation') }}</th><th>{{ t('table.contract') }}</th>
        <th>{{ t('table.accountShort') }}</th><th class="num">{{ t('table.debit') }}</th><th class="num">{{ t('table.credit') }}</th></tr>
    </thead>
    <tbody>
      <template v-for="op in journal" :key="op.id">
        <tr v-for="(entry, i) in op.entries" :key="op.id + '-' + i">
          <td>{{ i === 0 ? op.id : '' }}</td>
          <td>{{ i === 0 ? date(op.bankDate) : '' }}</td>
          <td>{{ i === 0 ? op.description : '' }}</td>
          <td class="mono">{{ i === 0 ? op.contractRef : '' }}</td>
          <td class="mono">{{ entry.account }} <span class="muted">{{ entry.accountName }}</span></td>
          <td class="num">{{ entry.side === 'DEBIT' ? money(entry.amount) : '' }}</td>
          <td class="num">{{ entry.side === 'CREDIT' ? money(entry.amount) : '' }}</td>
        </tr>
      </template>
    </tbody>
  </table>
</div>

```

```

        </template>
      </tbody>
    </table>
    <div v-if="!journal.length" class="empty">{{ t('accounts.noPostings') }}</div>
  </div>

  <div v-if="tab === 'chart'" class="card">
    <table id="chart">
      <thead><tr><th>{{ t('accounts.chartCode') }}</th><th>{{ t('accounts.chartName') }}</th><th>{{ t('table.activity') }}</th>
    </tr></thead>
    <tbody>
      <tr v-for="c in chart" :key="c.code">
        <td class="mono">{{ c.code }}</td>
        <td>{{ c.name }}</td>
        <td>{{ c.activityTitle }}</td>
      </tr>
    </tbody>
  </table>
</div>
</main>',
}, 'accounts.page');

app.mount('#app');

```

## Листинг А.70 – account-service/src/main/resources/static/i18n.js

```

/* Словарь web-клиента главной книги: русский, английский, белорусский. */
Bank.addMessages({
  ru: {
    'accounts.page': 'Счета',
    'accounts.title': 'Счета банка',
    'accounts.subtitle': 'Отчёт о состоянии счетов и процедура «Закрытие банковского дня»',
    'accounts.reportDate': 'Отчёт на дату',
    'accounts.count': 'Счетов в отчёте',
    'accounts.fund': 'Фонд развития, {0}',
    'accounts.cash': 'Касса, {0}',
    'accounts.tabReport': 'Отчёт по счетам',
    'accounts.tabJournal': 'Журнал проводок',
    'accounts.tabChart': 'План счетов',
    'accounts.name': 'Название счёта',
    'accounts.totals': 'Итого обороты, {0}',
    'accounts.noPostings': 'Проводок пока нет',
    'accounts.chartCode': 'Балансовый счёт',
    'accounts.chartName': 'Наименование',
  },
  en: {
    'accounts.page': 'Accounts',
    'accounts.title': 'Bank accounts',
    'accounts.subtitle': 'Account status report and the "Closing of the banking day" procedure',
    'accounts.reportDate': 'Report date',
    'accounts.count': 'Accounts in the report',
    'accounts.fund': 'Development fund, {0}',
    'accounts.cash': 'Cash desk, {0}',
    'accounts.tabReport': 'Account report',
    'accounts.tabJournal': 'Posting journal',
    'accounts.tabChart': 'Chart of accounts',
    'accounts.name': 'Account name',
    'accounts.totals': 'Total turnover, {0}',
    'accounts.noPostings': 'No postings yet',
    'accounts.chartCode': 'Balance account',
    'accounts.chartName': 'Name',
  },
  be: {
    'accounts.page': 'Рахункі',
    'accounts.title': 'Рахункі банка',
    'accounts.subtitle': 'Справаздача аб стане рахункаў і працэдура «Закрыццё банкаўскага дня»',
    'accounts.reportDate': 'Справаздача на дату',
    'accounts.count': 'Рахункаў у справаздачы',
    'accounts.fund': 'Фонд развіцця, {0}',
    'accounts.cash': 'Каса, {0}',
    'accounts.tabReport': 'Справаздача па рахунках',
    'accounts.tabJournal': 'Журнал праводак',
    'accounts.tabChart': 'План рахункаў',
    'accounts.name': 'Назва рахунку',
    'accounts.totals': 'Разам абароты, {0}',
    'accounts.noPostings': 'Праводак пакуль няма',
    'accounts.chartCode': 'Балансавы рахунак',
    'accounts.chartName': 'Найменне',
  },
});

```

## Листинг A.71 –

account-service/src/test/java/by/bsuir/bank/account/AccountNumbersTest.java

```
package by.bsuir.bank.account;

import by.bsuir.bank.account.domain.AccountNumbers;
import by.bsuir.bank.account.domain.Activity;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import java.math.BigDecimal;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

/** Модульные тесты правил формирования номера счёта и расчёта сальдо. */
class AccountNumbersTest {

    @ParameterizedTest
    @CsvSource({
        "1010, 0, 1, 1010000000019",
        "7327, 0, 2, 7327000000029",
        "3014, 4, 1, 301400004001",
        "2400, 12345, 17, 240012345017",
    })
    void numberConsistsOfBalanceAccountClientCodeSequenceAndKey(String chart, int client, int seq, String prefix) {
        String number = AccountNumbers.generate(chart, client, seq);

        assertThat(number).hasSize(13).containsOnlyDigits().startsWith(prefix.substring(0, 12));
        assertThat(number.substring(0, 4)).isEqualTo(chart);
        assertThat(Integer.parseInt(number.substring(4, 9))).isEqualTo(client);
        assertThat(Integer.parseInt(number.substring(9, 12))).isEqualTo(seq);
        assertThat(AccountNumbers.isValid(number)).isTrue();
    }

    @Test
    void checkDigitDetectsSingleDigitError() {
        String number = AccountNumbers.generate("3014", 4, 1);
        for (int position = 0; position < 12; position++) {
            char wrong = number.charAt(position) == '9' ? '0' : (char) (number.charAt(position) + 1);
            String corrupted = number.substring(0, position) + wrong + number.substring(position + 1);
            assertThat(AccountNumbers.isValid(corrupted)).as("ошибка в разряде %d", position + 1).isFalse();
        }
    }

    @Test
    void rejectsMalformedNumbers() {
        assertThat(AccountNumbers.isValid(null)).isFalse();
        assertThat(AccountNumbers.isValid("30140000400")).isFalse();
        assertThat(AccountNumbers.isValid("30140000400A1")).isFalse();
        assertThatThrownBy(() -> AccountNumbers.generate("3014", 100_000, 1)).isInstanceOf(IllegalArgumentException.class);
        assertThatThrownBy(() -> AccountNumbers.generate("3014", 1, 1000)).isInstanceOf(IllegalArgumentException.class);
        assertThatThrownBy(() -> AccountNumbers.generate("30", 1, 1)).isInstanceOf(IllegalArgumentException.class);
    }

    @Test
    void balanceDependsOnActivity() {
        BigDecimal debit = new BigDecimal("700.00");
        BigDecimal credit = new BigDecimal("1000.00");

        assertThat(Activity.ACTIVE.balance(debit, credit)).isEqualByComparingTo("-300.00");
        assertThat(Activity.PASSIVE.balance(debit, credit)).isEqualByComparingTo("300.00");
        assertThat(Activity.ACTIVE_PASSIVE.balance(debit, credit)).isEqualByComparingTo("-300.00");
    }
}
```

## Листинг A.72 –

account-service/src/test/java/by/bsuir/bank/account/BankDayIT.java

```
package by.bsuir.bank.account;

import by.bsuir.bank.account.service.BankDayService;
import by.bsuir.bank.account.service.DayCloseNotifier;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.HttpStatus;
```

```

import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.web.client.HttpServerErrorException;
import org.springframework.web.client.ResourceAccessException;

import java.time.LocalDate;
import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;
import static org.hamcrest.Matchers.containsString;
import static org.hamcrest.Matchers.hasItem;
import static org.hamcrest.Matchers.hasSize;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.ArgumentMatchers.contains;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.times;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/** Интеграционные тесты REST и процедуры закрытия дня; сервисы-участники заменены заглушкой. */
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@AutoConfigureMockMvc
class BankDayIT {

    @Autowired
    private MockMvc mvc;
    @Autowired
    private BankDayService bankDay;
    @MockitoBean
    private DayCloseNotifier notifier;

    @Test
    @DisplayName("Закрытие дня объявляет новую дату каждому участнику и сдвигает банковский день")
    void closesSeveralDays() throws Exception {
        LocalDate start = bankDay.current();
        when(notifier.dayOpened(anyString(), any())).thenReturn(List.of("Начислены проценты"));

        mvc.perform(post("/api/bank-day/close").param("days", "3"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.bankDate").value(start.plusDays(3).toString()))
            .andExpect(jsonPath("$.events", hasSize(6)));

        verify(notifier).dayOpened(contains("8083"), eq(start.plusDays(1)));
        verify(notifier).dayOpened(contains("8084"), eq(start.plusDays(3)));
        verify(notifier, times(6)).dayOpened(anyString(), any());
        mvc.perform(get("/api/bank-day")).andExpect(jsonPath("$.date").value(start.plusDays(3).toString()));
    }

    @Test
    @DisplayName("Недоступный участник пропускается, день закрывается")
    void unavailableParticipantIsSkipped() throws Exception {
        LocalDate start = bankDay.current();
        when(notifier.dayOpened(anyString(), any())).thenThrow(new ResourceAccessException("Connection refused"));

        mvc.perform(post("/api/bank-day/close"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.bankDate").value(start.plusDays(1).toString()))
            .andExpect(jsonPath("$.events", hasItem(containsString("недоступен"))));
    }

    @Test
    @DisplayName("Ошибка участника останавливает закрытие: дата не меняется")
    void participantFailureKeepsDayOpen() throws Exception {
        LocalDate start = bankDay.current();
        when(notifier.dayOpened(anyString(), any())).thenThrow(new HttpServerErrorException(HttpStatus.INTERNAL_SERVER_ERROR));

        mvc.perform(post("/api/bank-day/close"))
            .andExpect(status().isBadGateway())
            .andExpect(jsonPath("$.code").value("DAY_CLOSE_FAILED"));

        assertThat(bankDay.current()).isEqualTo(start);
    }

    @Test
    @DisplayName("Количество дней проверяется")
    void rejectsWrongDaysCount() throws Exception {

```

```

        mvc.perform(post("/api/bank-day/close").param("days", "0")).andExpect(status().isBadRequest());
    }

    @Test
    @DisplayName("Справочники «План счетов» и валют, отчёт по счетам")
    void dictionariesAndReport() throws Exception {
        mvc.perform(get("/api/chart-of-accounts"))
            .andExpect(jsonPath("$.code == '7327']").activity().value("PASSIVE"))
            .andExpect(jsonPath("$.code == '1010']").activity().value("ACTIVE"));
        mvc.perform(get("/api/currencies")).andExpect(jsonPath("$.code", hasItem("BYN")));
        mvc.perform(get("/api/report"))
            .andExpect(jsonPath("$.accounts", hasSize(6)))
            .andExpect(jsonPath("$.totals[?(@.currency == 'BYN')].credit").value(10000000000.0));
    }

    @Test
    @DisplayName("Запрос на проводку проверяется: сумма положительна, счёт существует")
    void validatesPostingRequest() throws Exception {
        String negative = """
            {"batchKey": "K1", "operations": [{"description": "Тест",
            "entries": [{"account": "1010000000019", "side": "DEBIT", "amount": -5}]}]}""";
        mvc.perform(post("/api/operations").contentType(MediaType.APPLICATION_JSON).content(negative))
            .andExpect(status().isBadRequest());

        String unknown = """
            {"batchKey": "K2", "operations": [{"description": "Тест",
            "entries": [{"account": "0000000000000", "side": "DEBIT", "amount": 5}]}]}""";
        mvc.perform(post("/api/operations").contentType(MediaType.APPLICATION_JSON).content(unknown))
            .andExpect(status().isNotFound());
    }
}

```

## Листинг А.73 – account-service/src/test/java/by/bsuir/bank/account/LedgerLanguageIT.java

```

package by.bsuir.bank.account;

import by.bsuir.bank.account.service.DayCloseNotifier;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.web.client.ResourceAccessException;

import static org.hamcrest.Matchers.hasItem;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/** Интеграционные тесты локализации главной книги: справочники, хранимые коды проводок, сообщения. */
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@AutoConfigureMockMvc
class LedgerLanguageIT {

    @Autowired
    private MockMvc mvc;

    @MockitoBean
    private DayCloseNotifier notifier;

    @Test
    @DisplayName("Отчёт по счетам: наименования счетов банка, плана счетов и активность — на языке запроса")
    void reportIsLocalized() throws Exception {
        mvc.perform(get("/api/report"))
            .andExpect(jsonPath("$.accounts[0].name").value("Касса банка, BYN"))
            .andExpect(jsonPath("$.accounts[0].chartName").value("Денежные средства в кассе"))
            .andExpect(jsonPath("$.accounts[0].activityTitle").value("Активный"));
        mvc.perform(get("/api/report").header(HttpHeaders.ACCEPT_LANGUAGE, "en"))
            .andExpect(jsonPath("$.accounts[0].name").value("Bank cash desk, BYN"))
            .andExpect(jsonPath("$.accounts[0].chartName").value("Cash on hand"))
            .andExpect(jsonPath("$.accounts[0].activityTitle").value("Active"));
        mvc.perform(get("/api/report").header(HttpHeaders.ACCEPT_LANGUAGE, "be"))
    }
}

```

```

        .andExpect(jsonPath("$.accounts[0].name").value("Каса банка, BYN"))
        .andExpect(jsonPath("$.accounts[0].chartName").value("Грашовыя сродкі ў касе"))
        .andExpect(jsonPath("$.accounts[0].activityTitle").value("Актыўны"));
    }

    @Test
    @DisplayName("Одна и та же проводка читается на трёх языках: в базе хранится её код")
    void postingDescriptionIsRenderedOnRead() throws Exception {
        mvc.perform(get("/api/operations"))
            .andExpect(jsonPath("$[*].description", hasItem("Формирование фонда развития банка")));
        mvc.perform(get("/api/operations").header(HttpHeaders.ACCEPT_LANGUAGE, "en"))
            .andExpect(jsonPath("$[*].description", hasItem("Bank development fund formed")))
            .andExpect(jsonPath("$[0].entries[0].accountName").value("Bank development fund, EUR"));
        mvc.perform(get("/api/operations").header(HttpHeaders.ACCEPT_LANGUAGE, "be"))
            .andExpect(jsonPath("$[*].description", hasItem("Фарміраванне фонду развіцця банка")));
    }

    @Test
    @DisplayName("Справочники «План счетов» и валют на английском и белорусском")
    void dictionaries() throws Exception {
        mvc.perform(get("/api/chart-of-accounts").header(HttpHeaders.ACCEPT_LANGUAGE, "en"))
            .andExpect(jsonPath("$[?(@.code == '7327')].name").value("Bank development fund"))
            .andExpect(jsonPath("$[?(@.code == '7327')].activityTitle").value("Passive"))
            .andExpect(jsonPath("$[0].nameBe").doesNotExist());
        mvc.perform(get("/api/currencies").header(HttpHeaders.ACCEPT_LANGUAGE, "be"))
            .andExpect(jsonPath("$[?(@.code == 'BYN')].name").value("Беларускі рубель"));
    }

    @Test
    @DisplayName("Отказ в проводке и протокол закрытия дня – на языке запроса")
    void errorsAndEvents() throws Exception {
        String overdraft = """
            {
                "batchKey": "LANG-1",
                "operations": [
                    {
                        "description": "@posting.deposit.cashOut",
                        "entries": [
                            {
                                "account": "1010000000019",
                                "side": "CREDIT",
                                "amount": 5
                            }
                        ]
                    }
                ]
            }
        """
        mvc.perform(post("/api/operations").header(HttpHeaders.ACCEPT_LANGUAGE, "en")
            .contentType(MediaType.APPLICATION_JSON).content(overdraft))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.message").value("Insufficient funds on account 1010000000019 "
                + "(" + "Bank cash desk, BYN" + ") for operation \"Cash paid out from the cash desk\""));

        when(notifier.dayOpened(anyString(), any())).thenThrow(new ResourceAccessException("Connection refused"));
        mvc.perform(post("/api/bank-day/close").header(HttpHeaders.ACCEPT_LANGUAGE, "be"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.events[0]").value(org.hamcrest.Matchers.endsWith(
                "Сэрвіс «Дэпазіты» недаступны, апрацоўка адкладзена")));
    }
}

```

## Листинг А.74 – account-service/src/test/java/by/bsuir/bank/account/LedgerServiceIT.java

```

package by.bsuir.bank.account;

import by.bsuir.bank.account.domain.AccountNumbers;
import by.bsuir.bank.account.domain.Side;
import by.bsuir.bank.account.dto.AccountView;
import by.bsuir.bank.account.dto.BatchRequest;
import by.bsuir.bank.account.dto.EntryRequest;
import by.bsuir.bank.account.dto.OpenAccountRequest;
import by.bsuir.bank.account.dto.OperationRequest;
import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.account.service.LedgerService;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.jdbc.core.JdbcTemplate;

import java.math.BigDecimal;
import java.util.List;
import java.util.UUID;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

/** Интеграционные тесты главной книги на H2: счета, проводки, атомарность и идемпотентность. */
@SpringBootTest(properties = {
    "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1",
    "bank.fund.capital.BYN=1000000"})
class LedgerServiceIT {

```

```

@Autowired
private LedgerService ledger;
@Autowired
private JdbcTemplate jdbc;

@Test
@DisplayName("При старте открыты касса и фонд развития со стартовым капиталом")
void systemAccountsAreOpenedOnStartup() {
    AccountView cash = ledger.system("1010", "BYN");
    AccountView fund = ledger.system("7327", "BYN");

    assertThat(cash.activity()).isEqualTo("ACTIVE");
    assertThat(fund.activity()).isEqualTo("PASSIVE");
    assertThat(fund.credit()).isEqualToByComparingTo("1000000.00");
    assertThat(fund.balance()).isEqualToByComparingTo("1000000.00");
    assertThat(ledger.report().accounts()).hasSizeGreaterThanOrEqualTo(6);
}

@Test
@DisplayName("Номер нового счёта: балансовый счёт, код клиента, порядковый номер, ключ")
void opensClientAccounts() {
    AccountView main = ledger.open(new OpenAccountRequest("3404", "BYN", 41, "Иванов Иван Иванович", "D-41"));
    AccountView interest = ledger.open(new OpenAccountRequest("3470", "BYN", 41, "Иванов Иван Иванович", "D-41"));

    assertThat(main.number()).startsWith("340400041001");
    assertThat(interest.number()).startsWith("347000041002");
    assertThat(AccountNumbers.isValid(main.number())).isTrue();
    assertThat(main.balance()).isEqualToByComparingTo("0");
    assertThat(ledger.find(41, null)).hasSize(2);
    assertThat(ledger.find(null, "D-41").extracting(AccountView::chartCode).containsExactly("3404", "3470"));
}

@Test
@DisplayName("Повторное открытие счёта по тому же договору возвращает существующий счёт")
void openingIsIdempotentPerContract() {
    OpenAccountRequest request = new OpenAccountRequest("3414", "BYN", 42, "Петров Пётр Петрович", "D-42");

    assertThat(ledger.open(request).number()).isEqualTo(ledger.open(request).number());
    assertThat(ledger.find(42, null)).hasSize(1);
}

@Test
@DisplayName("Счёт нельзя открыть по коду, которого нет в плане счетов, или в неизвестной валюте")
void rejectsUnknownChartCodeAndCurrency() {
    assertThatThrownBy(() -> ledger.open(new OpenAccountRequest("9999", "BYN", 1, "x", null)))
        .isInstanceOf(BankException.class).hasMessageContaining("нет в плане счетов");
    assertThatThrownBy(() -> ledger.open(new OpenAccountRequest("3014", "RUB", 1, "x", null)))
        .isInstanceOf(BankException.class).hasMessageContaining("нет в справочнике");
}

@Test
@DisplayName("Проводка меняет обороты; сальдо активного счёта растёт по дебету, пассивного – по кредиту")
void postingChangesTurnovers() {
    AccountView cash = ledger.system("1010", "USD");
    AccountView client = ledger.open(new OpenAccountRequest("3014", "USD", 43, "Сидоров", "T-43"));

    ledger.post(batch(
        operation("Внесение денег в кассу", entry(cash, Side.DEBIT, "500")),
        operation("Перевод денег с кассы на текущий счёт", entry(cash, Side.CREDIT, "500"), entry(client, Side.CREDIT, "500"))));

    assertThat(ledger.get(cash.number()).satisfies(a -> {
        assertThat(a.debit()).isEqualToByComparingTo("500");
        assertThat(a.credit()).isEqualToByComparingTo("500");
        assertThat(a.balance()).isEqualToByComparingTo("0");
    }));
    assertThat(ledger.get(client.number()).balance()).isEqualToByComparingTo("500");
    assertThat(ledger.journal("T-43", null, 10)).hasSize(2);
}

@Test
@DisplayName("При нехватке средств откатывается весь пакет операций")
void batchIsAtomic() {
    AccountView cash = ledger.system("1010", "EUR");
    AccountView client = ledger.open(new OpenAccountRequest("3014", "EUR", 44, "Кузнецов", "T-44"));
    String key = UUID.randomUUID().toString();

    assertThatThrownBy(() -> ledger.post(new BatchRequest(key, "T-44", null, List.of(
        operation("Внесение денег в кассу", entry(cash, Side.DEBIT, "100")),
        operation("Списание со счёта клиента", entry(client, Side.DEBIT, "1"), entry(cash, Side.DEBIT, "1")))))
        .isInstanceOf(Satisfying(BankException.class, e -> assertThat(e.getCode()).isEqualTo("INSUFFICIENT_FUNDS")));
}

```

```

        assertThat(ledger.get(cash.number()).debit()).isEqualByComparingTo("0");
        assertThat(ledger.journal("T-44", null, 10)).isEmpty();
        assertThat(jdbc.queryForObject("select count(*) from posting_batch where batch_key = ?", Integer.class, key)).isZero();
    }

    @Test
    @DisplayName("Повтор пакета с тем же ключом не проводит операции второй раз")
    void postingIsIdempotent() {
        AccountView cash = ledger.system("1010", "BYN");
        BigDecimal before = ledger.get(cash.number()).debit();
        BatchRequest request = batch(operation("Внесение денег в кассу", entry(cash, Side.DEBIT, "250")));

        var first = ledger.post(request);
        var second = ledger.post(request);

        assertThat(second).extracting("id").isEqualTo(first.stream().map(o -> o.id()).toList());
        assertThat(ledger.get(cash.number()).debit()).isEqualByComparingTo(before.add(new BigDecimal("250")));
    }

    @Test
    @DisplayName("В одной операции нельзя смешивать счета разных валют")
    void rejectsMixedCurrencies() {
        AccountView byn = ledger.system("1010", "BYN");
        AccountView usd = ledger.system("1010", "USD");

        assertThatThrownBy(() -> ledger.post(batch(
            operation("Перевод", entry(byn, Side.DEBIT, "10"), entry(usd, Side.DEBIT, "10"))))
            .isInstanceOf(BankException.class).hasMessageContaining("в одной валюте"));
    }

    private static BatchRequest batch(OperationRequest... operations) {
        return new BatchRequest(UUID.randomUUID().toString(), "T-43", null, List.of(operations));
    }

    private static OperationRequest operation(String description, EntryRequest... entries) {
        return new OperationRequest(description, List.of(entries));
    }

    private static EntryRequest entry(AccountView account, Side side, String amount) {
        return new EntryRequest(account.number(), side, new BigDecimal(amount));
    }
}

```

## Микросервис «Депозиты» (deposit-service)

### Листинг А.75 – deposit-service/pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>by.bsuir.bank</groupId>
        <artifactId>bank</artifactId>
        <version>1.0.0</version>
    </parent>

    <artifactId>deposit-service</artifactId>
    <name>deposit-service</name>
    <description>JP2. Микросервис депозитных операций с физическими лицами</description>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
    </dependencies>

```

```

    <dependency>
      <groupId>by.bsuir.bank</groupId>
      <artifactId>bank-common</artifactId>
    </dependency>
    <dependency>
      <groupId>by.bsuir.bank</groupId>
      <artifactId>ui-kit</artifactId>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>by.bsuir.bank</groupId>
      <artifactId>bank-common</artifactId>
      <version>${project.version}</version>
      <type>test-jar</type>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-failsafe-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>

```

## Листинг А.76 – deposit-service/src/main/resources/application.yml

```

server:
  port: ${DEPOSIT_SERVICE_PORT:8083}

spring:
  application:
    name: deposit-service
  datasource:
    url: jdbc:h2:mem:deposits;DB_CLOSE_DELAY=-1
    username: sa
  sql:
    init:
      mode: always
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
  h2:
    console:
      enabled: true
  http:
    client:
      connect-timeout: 2s
      read-timeout: 120s

bank:
  services:
    clients: ${CLIENT_SERVICE_URL:http://localhost:8081}
    accounts: ${ACCOUNT_SERVICE_URL:http://localhost:8082}

```

## Листинг А.77 – deposit-service/src/main/resources/schema.sql

```
-- Схема данных сервиса депозитов

-- Справочник депозитных программ («вид депозита»); название и описание – на трёх языках
create table deposit_product (
    id                bigint generated by default as identity primary key,
    name              varchar(100) not null,
    name_en           varchar(100) not null,
    name_be           varchar(100) not null,
    kind              varchar(12) not null check (kind in ('REVOCABLE', 'IRREVOCABLE')),
    currency           varchar(3) not null,
    rate              decimal(6, 2) not null,
    min_amount        decimal(19, 2) not null,
    min_term_months   int not null,
    max_term_months   int not null,
    description        varchar(300) not null,
    description_en     varchar(300) not null,
    description_be     varchar(300) not null
);

create table deposit_contract (
    id                bigint generated by default as identity primary key,
    number            varchar(20) not null unique,
    product_id        bigint not null references deposit_product,
    client_id         bigint not null, -- клиент из модуля «Клиенты» (ЛР1)
    client_name       varchar(200) not null,
    currency           varchar(3) not null,
    amount            decimal(19, 2) not null,
    rate              decimal(6, 2) not null,
    term_months       int not null,
    start_date        date not null,
    end_date          date not null,
    main_account       varchar(13) not null, -- счёт основной суммы вклада
    interest_account   varchar(13) not null, -- счёт процентов по вкладу
    status            varchar(8) not null check (status in ('ACTIVE', 'CLOSED')),
    accrued           decimal(19, 2) not null default 0, -- начислено процентов
    paid              decimal(19, 2) not null default 0, -- выплачено процентов
    paid_months       int not null default 0, -- оплачено полных месяцев
    closed_on         date
);

create index ix_deposit_client on deposit_contract (client_id);
```

## Листинг А.78 – deposit-service/src/main/resources/data.sql

```
-- Депозитные программы по образцу банка «Решение» (вариант 4): условия приближены к действующим на gbank.by
insert into deposit_product (name, name_en, name_be, kind, currency, rate, min_amount, min_term_months, max_term_months,
    description, description_en, description_be)
values ('Свободное накопление', 'Free Savings', 'Свабоднае наапапенне', 'REVOCABLE', 'BYN', 7.00, 100.00, 1, 12,
    'Отзывный вклад до востребования, проценты выплачиваются ежемесячно',
    'Revocable demand deposit, interest is paid monthly',
    'Адзыйны ўклад да запатрабавання, працэнты выплачваюцца штомесяц'),
('Online-решение', 'Online-reshenie', 'Online-рашэнне', 'IRREVOCABLE', 'BYN', 12.90, 500.00, 3, 13,
    'Срочный безотзывный вклад, проценты выплачиваются в конце срока',
    'Fixed-term irrevocable deposit, interest is paid at the end of the term',
    'Тэрміновы безадзыйны ўклад, працэнты выплачваюцца ў канцы тэрміну'),
('Свободное накопление', 'Free Savings', 'Свабоднае наапапенне', 'REVOCABLE', 'USD', 1.50, 100.00, 1, 12,
    'Отзывный вклад до востребования, проценты выплачиваются ежемесячно',
    'Revocable demand deposit, interest is paid monthly',
    'Адзыйны ўклад да запатрабавання, працэнты выплачваюцца штомесяц'),
('Online-решение', 'Online-reshenie', 'Online-рашэнне', 'IRREVOCABLE', 'USD', 3.00, 500.00, 3, 13,
    'Срочный безотзывный вклад, проценты выплачиваются в конце срока',
    'Fixed-term irrevocable deposit, interest is paid at the end of the term',
    'Тэрміновы безадзыйны ўклад, працэнты выплачваюцца ў канцы тэрміну'),
('Online-решение', 'Online-reshenie', 'Online-рашэнне', 'IRREVOCABLE', 'EUR', 2.00, 500.00, 3, 13,
    'Срочный безотзывный вклад, проценты выплачиваются в конце срока',
    'Fixed-term irrevocable deposit, interest is paid at the end of the term',
    'Тэрміновы безадзыйны ўклад, працэнты выплачваюцца ў канцы тэрміну');
```

## Листинг А.79 – deposit-service/src/main/resources/messages.properties

```
# Тексты сервиса депозитов – русский язык (по умолчанию)
deposit.numberFormat=Формат номера договора: Д-0000001
deposit.unknownProduct=Вид депозита отсутствует в справочнике
deposit.minAmount=Минимальная сумма вклада – {0} {1}
deposit.closed=Договор {0} уже закрыт
deposit.irrevocable=Безотзывный вклад нельзя забрать до окончания срока договора
deposit.notFound=Депозитный договор не найден
```

```
deposit.kind.REVOCABLE=Отзывный, до востребования  
deposit.kind.IRREVOCABLE=Срочный безотзывный
```

```
# Протокол закрытия дня: дата, номер договора, сумма, валюта  
deposit.event.accrued={0}: {1} – начислены проценты {2} {3}  
deposit.event.paid={0}: {1} – выплачены проценты {2} {3}  
deposit.event.matured={0}: {1} – срок договора истёк, вклад {2} {3} возвращён
```

## Листинг A.80 – deposit-service/src/main/resources/messages\_be.properties

```
# Тэксты сэрвісу дэпазітаў – беларуская мова  
deposit.numberFormat=Фармат нумара дагавора: Д-000001  
deposit.unknownProduct=Від дэпазіту адсутнічае ў даведніку  
deposit.minAmount=Мінімальная сума ўкладу – {0} {1}  
deposit.closed=Дагавор {0} ужо закрыты  
deposit.irrevocable=Безадзёўны ўклад нельга забраць да заканчэння тэрміну дагавора  
deposit.notFound=Дэпазітны дагавор не знойдзены  
  
deposit.kind.REVOCABLE=Адзёўны, да запатрабавання  
deposit.kind.IRREVOCABLE=Тэрміновы безадзёўны  
  
deposit.event.accrued={0}: {1} – налічаны працэнты {2} {3}  
deposit.event.paid={0}: {1} – выплачаны працэнты {2} {3}  
deposit.event.matured={0}: {1} – тэрмін дагавора скончыўся, уклад {2} {3} вернуты
```

## Листинг A.81 – deposit-service/src/main/resources/messages\_en.properties

```
# Deposit service texts – English  
deposit.numberFormat=Contract number format: Д-000001  
deposit.unknownProduct=The deposit type is missing from the dictionary  
deposit.minAmount=Minimum deposit amount: {0} {1}  
deposit.closed=Contract {0} is already closed  
deposit.irrevocable=An irrevocable deposit cannot be withdrawn before the end of the term  
deposit.notFound=Deposit contract not found  
  
deposit.kind.REVOCABLE=Revocable, on demand  
deposit.kind.IRREVOCABLE=Fixed-term, irrevocable  
  
deposit.event.accrued={0}: {1} – interest accrued {2} {3}  
deposit.event.paid={0}: {1} – interest paid {2} {3}  
deposit.event.matured={0}: {1} – the term has expired, deposit {2} {3} returned
```

## Листинг A.82 – deposit-service/src/main/java/by/bsuir/bank/deposit/DepositServiceApplication.java

```
package by.bsuir.bank.deposit;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
/** Вместе с пакетом сервиса сканируется общая библиотека: REST-клиенты смежных сервисов и обработчик ошибок. */  
@SpringBootApplication(scanBasePackages = {"by.bsuir.bank.deposit", "by.bsuir.bank.common"})  
public class DepositServiceApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(DepositServiceApplication.class, args);  
    }  
}
```

## Листинг A.83 – deposit-service/src/main/java/by/bsuir/bank/deposit/domain/DepositContract.java

```
package by.bsuir.bank.deposit.domain;  
  
import jakarta.persistence.Entity;  
import jakarta.persistence.EnumType;  
import jakarta.persistence.Enumerated;  
import jakarta.persistence.FetchType;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.GenerationType;  
import jakarta.persistence.Id;  
import jakarta.persistence.ManyToOne;  
import lombok.Getter;
```

```

import lombok.Setter;

import java.math.BigDecimal;
import java.time.LocalDate;

/** Депозитный договор с физическим лицом. */
@Entity
@Getter
@Setter
public class DepositContract {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String number;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    private DepositProduct product;

    private Long clientId;
    private String clientName;
    private String currency;
    private BigDecimal amount;
    private BigDecimal rate;
    private int termMonths;
    private LocalDate startDate;
    private LocalDate endDate;
    private String mainAccount;
    private String interestAccount;

    @Enumerated(EnumType.STRING)
    private DepositStatus status;

    private BigDecimal accrued = BigDecimal.ZERO;
    private BigDecimal paid = BigDecimal.ZERO;
    private int paidMonths;
    private LocalDate closedOn;

    public boolean isActive() {
        return status == DepositStatus.ACTIVE;
    }
}

```

## Листинг A.84 –

deposit-service/src/main/java/by/bsuir/bank/deposit/domain/DepositKind.java

```

package by.bsuir.bank.deposit.domain;

import by.bsuir.bank.common.i18n.Messages;
import lombok.Getter;
import lombok.RequiredArgsConstructor;

/** Вид вклада определяет порядок выплаты процентов и балансовые счета из плана счетов. */
@Getter
@RequiredArgsConstructor
public enum DepositKind {

    /** Отзывный, до востребования: проценты выплачиваются ежемесячно, вклад можно забрать досрочно. */
    REVOCABLE("3404", "3470"),

    /** Срочный безотзывный: проценты выплачиваются в конце срока, досрочный отзыв невозможен. */
    IRREVOCABLE("3414", "3471");

    private final String mainChartCode;
    private final String interestChartCode;

    /** Название вида вклада на языке запроса. */
    public String getTitle() {
        return Messages.get("deposit.kind." + name());
    }
}

```

## Листинг A.85 –

deposit-service/src/main/java/by/bsuir/bank/deposit/domain/DepositProduct.java

```

package by.bsuir.bank.deposit.domain;

```

```

import by.bsuir.bank.common.i18n.Messages;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.AccessLevel;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;

/** Депозитная программа — запись справочника «вид депозита». */
@Entity
@Getter
@Setter
public class DepositProduct {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Getter(AccessLevel.NONE)
    private String nameEn;

    @Getter(AccessLevel.NONE)
    private String nameBe;

    @Enumerated(EnumType.STRING)
    private DepositKind kind;

    private String currency;
    private BigDecimal rate;
    private BigDecimal minAmount;
    private int minTermMonths;
    private int maxTermMonths;
    private String description;

    @Getter(AccessLevel.NONE)
    private String descriptionEn;

    @Getter(AccessLevel.NONE)
    private String descriptionBe;

    /** Название и описание программы отдаются на языке запроса. */
    public String getName() {
        return Messages.pick(name, nameEn, nameBe);
    }

    public String getDescription() {
        return Messages.pick(description, descriptionEn, descriptionBe);
    }

    public String getKindTitle() {
        return kind.getTitle();
    }
}

```

## Листинг А.86 — deposit-service/src/main/java/by/bsuir/bank/deposit/domain/DepositStatus.java

```

package by.bsuir.bank.deposit.domain;

public enum DepositStatus {
    ACTIVE, CLOSED
}

```

## Листинг А.87 — deposit- service/src/main/java/by/bsuir/bank/deposit/repository/DepositContractRepository.java

```

package by.bsuir.bank.deposit.repository;

import by.bsuir.bank.deposit.domain.DepositContract;
import by.bsuir.bank.deposit.domain.DepositStatus;

```

```

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import java.util.List;

public interface DepositContractRepository extends JpaRepository<DepositContract, Long> {

    boolean existsByNumber(String number);

    List<DepositContract> findAllOrderByIdDesc();

    List<DepositContract> findByClientIdOrderById(Long clientId);

    @Query("select c.id from DepositContract c where c.status = :status order by c.id")
    List<Long> findIdsByStatus(DepositStatus status);

    @Query("select max(c.number) from DepositContract c")
    String maxNumber();
}

```

## Листинг A.88 – deposit-service/src/main/java/by/bsuir/bank/deposit/repository/DepositProductRepository.java

```

package by.bsuir.bank.deposit.repository;

import by.bsuir.bank.deposit.domain.DepositProduct;
import org.springframework.data.jpa.repository.JpaRepository;

public interface DepositProductRepository extends JpaRepository<DepositProduct, Long> {
}

```

## Листинг A.89 – deposit-service/src/main/java/by/bsuir/bank/deposit/dto/DepositDetails.java

```

package by.bsuir.bank.deposit.dto;

import by.bsuir.bank.common.ledger.AccountInfo;
import by.bsuir.bank.common.ledger.OperationInfo;

import java.util.List;

/** Договор вместе с состоянием его счетов и проводками из главной книги. */
public record DepositDetails(DepositView contract, List<AccountInfo> accounts, List<OperationInfo> operations) {
}

```

## Листинг A.90 – deposit-service/src/main/java/by/bsuir/bank/deposit/dto/DepositRequest.java

```

package by.bsuir.bank.deposit.dto;

import jakarta.validation.constraints.Digits;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Positive;

import java.math.BigDecimal;
import java.time.LocalDate;

import static by.bsuir.bank.common.api.CommonExceptionHandler.REQUIRED;

/** Данные формы заключения депозитного договора. */
public record DepositRequest(

    @NotBlank(message = REQUIRED)
    @Pattern(regexp = NUMBER_FORMAT, message = "{deposit.numberFormat}")
    String number,

    @NotNull(message = REQUIRED)
    Long productId,

    @NotBlank(message = REQUIRED)
    String currency,

    @NotNull(message = REQUIRED)

```

```

        Long clientId,

        @NotNull(message = REQUIRED)
        @Positive(message = "{validation.amountPositive}")
        @Digits(integer = 12, fraction = 2, message = "{validation.amountFormat}")
        BigDecimal amount,

        @NotNull(message = REQUIRED)
        BigDecimal rate,

        @NotNull(message = REQUIRED)
        @Positive(message = "{validation.termPositive}")
        Integer termMonths,

        @NotNull(message = REQUIRED)
        LocalDate startDate,

        @NotNull(message = REQUIRED)
        LocalDate endDate
    ) {

        public static final String NUMBER_FORMAT = "^д-\\d{6}$";
    }
}

```

## Листинг А.91 – deposit-service/src/main/java/by/bsuir/bank/deposit/dto/DepositView.java

```

package by.bsuir.bank.deposit.dto;

import by.bsuir.bank.deposit.domain.DepositContract;

import java.math.BigDecimal;
import java.time.LocalDate;

public record DepositView(
    Long id,
    String number,
    Long productId,
    String productName,
    String kind,
    String kindTitle,
    Long clientId,
    String clientName,
    String currency,
    BigDecimal amount,
    BigDecimal rate,
    int termMonths,
    LocalDate startDate,
    LocalDate endDate,
    String mainAccount,
    String interestAccount,
    String status,
    BigDecimal accrued,
    BigDecimal paid,
    LocalDate closedOn
) {

    public static DepositView from(DepositContract c) {
        return new DepositView(c.getId(), c.getNumber(), c.getProduct().getId(), c.getProduct().getName(),
            c.getProduct().getKind().name(), c.getProduct().getKind().getTitle(), c.getClientId(),
            c.getClientName(), c.getCurrency(), c.getAmount(), c.getRate(), c.getTermMonths(),
            c.getStartDate(), c.getEndDate(), c.getMainAccount(), c.getInterestAccount(),
            c.getStatus().name(), c.getAccrued(), c.getPaid(), c.getClosedOn());
    }
}

```

## Листинг А.92 – deposit-service/src/main/java/by/bsuir/bank/deposit/dto/Meta.java

```

package by.bsuir.bank.deposit.dto;

import by.bsuir.bank.common.ledger.CurrencyInfo;

import java.time.LocalDate;
import java.util.List;

/** Данные для формы договора: банковский день, справочник валют, следующий свободный номер договора. */
public record Meta(LocalDate bankDate, List<CurrencyInfo> currencies, String nextNumber) {
}

```

```
}
```

## Листинг А.93 – deposit-service/src/main/java/by/bsuir/bank/deposit/service/DayCloseProcessor.java

```
package by.bsuir.bank.deposit.service;

import by.bsuir.bank.deposit.domain.DepositStatus;
import by.bsuir.bank.deposit.repository.DepositContractRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Component;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/** Участие сервиса депозитов в процедуре «Закрытие банковского дня»: каждый договор – в своей транзакции. */
@Component
@RequiredArgsConstructor
public class DayCloseProcessor {

    private final DepositContractRepository contracts;
    private final DepositService deposits;

    public List<String> process(LocalDate date) {
        List<String> events = new ArrayList<>();
        for (Long id : contracts.findIdsByStatus(DepositStatus.ACTIVE)) {
            events.addAll(deposits.processDay(id, date));
        }
        return events;
    }
}
```

## Листинг А.94 – deposit-service/src/main/java/by/bsuir/bank/deposit/service/DepositPostings.java

```
package by.bsuir.bank.deposit.service;

import by.bsuir.bank.common.ledger.Entry;
import by.bsuir.bank.common.ledger.Posting;

import static by.bsuir.bank.common.i18n.Localized.code;

import java.math.BigDecimal;
import java.util.List;

/**
 * Проводки процесса «Депозитная программа» – в точности по упрощённой схеме из задания.
 * Касса – активный счёт; фонд развития, счёт вклада и процентный счёт – пассивные.
 * Название операции передаётся в главную книгу кодом: текст подставляется при чтении на языке пользователя.
 */
public final class DepositPostings {

    /** Заключение договора: деньги вносятся в кассу, зачисляются на счёт вклада и передаются в фонд развития. */
    public static List<Posting> open(String cash, String fund, String main, BigDecimal amount) {
        return List.of(
            // внесение денег в кассу
            Posting.of(code("posting.deposit.cashIn"), Entry.debit(cash, amount)),
            // перевод денег с кассы на текущий счёт
            Posting.of(code("posting.deposit.toAccount"), Entry.credit(cash, amount), Entry.credit(main, amount)),
            // использование денег банком
            Posting.of(code("posting.deposit.toFund"), Entry.debit(main, amount), Entry.credit(fund, amount)));
    }

    public static Posting accrual(String fund, String interest, BigDecimal amount) {
        // начисление процентов по депозиту
        return Posting.of(code("posting.deposit.accrual"), Entry.debit(fund, amount), Entry.credit(interest, amount));
    }

    /** Выплата процентов: с процентного счёта в кассу и из кассы клиенту. */
    public static List<Posting> payout(String cash, String interest, BigDecimal amount) {
        return List.of(
            // перевод процентов в кассу
            Posting.of(code("posting.deposit.interestToCash"), Entry.debit(interest, amount), Entry.debit(cash, amount)),
            // вывод процентов из кассы
            Posting.of(code("posting.deposit.interestOut"), Entry.credit(cash, amount)));
    }
}
```

```

    /** Окончание депозита: сумма возвращается из фонда на счёт вклада, затем через кассу клиенту. */
    public static List<Posting> close(String cash, String fund, String main, BigDecimal amount) {
        return List.of(
            // окончание депозита
            Posting.of(code("posting.deposit.maturity"), Entry.debit(fund, amount), Entry.credit(main, amount)),
            // перевод депозита в кассу
            Posting.of(code("posting.deposit.toCash"), Entry.debit(main, amount), Entry.debit(cash, amount)),
            // вывод денег из кассы
            Posting.of(code("posting.deposit.cashOut"), Entry.credit(cash, amount)));
    }

    private DepositPostings() {
    }
}

```

## Листинг А.95 –

### deposit-service/src/main/java/by/bsuir/bank/deposit/service/DepositService.java

```

package by.bsuir.bank.deposit.service;

import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.i18n.Messages;
import by.bsuir.bank.common.ledger.Batch;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.common.ledger.OpenAccount;
import by.bsuir.bank.common.ledger.Posting;
import by.bsuir.bank.deposit.domain.DepositContract;
import by.bsuir.bank.deposit.domain.DepositKind;
import by.bsuir.bank.deposit.domain.DepositProduct;
import by.bsuir.bank.deposit.domain.DepositStatus;
import by.bsuir.bank.deposit.dto.DepositDetails;
import by.bsuir.bank.deposit.dto.DepositRequest;
import by.bsuir.bank.deposit.dto.DepositView;
import by.bsuir.bank.deposit.dto.Meta;
import by.bsuir.bank.deposit.repository.DepositContractRepository;
import by.bsuir.bank.deposit.repository.DepositProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.client.HttpClientErrorException;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

@Service
@RequiredArgsConstructor
public class DepositService {

    private static final DateTimeFormatter RU_DATE = DateTimeFormatter.ofPattern("dd.MM.yyyy");

    private final DepositContractRepository contracts;
    private final DepositProductRepository products;
    private final ClientApi clients;
    private final LedgerApi ledger;

    public Meta meta() {
        String last = contracts.maxNumber();
        int next = last == null ? 1 : Integer.parseInt(last.substring(2)) + 1;
        return new Meta(ledger.bankDay().date(), ledger.currencies(), String.format("Д-№%06d", next));
    }

    @Transactional(readOnly = true)
    public List<DepositView> list(Long clientId) {
        List<DepositContract> found = clientId == null
            ? contracts.findAllByOrderByIdDesc()
            : contracts.findByClientIdOrderByById(clientId);
        return found.stream().map(DepositView::from).toList();
    }

    @Transactional(readOnly = true)
    public DepositDetails details(Long id) {
    }
}

```

```

        DepositContract contract = find(id);
        return new DepositDetails(DepositView.from(contract),
            ledger.accounts(contract.getNumber()), ledger.operations(contract.getNumber()));
    }

    /**
     * Заключение договора: проверка условий программы, открытие двух счетов (основной суммы и процентов)
     * и проводки приёма вклада. Запросы к главной книге идемпотентны по номеру договора,
     * поэтому повтор после сбоя не создаёт счета и проводки дважды.
     */
    @Transactional
    public DepositView open(DepositRequest request) {
        DepositProduct product = products.findById(request.productId())
            .orElseThrow(() -> BankException.fields(Map.of("productId", Messages.get("deposit.unknownProduct"))));
        LocalDate today = ledger.bankDay().date();
        checkTerms(request, product, today);
        if (contracts.existsByNumber(request.number())) {
            throw new BankException(HttpStatus.CONFLICT, "DUPLICATE_CONTRACT", Messages.get("contract.duplicate"),
                Map.of("number", Messages.get("contract.duplicateNumber")));
        }
        ClientInfo client = findClient(request.clientId());

        DepositKind kind = product.getKind();
        String number = request.number();
        String currency = product.getCurrency();
        String main = ledger.open(new OpenAccount(kind.getMainChartCode(), currency,
            client.id().intValue(), client.fullName(), number)).number();
        String interest = ledger.open(new OpenAccount(kind.getInterestChartCode(), currency,
            client.id().intValue(), client.fullName(), number)).number();
        post(key(number, "OPEN"), number, today,
            DepositPostings.open(cash(currency), fund(currency), main, request.amount()));

        DepositContract contract = new DepositContract();
        contract.setNumber(number);
        contract.setProduct(product);
        contract.setClientId(client.id());
        contract.setClientName(client.fullName());
        contract.setCurrency(currency);
        contract.setAmount(request.amount());
        contract.setRate(product.getRate());
        contract.setTermMonths(request.termMonths());
        contract.setStartDate(today);
        contract.setEndDate(request.endDate());
        contract.setMainAccount(main);
        contract.setInterestAccount(interest);
        contract.setStatus(DepositStatus.ACTIVE);
        return DepositView.from(contracts.save(contract));
    }

    /** Контроль корректности условий договора относительно выбранной депозитной программы. */
    private void checkTerms(DepositRequest request, DepositProduct product, LocalDate today) {
        Map<String, String> errors = new LinkedHashMap<>();
        if (!product.getCurrency().equals(request.currency())) {
            errors.put("productId", Messages.get("contract.productCurrency", request.currency()));
        }
        if (request.amount().compareTo(product.getMinAmount()) < 0) {
            errors.put("amount", Messages.get("deposit.minAmount", product.getMinAmount(), product.getCurrency()));
        }
        if (request.rate().compareTo(product.getRate()) != 0) {
            errors.put("rate", Messages.get("contract.rate", product.getRate()));
        }
        if (request.termMonths() < product.getMinTermMonths() || request.termMonths() > product.getMaxTermMonths()) {
            errors.put("termMonths", Messages.get("contract.termRange",
                product.getMinTermMonths(), product.getMaxTermMonths()));
        }
        if (!request.startDate().equals(today)) {
            errors.put("startDate", Messages.get("contract.startDate", RU_DATE.format(today)));
        }
        if (!request.endDate().equals(request.startDate().plusMonths(request.termMonths()))) {
            errors.put("endDate", Messages.get("contract.endDate"));
        }
        if (!errors.isEmpty()) {
            throw BankException.fields(errors);
        }
    }

    private ClientInfo findClient(Long clientId) {
        try {
            return clients.get(clientId);
        } catch (HttpClientErrorException.NotFound e) {
            throw BankException.fields(Map.of("clientId", Messages.get("contract.clientNotFound")));
        }
    }

```

```

}

/** Досрочный отзыв вклада: доступен только для отзывных программ. */
@Transactional
public DepositView withdraw(Long id) {
    DepositContract contract = find(id);
    if (!contract.isActive()) {
        throw BankException.conflict("CONTRACT_CLOSED", Messages.get("deposit.closed", contract.getNumber()));
    }
    if (contract.getProduct().getKind() == DepositKind.IRREVOCABLE) {
        throw BankException.conflict("IRREVOCABLE", Messages.get("deposit.irrevocable"));
    }
    LocalDate today = ledger.bankDay().date();
    payInterest(contract, today);
    close(contract, today);
    return DepositView.from(contract);
}

/**
 * Обработка договора при открытии банковского дня date: начисление процентов за прошедшие дни,
 * выплата (ежемесячно либо в конце срока) и возврат вклада по окончании договора.
 * Обработка накопительная и идемпотентная: пропуск дня или повтор не искажают суммы.
 */
@Transactional
public List<String> processDay(Long id, LocalDate date) {
    DepositContract contract = find(id);
    List<String> events = new ArrayList<>();
    if (!contract.isActive() || !date.isAfter(contract.getStartDate())) {
        return events;
    }
    String number = contract.getNumber();
    String currency = contract.getCurrency();
    boolean matured = !date.isBefore(contract.getEndDate());
    LocalDate upTo = matured ? contract.getEndDate() : date;

    BigDecimal target = InterestCalculator.accumulated(contract.getAmount(), contract.getRate(), contract.getStartDate(), upTo);
    BigDecimal delta = target.subtract(contract.getAccrued());
    if (delta.signum() > 0) {
        post(key(number, "ACCUE-" + date), number, date,
            List.of(DepositPostings.accumual(fund(currency), contract.getInterestAccount(), delta)));
        contract.setAccrued(target);
        events.add(Messages.get("deposit.event.accumual", RU_DATE.format(date), number, delta, currency));
    }

    int months = InterestCalculator.fullMonths(contract.getStartDate(), upTo);
    boolean monthlyPayout = contract.getProduct().getKind() == DepositKind.REVOCABLE && months > contract.getPaidMonths();
    if (matured || monthlyPayout) {
        BigDecimal paid = payInterest(contract, date);
        contract.setPaidMonths(months);
        if (paid.signum() > 0) {
            events.add(Messages.get("deposit.event.paid", RU_DATE.format(date), number, paid, currency));
        }
    }
    if (matured) {
        close(contract, date);
        events.add(Messages.get("deposit.event.matured", RU_DATE.format(date), number, contract.getAmount(), currency));
    }
    return events;
}

private BigDecimal payInterest(DepositContract contract, LocalDate date) {
    BigDecimal due = contract.getAccrued().subtract(contract.getPaid());
    if (due.signum() > 0) {
        post(key(contract.getNumber(), "PAYOUT-" + date), contract.getNumber(), date,
            DepositPostings.payout(cash(contract.getCurrency()), contract.getInterestAccount(), due));
        contract.setPaid(contract.getAccrued());
    }
    return due;
}

private void close(DepositContract contract, LocalDate date) {
    String currency = contract.getCurrency();
    post(key(contract.getNumber(), "CLOSE"), contract.getNumber(), date,
        DepositPostings.close(cash(currency), fund(currency), contract.getMainAccount(), contract.getAmount()));
    contract.setStatus(DepositStatus.CLOSED);
    contract.setClosedOn(date);
}

private void post(String batchKey, String contractRef, LocalDate date, List<Posting> postings) {
    ledger.post(new Batch(batchKey, contractRef, date, postings));
}

```

```

private static String key(String number, String step) {
    return "DEP-" + number + "-" + step;
}

private String cash(String currency) {
    return ledger.systemAccount(LedgerApi.CASH, currency).number();
}

private String fund(String currency) {
    return ledger.systemAccount(LedgerApi.FUND, currency).number();
}

private DepositContract find(Long id) {
    return contracts.findById(id).orElseThrow(() -> BankException.notFound(Messages.get("deposit.notFound")));
}
}

```

## Листинг А.96 – deposit-service/src/main/java/by/bsuir/bank/deposit/service/InterestCalculator.java

```

package by.bsuir.bank.deposit.service;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

/** Расчёт процентов по вкладу: простые проценты, фактическое число дней, 365 дней в году. */
public final class InterestCalculator {

    private static final BigDecimal YEAR_BASIS = BigDecimal.valueOf(100L * 365);

    /**
     * Проценты, накопленные с даты начала по дату расчёта:  $S * r * \text{дни} / (100 * 365)$ .
     * Считается накопленная сумма целиком, а не прирост за день, – так копейки округления не теряются.
     */
    public static BigDecimal accrued(BigDecimal amount, BigDecimal rate, LocalDate start, LocalDate upTo) {
        long days = Math.max(0, ChronoUnit.DAYS.between(start, upTo));
        return amount.multiply(rate).multiply(BigDecimal.valueOf(days)).divide(YEAR_BASIS, 2, RoundingMode.HALF_UP);
    }

    /** Число полных месяцев договора, истёкших к дате расчёта. */
    public static int fullMonths(LocalDate start, LocalDate upTo) {
        int months = 0;
        while (!start.plusMonths(months + 1L).isAfter(upTo)) {
            months++;
        }
        return months;
    }

    private InterestCalculator() {
    }
}

```

## Листинг А.97 – deposit-service/src/main/java/by/bsuir/bank/deposit/web/DayCloseController.java

```

package by.bsuir.bank.deposit.web;

import by.bsuir.bank.deposit.service.DayCloseProcessor;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDate;
import java.util.List;

/** Внутренний REST для сервиса «Счета»: обработка открытия нового банковского дня. */
@RestController
@RequiredArgsConstructor
public class DayCloseController {

    private final DayCloseProcessor processor;

    public record DayOpened(LocalDate date) {
    }
}

```

```

    @PostMapping("/api/internal/day-close")
    public List<String> dayOpened(@RequestBody DayOpened event) {
        return processor.process(event.date());
    }
}

```

## Листинг А.98 – deposit-service/src/main/java/by/bsuir/bank/deposit/web/DepositController.java

```

package by.bsuir.bank.deposit.web;

import by.bsuir.bank.deposit.dto.DepositDetails;
import by.bsuir.bank.deposit.dto.DepositRequest;
import by.bsuir.bank.deposit.dto.DepositView;
import by.bsuir.bank.deposit.service.DepositService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/deposits")
@RequiredArgsConstructor
public class DepositController {

    private final DepositService service;

    @GetMapping
    public List<DepositView> list(@RequestParam(required = false) Long clientId) {
        return service.list(clientId);
    }

    @GetMapping("/{id}")
    public DepositDetails details(@PathVariable Long id) {
        return service.details(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public DepositView open(@Valid @RequestBody DepositRequest request) {
        return service.open(request);
    }

    /** Досрочный отзыв вклада. */
    @PostMapping("/{id}/withdraw")
    public DepositView withdraw(@PathVariable Long id) {
        return service.withdraw(id);
    }
}

```

## Листинг А.99 – deposit- service/src/main/java/by/bsuir/bank/deposit/web/ReferenceController.java

```

package by.bsuir.bank.deposit.web;

import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.BankDayInfo;
import by.bsuir.bank.common.ledger.DayCloseInfo;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.deposit.domain.DepositProduct;
import by.bsuir.bank.deposit.dto.Meta;
import by.bsuir.bank.deposit.repository.DepositProductRepository;
import by.bsuir.bank.deposit.service.DepositService;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.GetMapping;

```

```

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

/** Справочные данные формы договора. Клиенты и банковский день запрашиваются у смежных сервисов. */
@RestController
@RequestMapping("/api")
@RequiredArgsConstructor
public class ReferenceController {

    private final DepositProductRepository products;
    private final DepositService deposits;
    private final ClientApi clients;
    private final LedgerApi ledger;

    @GetMapping("/products")
    public List<DepositProduct> products() {
        return products.findAll(Sort.by("id"));
    }

    @GetMapping("/clients")
    public List<ClientInfo> clients() {
        return clients.list();
    }

    @GetMapping("/meta")
    public Meta meta() {
        return deposits.meta();
    }

    @GetMapping("/bank-day")
    public BankDayInfo bankDay() {
        return ledger.bankDay();
    }

    @PostMapping("/bank-day/close")
    public DayCloseInfo closeDay(@RequestParam(defaultValue = "1") int days) {
        return ledger.closeDay(days);
    }
}

```

## Листинг А.100 – deposit-service/src/main/resources/static/index.html

```

<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>BankEt</title>
  <link rel="stylesheet" href="/ui/bank.css">
</head>
<body>
  <div id="app" v-cloak></div>

  <script src="/webjars/vue/dist/vue.global.prod.js"></script>
  <script src="/ui/bank.js"></script>
  <script src="/ui/i18n.js"></script>
  <script src="/i18n.js"></script>
  <script src="/app.js"></script>
</body>
</html>

```

## Листинг А.101 – deposit-service/src/main/resources/static/app.js

```

/* Web-клиент сервиса депозитов: список договоров, форма заключения договора, карточка договора. */

const t = Bank.t;

/* Дата окончания: как LocalDate.plusMonths — день месяца сохраняется либо прижимается к концу месяца. */
function addMonths(iso, months) {
  const [year, month, day] = iso.split('-').map(Number);
  const first = new Date(year, month - 1 + months, 1);
  const lastDay = new Date(first.getFullYear(), first.getMonth() + 1, 0).getDate();
  const pad = n => String(n).padStart(2, '0');
  return first.getFullYear() + '-' + pad(first.getMonth() + 1) + '-' + pad(Math.min(day, lastDay));
}

```

```

const app = Bank.createApp({
  data() {
    return {
      view: 'list',           // list | form | details
      deposits: [],
      products: [],
      clients: [],
      meta: { bankDate: null, currencies: [], nextNumber: '' },
      form: {},
      errors: {},
      formError: '',
      details: null,
      notice: '',
      error: '',
      events: [],
      saving: false,
    };
  },
  computed: {
    availableProducts() {
      return this.products.filter(p => p.currency === this.form.currency);
    },
    product() {
      return this.products.find(p => p.id === this.form.productId) || null;
    },
    endDate() {
      const months = Number(this.form.termMonths);
      return this.meta.bankDate && months > 0 && months <= 600 ? addMonths(this.meta.bankDate, months) : null;
    },
  },
  async mounted() {
    try {
      this.products = await Bank.api('GET', '/api/products');
      await this.reload();
    } catch (e) {
      this.error = e.message;
    }
  },
  methods: {
    async reload() {
      this.deposits = await Bank.api('GET', '/api/deposits');
    },
    async openForm() {
      this.error = '';
      try {
        this.meta = await Bank.api('GET', '/api/meta');
        this.clients = await Bank.api('GET', '/api/clients');
      } catch (e) {
        this.error = e.message;
        return;
      }
      this.form = { clientId: null, currency: 'BYN', productId: null, number: this.meta.nextNumber, amount: '', termMonths: '' };
      this.errors = {};
      this.formError = '';
      this.notice = '';
      this.view = 'form';
    },
    /* Первичный контроль корректности данных договора; сервис повторяет все проверки. */
    validate() {
      const errors = {};
      const form = this.form;
      if (!form.clientId) errors.clientId = t('common.required');
      if (!form.productId) errors.productId = t('common.required');
      if (!form.number) errors.number = t('common.required');
      else if (!/^Д-\d{6}$/.test(form.number)) errors.number = t('deposits.numberFormat');
      const amount = String(form.amount).trim().replace(',', '.');
      if (!amount) errors.amount = t('common.required');
      else if (!/^^\d{1,12}(\.\d{1,2})?$/ .test(amount)) errors.amount = t('contract.moneyFormat');
      else if (this.product && Number(amount) < this.product.minAmount)
        errors.amount = t('deposits.minAmount', Bank.money(this.product.minAmount), this.product.currency);
      const term = String(form.termMonths).trim();
      if (!term) errors.termMonths = t('common.required');
      else if (!/^^\d{1,3}$/.test(term)) errors.termMonths = t('contract.termInteger');
      else if (this.product && (Number(term) < this.product.minTermMonths || Number(term) > this.product.maxTermMonths))
        errors.termMonths = t('contract.termRange', this.product.minTermMonths, this.product.maxTermMonths);
      return errors;
    },
    async save() {
      this.errors = this.validate();
      this.formError = Object.keys(this.errors).length ? t('common.formErrors') : '';
      if (this.formError) return;
    }
  }
});

```

```

this.saving = true;
try {
  const created = await Bank.api('POST', '/api/deposits', {
    number: this.form.number,
    productId: this.form.productId,
    currency: this.form.currency,
    clientId: this.form.clientId,
    amount: Number(String(this.form.amount).trim().replace(',', '.')),
    rate: this.product.rate,
    termMonths: Number(this.form.termMonths),
    startDate: this.meta.bankDate,
    endDate: this.endDate,
  });
  await this.reload();
  await this.openDetails(created.id);
  this.notice = t('deposits.opened', created.number, created.mainAccount, created.interestAccount);
} catch (e) {
  this.errors = e.fields;
  this.formError = e.message;
} finally {
  this.saving = false;
}
},
async openDetails(id) {
  this.notice = '';
  this.error = '';
  try {
    this.details = await Bank.api('GET', '/api/deposits/' + id);
    this.view = 'details';
  } catch (e) {
    this.error = e.message;
  }
},
async withdraw() {
  const contract = this.details.contract;
  if (!confirm(t('deposits.confirmWithdraw', contract.number))) return;
  try {
    await Bank.api('POST', '/api/deposits/' + contract.id + '/withdraw');
    await this.reload();
    await this.openDetails(contract.id);
    this.notice = t('deposits.withdrawn');
  } catch (e) {
    this.error = e.message;
  }
},
async dayClosed(result) {
  this.events = result.events;
  this.meta.bankDate = result.bankDate;
  await this.reload();
  if (this.view === 'details') await this.openDetails(this.details.contract.id);
},
/* Возврат к списку: сообщения карточки договора больше не нужны. */
toList() {
  this.notice = '';
  this.view = 'list';
},
money: Bank.money,
date: Bank.date,
},
watch: {
  'form.currency'() {
    if (this.product && this.product.currency !== this.form.currency) this.form.productId = null;
  },
},
template: `
<bank-nav active="deposits"></bank-nav>
<main>
  <div class="toolbar">
    <div>
      <h1>{{ t('deposits.title') }}</h1>
      <p class="subtitle">{{ t('deposits.subtitle') }}</p>
    </div>
    <bank-day @changed="dayClosed" @failed="e => error = e.message"></bank-day>
  </div>

  <div v-if="error" id="error" class="alert error">{{ error }}</div>
  <div v-if="notice" id="notice" class="alert success">{{ notice }}</div>
  <div v-if="events.length" id="events" class="alert info">
    <b>{{ t('events.title') }}</b>
    <div v-for="(event, i) in events.slice(-8)" :key="i">{{ event }}</div>
    <div v-if="events.length > 8" class="muted">{{ t('events.more', events.length) }}</div>
  </div>

```

```

<section v-if="view === 'list'">
  <div class="toolbar">
    <span class="muted">{{ t('contract.count', deposits.length) }}</span>
    <button id="btn-add" class="primary" @click="openForm">{{ t('contract.conclude') }}</button>
  </div>
  <div class="card">
    <table id="deposits">
      <thead>
        <tr>
          <th>{{ t('table.contract') }}</th><th>{{ t('contract.client') }}</th><th>{{ t('deposits.kind') }}</th>
          <th class="num">{{ t('contract.sum') }}</th><th class="num">{{ t('contract.rate') }}</th>
          <th>{{ t('contract.start') }}</th><th>{{ t('contract.end') }}</th><th class="num">{{ t('deposits.accrued') }}</th>
          <th class="num">{{ t('deposits.paid') }}</th><th>{{ t('contract.status') }}</th>
        </tr>
      </thead>
      <tbody>
        <tr v-for="d in deposits" :key="d.id">
          <td class="mono"><a class="link" @click="openDetails(d.id)">{{ d.number }}</a></td>
          <td>{{ d.clientName }}</td>
          <td>{{ d.productName }}<div class="muted" style="font-size: 12.5px">{{ d.kindTitle }}</div></td>
          <td class="num">{{ money(d.amount) }} {{ d.currency }}</td>
          <td class="num">{{ d.rate }} %</td>
          <td>{{ date(d.startDate) }}</td>
          <td>{{ date(d.endDate) }}</td>
          <td class="num">{{ money(d.accrued) }}</td>
          <td class="num">{{ money(d.paid) }}</td>
          <td><span class="badge" :class="d.status === 'ACTIVE' ? 'ok' : 'off'">{{ d.status === 'ACTIVE' ? t('contract.
active') : t('deposits.closed') }}</span></td>
        </tr>
      </tbody>
    </table>
    <div v-if="!deposits.length" class="empty">{{ t('contract.none') }}</div>
  </div>
</section>

<section v-if="view === 'form'">
  <h2>{{ t('deposits.new') }}</h2>
  <div v-if="formError" id="form-alert" class="alert error">{{ formError }}</div>
  <form class="card" novalidate @submit.prevent="save">
    <div class="form-grid">
      <div class="field wide" :class="{ invalid: errors.clientId }">
        <label for="f-clientId">{{ t('contract.client') }} <span class="req">*</span></label>
        <select id="f-clientId" v-model="form.clientId">
          <option :value="null">{{ t('contract.chooseClient') }}</option>
          <option v-for="c in clients" :key="c.id" :value="c.id">
            {{ c.fullName }} ({{ t('contract.passport') }} {{ c.passportSeries }} {{ c.passportNumber }})
          </option>
        </select>
        <div v-if="errors.clientId" class="error" id="e-clientId">{{ errors.clientId }}</div>
      </div>
      <div class="field" :class="{ invalid: errors.number }">
        <label for="f-number">{{ t('contract.number') }} <span class="req">*</span></label>
        <input id="f-number" type="text" v-model="form.number" v-mask="'Д-####'" placeholder="Д-000001">
        <div v-if="errors.number" class="error" id="e-number">{{ errors.number }}</div>
      </div>

      <div class="field">
        <label for="f-currency">{{ t('contract.currency') }} <span class="req">*</span></label>
        <select id="f-currency" v-model="form.currency">
          <option v-for="c in meta.currencies" :key="c.code" :value="c.code">{{ c.code }} - {{ c.name }}</option>
        </select>
      </div>
      <div class="field wide" :class="{ invalid: errors.productId }">
        <label for="f-productId">{{ t('deposits.kind') }} <span class="req">*</span></label>
        <select id="f-productId" v-model="form.productId">
          <option :value="null">{{ t('contract.chooseProduct') }}</option>
          <option v-for="p in availableProducts" :key="p.id" :value="p.id">
            {{ t('deposits.option', p.name, p.kindTitle.toLowerCase(), p.rate) }}
          </option>
        </select>
        <div v-if="errors.productId" class="error" id="e-productId">{{ errors.productId }}</div>
        <div v-else-if="product" class="hint">{{ t('deposits.hint', product.description, money(product.minAmount),
product.currency, product.minTermMonths, product.maxTermMonths) }}</div>
      </div>

      <div class="field" :class="{ invalid: errors.amount }">
        <label for="f-amount">{{ t('deposits.amount', form.currency) }} <span class="req">*</span></label>
        <input id="f-amount" type="text" v-model="form.amount" placeholder="0.00">
        <div v-if="errors.amount" class="error" id="e-amount">{{ errors.amount }}</div>
      </div>
      <div class="field" :class="{ invalid: errors.termMonths }">

```

```

<label for="f-termMonths">{{ t('contract.term') }} <span class="req">*</span></label>
<input id="f-termMonths" type="text" v-model="form.termMonths" v-mask="###">
<div v-if="errors.termMonths" class="error" id="e-termMonths">{{ errors.termMonths }}</div>
</div>
<div class="field" :class="{ invalid: errors.rate }">
<label for="f-rate">{{ t('deposits.rate') }}</label>
<input id="f-rate" type="text" :value="product ? product.rate : ''" readonly>
<div v-if="errors.rate" class="error" id="e-rate">{{ errors.rate }}</div>
</div>

<div class="field" :class="{ invalid: errors.startDate }">
<label for="f-startDate">{{ t('contract.startDate') }}</label>
<input id="f-startDate" type="text" :value="date(meta.bankDate)" readonly>
<div v-if="errors.startDate" class="error" id="e-startDate">{{ errors.startDate }}</div>
</div>
<div class="field" :class="{ invalid: errors.endDate }">
<label for="f-endDate">{{ t('contract.endDate') }}</label>
<input id="f-endDate" type="text" :value="date(endDate)" readonly>
<div v-if="errors.endDate" class="error" id="e-endDate">{{ errors.endDate }}</div>
</div>
</div>
<div class="form-actions" style="margin-top: 18px">
<button id="btn-save" type="submit" class="primary" :disabled="saving">{{ t('contract.conclude') }}</button>
<button id="btn-cancel" type="button" @click="view = 'list'">{{ t('common.cancel') }}</button>
</div>
</form>
</section>

<section v-if="view === 'details' && details">
<div class="toolbar">
<h2>{{ t('deposits.contract', details.contract.number, details.contract.clientName) }}</h2>
<div style="display: flex; gap: 8px">
<button v-if="details.contract.status === 'ACTIVE' && details.contract.kind === 'REVOCABLE'"
id="btn-withdraw" class="danger" @click="withdraw">{{ t('deposits.withdraw') }}</button>
<button id="btn-back" @click="toList">{{ t('common.toList') }}</button>
</div>
</div>
<div class="card details">
<div><div class="label">{{ t('deposits.kind') }}</div><div class="value">{{ details.contract.productName }}</div><div
class="muted">{{ details.contract.kindTitle }}</div></div>
<div><div class="label">{{ t('deposits.amountShort') }}</div><div class="value">{{ money(details.contract.amount) }} {{
details.contract.currency }}</div></div>
<div><div class="label">{{ t('contract.rate') }}</div><div class="value">{{ details.contract.rate }} {{ t('contract.
perAnnum') }}</div></div>
<div><div class="label">{{ t('contract.status') }}</div><div class="value">{{ details.contract.status === 'ACTIVE' ? t('
contract.active') : t('deposits.closedOn', date(details.contract.closedOn)) }}</div></div>
<div><div class="label">{{ t('contract.termShort') }}</div><div class="value">{{ details.contract.termMonths }} {{ t('
contract.months') }}</div></div>
<div><div class="label">{{ t('contract.period') }}</div><div class="value">{{ date(details.contract.startDate) }} – {{
date(details.contract.endDate) }}</div></div>
<div><div class="label">{{ t('deposits.accumulatedInterest') }}</div><div class="value">{{ money(details.contract.accumued)
}} {{ details.contract.currency }}</div></div>
<div><div class="label">{{ t('deposits.paidInterest') }}</div><div class="value">{{ money(details.contract.paid) }} {{
details.contract.currency }}</div></div>
</div>

<div class="card">
<h2>{{ t('contract.accounts') }}</h2>
<table id="contract-accounts">
<thead><tr><th>{{ t('table.account') }}</th><th>{{ t('table.code') }}</th><th>{{ t('table.purpose') }}</th><th>{{ t('
table.activity') }}</th>
<th class="num">{{ t('table.debit') }}</th><th class="num">{{ t('table.credit') }}</th><th class="num">{{ t('table
.balance') }}</th></tr></thead>
<tbody>
<tr v-for="a in details.accounts" :key="a.number">
<td class="mono">{{ a.number }}</td>
<td class="mono">{{ a.chartCode }}</td>
<td>{{ a.chartName }}<div class="muted" style="font-size: 12.5px">{{ a.name }}</div></td>
<td>{{ a.activityTitle }}</td>
<td class="num">{{ money(a.debit) }}</td>
<td class="num">{{ money(a.credit) }}</td>
<td class="num"><b>{{ money(a.balance) }}</b></td>
</tr>
</tbody>
</table>
</div>

<div class="card">
<h2>{{ t('contract.postings') }}</h2>
<table id="contract-operations">
<thead><tr><th>{{ t('table.date') }}</th><th>{{ t('table.operation') }}</th><th>{{ t('table.accountShort') }}</th>
<th class="num">{{ t('table.debit') }}</th><th class="num">{{ t('table.credit') }}</th></tr></thead>

```

```

<tbody>
  <template v-for="op in details.operations" :key="op.id">
    <tr v-for="(entry, i) in op.entries" :key="op.id + '-' + i">
      <td>{{ i === 0 ? date(op.bankDate) : '' }}</td>
      <td>{{ i === 0 ? op.description : '' }}</td>
      <td class="mono">{{ entry.account }} <span class="muted">{{ entry.chartCode }}</span></td>
      <td class="num">{{ entry.side === 'DEBIT' ? money(entry.amount) : '' }}</td>
      <td class="num">{{ entry.side === 'CREDIT' ? money(entry.amount) : '' }}</td>
    </tr>
  </template>
</tbody>
</table>
</div>
</section>
</main>',
}, 'deposits.page'));

app.mount('#app');

```

## Листинг А.102 – deposit-service/src/main/resources/static/i18n.js

```

/* Словарь web-клиента сервиса депозитов: русский, английский, белорусский. */
Bank.addMessages({
  ru: {
    'deposits.page': 'Депозиты',
    'deposits.title': 'Депозитные договоры',
    'deposits.subtitle': 'Вклады физических лиц: заключение договора, начисление и выплата процентов',
    'deposits.new': 'Новый депозитный договор',
    'deposits.kind': 'Вид депозита',
    'deposits.option': '«{0}» — {1}, {2} % годовых',
    'deposits.hint': '{0}. Сумма от {1} {2}, срок {3}—{4} мес.',
    'deposits.amount': 'Сумма вклада, {0}',
    'deposits.amountShort': 'Сумма вклада',
    'deposits.rate': 'Процент по вкладу, % годовых',
    'deposits.accrued': 'Начислено', 'deposits.paid': 'Выплачено',
    'deposits.accruedInterest': 'Начислено процентов', 'deposits.paidInterest': 'Выплачено процентов',
    'deposits.closed': 'Закрыт',
    'deposits.closedOn': 'Закрыт {0}',
    'deposits.contract': 'Договор {0} · {1}',
    'deposits.withdraw': 'Отозвать вклад',
    'deposits.confirmWithdraw': 'Отозвать вклад по договору {0}?',
    'deposits.withdrawn': 'Вклад возвращён клиенту, договор закрыт',
    'deposits.opened': 'Договор {0} заключён, открыты счета {1} и {2}',
    'deposits.numberFormat': 'Формат номера договора: Д-000001',
    'deposits.minAmount': 'Минимальная сумма вклада — {0} {1}',
  },
  en: {
    'deposits.page': 'Deposits',
    'deposits.title': 'Deposit contracts',
    'deposits.subtitle': 'Deposits of individuals: signing a contract, accrual and payment of interest',
    'deposits.new': 'New deposit contract',
    'deposits.kind': 'Deposit type',
    'deposits.option': '"{0}" — {1}, {2} % per annum',
    'deposits.hint': '{0}. Amount from {1} {2}, term {3}—{4} months.',
    'deposits.amount': 'Deposit amount, {0}',
    'deposits.amountShort': 'Deposit amount',
    'deposits.rate': 'Deposit rate, % per annum',
    'deposits.accrued': 'Accrued', 'deposits.paid': 'Paid',
    'deposits.accruedInterest': 'Interest accrued', 'deposits.paidInterest': 'Interest paid',
    'deposits.closed': 'Closed',
    'deposits.closedOn': 'Closed {0}',
    'deposits.contract': 'Contract {0} · {1}',
    'deposits.withdraw': 'Withdraw deposit',
    'deposits.confirmWithdraw': 'Withdraw the deposit under contract {0}?',
    'deposits.withdrawn': 'The deposit has been returned to the client, the contract is closed',
    'deposits.opened': 'Contract {0} signed, accounts {1} and {2} opened',
    'deposits.numberFormat': 'Contract number format: Д-000001',
    'deposits.minAmount': 'Minimum deposit amount: {0} {1}',
  },
  be: {
    'deposits.page': 'Дэпазіты',
    'deposits.title': 'Дэпазітныя дагаворы',
    'deposits.subtitle': 'Уклады фізічных асоб: заключэнне дагавора, налічэнне і выплата працэнтаў',
    'deposits.new': 'Новы дэпазітны дагавор',
    'deposits.kind': 'Від дэпазіту',
    'deposits.option': '«{0}» — {1}, {2} % гадавых',
    'deposits.hint': '{0}. Сума ад {1} {2}, тэрмін {3}—{4} мес.',
    'deposits.amount': 'Сума ўкладу, {0}',
    'deposits.amountShort': 'Сума ўкладу',
    'deposits.rate': 'Працэнт па ўкладзе, % гадавых',
  }
});

```

```

        'deposits.accrued': 'Налічана', 'deposits.paid': 'Виплачена',
        'deposits.accruedInterest': 'Налічана працэнтаў', 'deposits.paidInterest': 'Виплачена працэнтаў',
        'deposits.closed': 'Закрыты',
        'deposits.closedOn': 'Закрыты {0}',
        'deposits.contract': 'Дагавор {0} · {1}',
        'deposits.withdraw': 'Адазваць уклад',
        'deposits.confirmWithdraw': 'Адазваць уклад па дагаворы {0}?',
        'deposits.withdrawn': 'Уклад вернуты кліенту, дагавор закрыты',
        'deposits.opened': 'Дагавор {0} заключаны, адкрыты рахункі {1} і {2}',
        'deposits.numberFormat': 'Фармат нумара дагавора: Д-000001',
        'deposits.minAmount': 'Мінімальная сума ўкладу — {0} {1}',
    },
});

```

## Листинг А.103 – deposit-service/src/test/java/by/bsuir/bank/deposit/DepositApiIT.java

```

package by.bsuir.bank.deposit;

import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.InMemoryLedger;
import by.bsuir.bank.deposit.repository.DepositContractRepository;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.context.bean.override.mockito.MockitoSpyBean;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.ResultActions;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.ResourceAccessException;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

import static org.assertj.core.api.Assertions.assertThat;
import static org.hamcrest.Matchers.hasSize;
import static org.mockito.Mockito.doThrow;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/** Интеграционные тесты REST сервиса депозитов: контроль корректности данных договора и ошибки смежных сервисов. */
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@AutoConfigureMockMvc
@Import(TestLedgerConfig.class)
class DepositApiIT {

    @Autowired
    private MockMvc mvc;
    @Autowired
    private ObjectMapper json;
    @Autowired
    private DepositContractRepository contracts;
    @MockitoSpyBean
    private InMemoryLedger ledger;
    @MockitoBean
    private ClientApi clients;

    @BeforeEach
    void cleanState() {
        contracts.deleteAll();
        ledger.reset();
        ClientInfo client = new ClientInfo(1L, "Иванов Иван Иванович", "МП", "7654321", "3170590A077PB4");
        when(clients.get(1L)).thenReturn(client);
        when(clients.list()).thenReturn(List.of(client));
    }

```

```

}

@Test
@DisplayName("Справочники формы: виды депозита, валюты, клиенты, следующий номер договора")
void references() throws Exception {
    mvc.perform(get("/api/products")).andExpect(jsonPath("$", hasSize(5)))
        .andExpect(jsonPath("$[0].kindTitle").value("Отзывный, до востребования"));
    mvc.perform(get("/api/clients")).andExpect(jsonPath("$[0].fullName").value("Иванов Иван Иванович"));
    mvc.perform(get("/api/meta"))
        .andExpect(jsonPath("$.bankDate").value("2026-10-01"))
        .andExpect(jsonPath("$.currencies", hasSize(3)))
        .andExpect(jsonPath("$.nextNumber").value("Д-000001"));
}

@Test
@DisplayName("Договор заключается и появляется в списке, номер следующего договора увеличивается")
void createsContract() throws Exception {
    create(valid()).andExpect(status().isCreated())
        .andExpect(jsonPath("$.number").value("Д-000001"))
        .andExpect(jsonPath("$.clientName").value("Иванов Иван Иванович"))
        .andExpect(jsonPath("$.mainAccount").isString())
        .andExpect(jsonPath("$.interestAccount").isString());

    mvc.perform(get("/api/deposits")).andExpect(jsonPath("$", hasSize(1)));
    mvc.perform(get("/api/deposits/" + contracts.findAll().get(0).getId()))
        .andExpect(jsonPath("$.accounts", hasSize(2)))
        .andExpect(jsonPath("$.operations", hasSize(3)));
    mvc.perform(get("/api/meta")).andExpect(jsonPath("$.nextNumber").value("Д-000002"));
}

@ParameterizedTest(name = "без поля {0}")
@ValueSource(strings = {"number", "productId", "currency", "clientId", "amount", "rate", "termMonths", "startDate", "endDate"})
@DisplayName("Все поля договора обязательны")
void requiredFields(String field) throws Exception {
    Map<String, Object> request = valid();
    request.remove(field);

    create(request).andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields." + field).value("Обязательное поле"));
    assertThat(contracts.count()).isZero();
}

@ParameterizedTest(name = "{0} = {1}")
@CsvSource({
    "number, 000001",
    "number, D-1",
    "amount, 99.99",
    "amount, -5",
    "amount, 100.001",
    "rate, 8.50",
    "termMonths, 0",
    "termMonths, 13",
    "startDate, 2026-10-02",
    "endDate, 2027-01-02",
    "currency, USD",
    "productId, 77",
})
@DisplayName("Условия договора сверяются с депозитной программой и банковским днём")
void termsAreChecked(String field, String value) throws Exception {
    Map<String, Object> request = valid();
    request.put(field, value);

    String errorField = "currency".equals(field) ? "productId" : field;
    create(request).andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields." + errorField).exists());
    assertThat(contracts.count()).isZero();
    assertThat(ledger.journal()).isEmpty();
}

@Test
@DisplayName("Номер договора уникален")
void duplicateNumber() throws Exception {
    create(valid()).andExpect(status().isCreated());

    create(valid()).andExpect(status().isConflict())
        .andExpect(jsonPath("$.fields.number").exists());
    assertThat(contracts.count()).isEqualTo(1);
}

@Test
@DisplayName("Клиент должен существовать в модуле «Клиенты»")

```

```

void unknownClient() throws Exception {
    when(clients.get(99L)).thenThrow(HttpClientErrorException.create(HttpStatus.NOT_FOUND, "Not Found", null, null, null));
    Map<String, Object> request = valid();
    request.put("clientId", 99);

    create(request).andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields.clientId").value("Клиент не найден в модуле «Клиенты»"));
    assertThat(ledger.journal()).isEmpty();
}

@Test
@DisplayName("Сервис «Счета» недоступен – ответ 503, договор не создаётся")
void ledgerIsDown() throws Exception {
    doThrow(new ResourceAccessException("Connection refused")).when(ledger).bankDay();

    create(valid()).andExpect(status().isServiceUnavailable())
        .andExpect(jsonPath("$.code").value("SERVICE_UNAVAILABLE"));
    assertThat(contracts.count()).isZero();
}

@Test
@DisplayName("Внутренний REST закрытия дня возвращает протокол начислений")
void dayCloseEndpoint() throws Exception {
    create(valid()).andExpect(status().isCreated());

    mvc.perform(post("/api/internal/day-close").contentType(MediaType.APPLICATION_JSON).content("{\"date\": \"2026-10-11\"}"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$", hasSize(1)))
        .andExpect(jsonPath("$[0]").value("11.10.2026: Д-000001 – начислены проценты 1.92 BYN"));
}

@Test
@DisplayName("Безотзывный вклад нельзя отозвать через REST")
void irrevocableWithdrawIsRejected() throws Exception {
    Map<String, Object> request = valid();
    request.put("productId", 2);
    request.put("amount", 500);
    request.put("rate", 12.90);
    create(request).andExpect(status().isCreated());

    mvc.perform(post("/api/deposits/" + contracts.findAll().get(0).getId() + "/withdraw"))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.code").value("IRREVOCABLE"));
}

private Map<String, Object> valid() {
    Map<String, Object> request = new LinkedHashMap<>();
    request.put("number", "Д-000001");
    request.put("productId", 1);
    request.put("currency", "BYN");
    request.put("clientId", 1);
    request.put("amount", 1000);
    request.put("rate", 7.00);
    request.put("termMonths", 3);
    request.put("startDate", "2026-10-01");
    request.put("endDate", "2027-01-01");
    return request;
}

private ResultActions create(Map<String, Object> request) throws Exception {
    return mvc.perform(post("/api/deposits").contentType(MediaType.APPLICATION_JSON)
        .content(json.writeValueAsString(request)));
}
}

```

## Листинг А.104 – deposit-service/src/test/java/by/bsuir/bank/deposit/DepositFlowIT.java

```

package by.bsuir.bank.deposit;

import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.AccountInfo;
import by.bsuir.bank.common.ledger.InMemoryLedger;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.common.ledger.OperationInfo;
import by.bsuir.bank.deposit.dto.DepositRequest;
import by.bsuir.bank.deposit.dto.DepositView;
import by.bsuir.bank.deposit.repository.DepositContractRepository;

```

```

import by.bsuir.bank.deposit.service.DayCloseProcessor;
import by.bsuir.bank.deposit.service.DepositService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.test.context.bean.override.mockito.MockitoBean;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

import static by.bsuir.bank.common.ledger.InMemoryLedger.START;
import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.anyLong;
import static org.mockito.Mockito.when;

/**
 * Интеграционные тесты депозитного процесса: сервис, база H2 и главная книга в памяти.
 * Обороты счетов сверяются со схемой проводок «Депозитная программа» из задания.
 */
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@Import(TestLedgerConfig.class)
class DepositFlowIT {

    private static final long REVOCABLE_BYN = 1; // «Свободное накопление», 7 %, проценты ежемесячно
    private static final long IRREVOCABLE_BYN = 2; // «Online-решение», 12,9 %, проценты в конце срока

    @Autowired
    private DepositService deposits;
    @Autowired
    private DayCloseProcessor dayClose;
    @Autowired
    private DepositContractRepository contracts;
    @Autowired
    private InMemoryLedger ledger;
    @MockitoBean
    private ClientApi clients;

    private String cash;
    private String fund;

    @BeforeEach
    void cleanState() {
        contracts.deleteAll();
        ledger.reset();
        cash = ledger.systemAccount(LedgerApi.CASH, "BYN").number();
        fund = ledger.systemAccount(LedgerApi.FUND, "BYN").number();
        when(clients.get(anyLong())).thenAnswer(call ->
            new ClientInfo(call.getArgument(0), "Иванов Иван Иванович", "MP", "7654321", "3170590A077PB4"));
    }

    @Test
    @DisplayName("Заклучение договора: два счёта по плану счетов и три проводки приёма вклада")
    void openingContract() {
        DepositView deposit = open("Д-000001", REVOCABLE_BYN, "1000.00", 3);

        List<AccountInfo> accounts = ledger.accounts("Д-000001");
        assertThat(accounts).extracting(AccountInfo::chartCode).containsExactly("3404", "3470");
        assertThat(accounts).allSatisfy(a -> {
            assertThat(a.activity()).isEqualTo("PASSIVE");
            assertThat(a.name()).isEqualTo("Иванов Иван Иванович");
        });
        assertThat(ledger.operations("Д-000001")).extracting(OperationInfo::description).containsExactly(
            "Внесение денег в кассу", "Перевод денег с кассы на текущий счёт", "Использование денег банком");

        // касса: дебет +1000 и кредит -1000; счёт вклада: кредит +1000 и дебет -1000; фонд: кредит +1000
        assertTurnover(cash, "1000.00", "1000.00", "0.00");
        assertTurnover(deposit.mainAccount(), "1000.00", "1000.00", "0.00");
        assertThat(ledger.balance(fund)).isEqualByComparingTo(InMemoryLedger.CAPITAL.add(new BigDecimal("1000.00")));
        assertThat(deposit.status()).isEqualTo("ACTIVE");
        assertThat(deposit.endDate()).isEqualTo(START.plusMonths(3));
    }

    @Test
    @DisplayName("Отзывный вклад: проценты начисляются ежедневно и выплачиваются раз в месяц")
    void revocableDepositPaysInterestMonthly() {
        DepositView deposit = open("Д-000001", REVOCABLE_BYN, "1000.00", 3);
    }

```

```

        closeDays(30); // 31.10: начислено за 30 дней, выплаты ещё нет
        assertTurnover(deposit.interestAccount(), "0.00", "5.75", "5.75");
        assertThat(view(deposit).paid()).isEqualToComparingTo("0");

        closeDays(1); // 01.11: прошёл месяц – выплата 5,95
        assertTurnover(deposit.interestAccount(), "5.95", "5.95", "0.00");
        assertTurnover(cash, "1005.95", "1005.95", "0.00");
        assertThat(view(deposit).accrued()).isEqualToComparingTo("5.95");
        assertThat(view(deposit).paid()).isEqualToComparingTo("5.95");
        assertThat(view(deposit).status()).isEqualTo("ACTIVE");
    }

    @Test
    @DisplayName("Срочный вклад: проценты выплачиваются в конце срока вместе с возвратом суммы")
    void irrevocableDepositPaysAtMaturity() {
        DepositView deposit = open("Д-000002", IRREVOCABLE_BYN, "5000.00", 3);

        closeDays(91); // 31.12: до окончания срока один день
        assertThat(view(deposit).paid()).isEqualToComparingTo("0");
        assertThat(ledger.balance(deposit.interestAccount())).isEqualToComparingTo("160.81");

        List<String> events = closeDays(1); // 01.01.2027: окончание договора, 92 дня
        assertThat(events).anyMatch(e -> e.contains("срок договора истёк"));
        DepositView closed = view(deposit);
        assertThat(closed.status()).isEqualTo("CLOSED");
        assertThat(closed.closedOn()).isEqualTo(LocalDate.of(2027, 1, 1));
        assertThat(closed.interestAccount()).isEqualToComparingTo("162.58");
        assertThat(closed.paid()).isEqualToComparingTo("162.58");

        // все счета клиента обнулились, касса транзитом пропустила вклад и проценты, фонд потратил проценты
        assertTurnover(deposit.mainAccount(), "10000.00", "10000.00", "0.00");
        assertTurnover(deposit.interestAccount(), "162.58", "162.58", "0.00");
        assertTurnover(cash, "10162.58", "10162.58", "0.00");
        assertThat(ledger.balance(fund)).isEqualToComparingTo(InMemoryLedger.CAPITAL.subtract(new BigDecimal("162.58")));
        assertThat(ledger.operations("Д-000002").extracting(OperationInfo::description).contains(
            "Перевод процентов в кассу", "Вывод процентов из кассы",
            "Окончание депозита", "Перевод депозита в кассу", "Вывод денег из кассы");

        assertThat(closeDays(5)).isEmpty(); // закрытый договор больше не обрабатывается
    }

    @Test
    @DisplayName("Повторная обработка того же дня не начисляет проценты второй раз")
    void dayProcessingIsIdempotent() {
        DepositView deposit = open("Д-000001", REVOCABLE_BYN, "1000.00", 3);
        LocalDate date = START.plusDays(10);

        dayClose.process(date);
        BigDecimal accrued = ledger.credit(deposit.interestAccount());
        assertThat(dayClose.process(date)).isEmpty();

        assertThat(accrued).isEqualToComparingTo("1.92");
        assertThat(ledger.credit(deposit.interestAccount())).isEqualToComparingTo(accrued);
    }

    @Test
    @DisplayName("Пропущенные дни навёрстываются при следующей обработке")
    void skippedDaysAreCaughtUp() {
        DepositView deposit = open("Д-000001", REVOCABLE_BYN, "1000.00", 3);

        dayClose.process(START.plusDays(45)); // сервис «проспал» 44 дня

        assertThat(view(deposit).accrued()).isEqualToComparingTo("8.63");
        assertThat(view(deposit).paid()).isEqualToComparingTo("8.63");
    }

    @Test
    @DisplayName("Досрочный отзыв: отзывный вклад возвращается с процентами, безотзывный – отказ")
    void earlyWithdrawal() {
        DepositView revocable = open("Д-000001", REVOCABLE_BYN, "1000.00", 6);
        DepositView irrevocable = open("Д-000002", IRREVOCABLE_BYN, "5000.00", 6);
        closeDays(10);

        DepositView closed = deposits.withdraw(irrevocable.id());
        assertThat(closed.status()).isEqualTo("CLOSED");
        assertThat(closed.paid()).isEqualToComparingTo("1.92");
        assertThat(ledger.balance(revocable.mainAccount())).isEqualToComparingTo("0");
        assertThat(ledger.balance(revocable.interestAccount())).isEqualToComparingTo("0");

        assertThatThrownBy(() -> deposits.withdraw(irrevocable.id()))
            .isInstanceOfSatisfying(BankException.class, e -> assertThat(e.getCode()).isEqualTo("IRREVOCABLE"));
        assertThatThrownBy(() -> deposits.withdraw(revocable.id()))

```

```

        .isInstanceOfSatisfying(BankException.class, e -> assertThat(e.getCode()).isEqualTo("CONTRACT_CLOSED"));
        assertThat(view(irrevocable).status()).isEqualTo("ACTIVE");
    }

    @Test
    @DisplayName("При двух активных депозитных программах в отчёте не менее шести счетов")
    void twoProgramsGiveSixAccounts() {
        DepositView first = open("Д-000001", REVOCABLE_BYN, "1000.00", 3);
        DepositView second = open("Д-000002", IRREVOCABLE_BYN, "5000.00", 3);

        assertThat(List.of(cash, fund, first.mainAccount(), first.interestAccount(),
            second.mainAccount(), second.interestAccount())).doesNotHaveDuplicates().hasSize(6);
        assertThat(ledger.accounts("Д-000002")).extracting(AccountInfo::chartCode).containsExactly("3414", "3471");
    }

    private DepositView open(String number, long productId, String amount, int months) {
        return deposits.open(new DepositRequest(number, productId, "BYN", 1L, new BigDecimal(amount),
            productId == REVOCABLE_BYN ? new BigDecimal("7.00") : new BigDecimal("12.90"),
            months, START, START.plusMonths(months)));
    }

    /** Закрытие дней подряд – то, что делает сервис «Счета», вызывая участников. */
    private List<String> closeDays(int days) {
        List<String> events = new java.util.ArrayList<>();
        for (int i = 0; i < days; i++) {
            LocalDate next = ledger.bankDay().date().plusDays(1);
            events.addAll(dayClose.process(next));
            ledger.setDate(next);
        }
        return events;
    }

    private DepositView view(DepositView deposit) {
        return deposits.details(deposit.id()).contract();
    }

    private void assertTurnover(String account, String debit, String credit, String balance) {
        assertThat(ledger.debit(account)).as("дебет %s", account).isEqualByComparingTo(debit);
        assertThat(ledger.credit(account)).as("кредит %s", account).isEqualByComparingTo(credit);
        assertThat(ledger.balance(account)).as("сальдо %s", account).isEqualByComparingTo(balance);
    }
}

```

## Листинг А.105 – deposit-service/src/test/java/by/bsuir/bank/deposit/DepositLanguageIT.java

```

package by.bsuir.bank.deposit;

import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.InMemoryLedger;
import by.bsuir.bank.deposit.repository.DepositContractRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.ResultActions;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/** Интеграционные тесты локализации сервиса депозитов: справочник программ, проверки договора, протокол дня. */
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@AutoConfigureMockMvc
@Import(TestLedgerConfig.class)
class DepositLanguageIT {

    private static final String CONTRACT = """
        {"number": "%s", "productId": 1, "currency": "BYN", "clientId": 1, "amount": %s, "rate": 7.00,
        "termMonths": 3, "startDate": "2026-10-01", "endDate": "2027-01-01"}""";
    """

```

```

@Autowired
private MockMvc mvc;
@Autowired
private DepositContractRepository contracts;
@Autowired
private InMemoryLedger ledger;
@MockitoBean
private ClientApi clients;

@BeforeEach
void cleanState() {
    contracts.deleteAll();
    ledger.reset();
    when(clients.get(1L)).thenReturn(new ClientInfo(1L, "Иванов Иван Иванович", "MP", "7654321", "3170590A077PB4"));
}

@Test
@DisplayName("Виды депозита: название, описание и вид вклада на языке запроса")
void products() throws Exception {
    mvc.perform(get("/api/products").header(HttpHeaders.ACCEPT_LANGUAGE, "en"))
        .andExpect(jsonPath("$.name").value("Free Savings"))
        .andExpect(jsonPath("$.kindTitle").value("Revocable, on demand"))
        .andExpect(jsonPath("$.description").value("Revocable demand deposit, interest is paid monthly"))
        .andExpect(jsonPath("$.nameEn").doesNotExist());
    mvc.perform(get("/api/products").header(HttpHeaders.ACCEPT_LANGUAGE, "be"))
        .andExpect(jsonPath("$.name").value("Online-рашэньне"))
        .andExpect(jsonPath("$.kindTitle").value("Тэрміновы безадзўны"));
}

@Test
@DisplayName("Ошибки формы договора на английском и белорусском")
void validation() throws Exception {
    create("en", "000001", "50")
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.message").value("Please check the highlighted fields"))
        .andExpect(jsonPath("$.fields.number").value("Contract number format: Д-000001"));
    create("en", "Д-000001", "50")
        .andExpect(jsonPath("$.fields.amount").value("Minimum deposit amount: 100.00 BYN"));
    create("be", "Д-000001", "50")
        .andExpect(jsonPath("$.fields.amount").value("Мінімальная сума ўкладу – 100.00 BYN"));
}

@Test
@DisplayName("Протокол закрытия дня и проводки договора читаются на языке запроса")
void eventsAndPostings() throws Exception {
    create("ru", "Д-000001", "1000").andExpect(status().isCreated());
    long id = contracts.findAll().get(0).getId();

    mvc.perform(post("/api/internal/day-close").header(HttpHeaders.ACCEPT_LANGUAGE, "en")
        .contentType(MediaType.APPLICATION_JSON).content("{\"date\": \"2026-10-11\"}"))
        .andExpect(jsonPath("$.value").value("11.10.2026: Д-000001 – interest accrued 1.92 BYN"));

    mvc.perform(get("/api/deposits/" + id).header(HttpHeaders.ACCEPT_LANGUAGE, "en"))
        .andExpect(jsonPath("$.contract.productName").value("Free Savings"))
        .andExpect(jsonPath("$.operations[0].description").value("Cash paid in at the cash desk"))
        .andExpect(jsonPath("$.operations[3].description").value("Deposit interest accrued"));
    mvc.perform(get("/api/deposits/" + id).header(HttpHeaders.ACCEPT_LANGUAGE, "be"))
        .andExpect(jsonPath("$.contract.kindTitle").value("Адзўны, да запатрабавання"))
        .andExpect(jsonPath("$.operations[0].description").value("Унясенне грошай у касу"));
}

private ResultActions create(String language, String number, String amount) throws Exception {
    return mvc.perform(post("/api/deposits").header(HttpHeaders.ACCEPT_LANGUAGE, language)
        .contentType(MediaType.APPLICATION_JSON).content(CONTRACT.formatted(number, amount)));
}
}

```

## Листинг А.106 –

### deposit-service/src/test/java/by/bsuir/bank/deposit/InterestCalculatorTest.java

```

package by.bsuir.bank.deposit;

import by.bsuir.bank.deposit.service.InterestCalculator;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import java.math.BigDecimal;
import java.time.LocalDate;

```

```

import static org.assertj.core.api.Assertions.assertThat;

/** Модульные тесты расчёта процентов по вкладу. */
class InterestCalculatorTest {

    private static final LocalDate START = LocalDate.of(2026, 10, 1);

    @ParameterizedTest(name = "{0} под {1} % за {2} дн. = {3}")
    @CsvSource({
        "1000.00, 7.00, 31, 5.95",
        "1000.00, 7.00, 1, 0.19",
        "5000.00, 12.90, 365, 645.00",
        "5000.00, 12.90, 396, 699.78",
        "100.00, 1.50, 0, 0.00",
    })
    void simpleInterestOnActualDays(String amount, String rate, int days, String expected) {
        BigDecimal accrued = InterestCalculator.accrued(new BigDecimal(amount), new BigDecimal(rate), START, START.plusDays(days));

        assertThat(accrued).isEqualToByComparingTo(expected);
    }

    @Test
    void nothingIsAccruedBeforeStart() {
        assertThat(InterestCalculator.accrued(new BigDecimal("1000"), new BigDecimal("7"), START, START.minusDays(5)))
            .isEqualToByComparingTo("0");
    }

    @Test
    void dailyIncrementsAddUpToTotalWithoutRoundingLoss() {
        BigDecimal amount = new BigDecimal("1234.56");
        BigDecimal rate = new BigDecimal("7.00");
        BigDecimal posted = BigDecimal.ZERO;
        for (int day = 1; day <= 92; day++) {
            BigDecimal target = InterestCalculator.accrued(amount, rate, START, START.plusDays(day));
            assertThat(target).isGreaterThanOrEqualTo(posted);
            posted = target;
        }
        assertThat(posted).isEqualToByComparingTo(InterestCalculator.accrued(amount, rate, START, START.plusDays(92)));
        assertThat(posted).isEqualToByComparingTo("21.78");
    }

    @ParameterizedTest(name = "с {0} по {1} – полных месяцев: {2}")
    @CsvSource({
        "2026-10-01, 2026-10-31, 0",
        "2026-10-01, 2026-11-01, 1",
        "2026-10-01, 2027-01-01, 3",
        "2026-01-31, 2026-02-27, 0",
        "2026-01-31, 2026-02-28, 1",
        "2026-10-01, 2026-10-01, 0",
    })
    void fullMonthsFollowCalendar(LocalDate start, LocalDate upTo, int months) {
        assertThat(InterestCalculator.fullMonths(start, upTo)).isEqualTo(months);
    }
}

```

## Листинг А.107 –

### deposit-service/src/test/java/by/bsuir/bank/deposit/TestLedgerConfig.java

```

package by.bsuir.bank.deposit;

import by.bsuir.bank.common.ledger.InMemoryLedger;
import org.springframework.boot.test.context.TestConfiguration;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Primary;

/** В тестах вместо REST-клиента сервиса «Счета» подставляется главная книга в памяти. */
@TestConfiguration
public class TestLedgerConfig {

    @Bean
    @Primary
    public InMemoryLedger inMemoryLedger() {
        return new InMemoryLedger();
    }
}

```